Right Order, Wrong Size: A Verified Reproduction of the Robotic World Model and the Uncertainty It Reports

Anonymous authors
Paper under double-blind review

Abstract

We rebuild the dynamics model of the *Robotic World Model* (RWM; arXiv:2501.10100v1) and its uncertainty-aware follow-up (arXiv:2504.16680v1) on CPU, matching the released implementation's outputs, losses and gradients exactly before training. The base paper's central training claim reproduces under a rule committed in advance, run on seed 1 of each arm; 3 seeds extend it: training on the model's own rollouts beats teacher forcing 4.61× at 368 steps and 2.58× at 100, though on short windows teacher forcing leads at one step. At 368 steps RWM beats MLP, RSSM and transformer baselines built to our reading of the original and trained with RWM's settings, though each does worse than predicting no change. On accuracy alone, two shorter histories and both longer training forecasts beat the original's setting at our budget (one reading unresolved); trained longer, it passes each on some reading (post hoc): the ranking depends on training budget. Its trade-off with training time is untested. These rest on 0.133% of the reference's world-model data, one robot, gait and terrain, and 4 independent held-out trajectories. The follow-up's uncertainty gets the order right and the size wrong. Ensemble disagreement, the method's reward penalty, correlates +0.605 with realised error (+0.419 with rollout and depth held fixed), yet on the checkpoint's training data is 8.3× smaller than that error at one step and 33.4× at the method's 100-step horizon. A free signal, the model's predicted step size, ranks error nearly as well (+0.470; margin unresolved). The members share 89.15% of their parameters; at 100 steps, independent models are 2.03× better calibrated than our shared-trunk ones, still 5.2× overconfident. The implemented loss provably drives the per-member σ the method discards to zero, as data with known noise confirm. A per-horizon rescaling brings the released checkpoint's coverage within 10 points of nominal on its training episodes (no cell resolvable), and only 17 of 36 disagreement cells on episodes our ensembles never saw: a recipe to refit, not a demonstrated fix. The released evaluation pairs each prediction with the previous step's action; over all ten episodes this raises the checkpoint's short-horizon relative-L1 error, and from 100 steps the change is unresolved. We train no policy, so we bound what the uncertainty reports, not what its miscalibration costs.

1 Introduction

A world model that reports its own uncertainty is more useful than one that does not, and the uncertainty-aware Robotic World Model reports one. This paper asks what that number means, and the title gives the answer: it gets the order of the model's errors right and their size wrong.

We set out to reproduce the base paper: rebuild the proprioceptive dynamics model from scratch, check it against the released implementation, and test the central training claim, which holds at long horizons. The rebuild made a second question cheap to ask: *is the predicted σ calibrated?* Neither of the two the checkpoint emits is. On data it trained on, the per-member σ is too small by 1,827× at h = 1 to 20,669× at h = 368, by construction (§6.3), and the ensemble disagreement the method uses by 8.3× to 34.4×; the disagreement still ranks realised error, the use the method makes of it (§6.6).

This is a reproduction in the stronger sense: the contribution is not that the numbers came out the same, but what re-measuring the method reveals about where it is robust and where it is not. Three things distinguish it from a re-run of the authors' code. **We rebuilt rather than imported**, and matched the rebuild to the reference before any training (Appendix A), so a later discrepancy belongs to the method, not to our wiring. **Decision rules were committed to git before the data**, with timestamps a reader can check (§8, Figure 1); several returned verdicts against the original or could not settle the question, and we report each (Appendix E). **We keep our withdrawn findings in the record.** The ledger keeps 21 superseded entries, each beside the evidence that withdrew it: eight claims withdrawn on evidence, six framings withdrawn, and the rest early hypotheses closed as housekeeping (§8 and the supplementary `docs/BUILD_CHECKS.md`; every pre-registered rule, with its lead time and verdict, is in Appendix E). §9 gives the lessons in a form a practitioner can use without reading the rest.

Contributions.

- **The uncertainty gets the order right and the size wrong, in the first calibration measurement we are aware of for this released checkpoint** (Lu et al. (2022) measure this family of penalties on models they train themselves; §2). Ensemble disagreement ranks realised error, correlating +0.419 with it with rollout and forecast depth held fixed, yet on data the checkpoint trained on it is 8.3× smaller than that error at h = 1 and 33.4× at h = 100 (§6.2, §6.6). It beats 1 of the 2 free baselines added here; the model's own predicted step size ranks error at +0.4697 against its +0.6053, a margin 25 independent trajectories would resolve if it is as large as observed, against the 20 here (§6.6, §11).

- **The base paper's central training claim reproduces, and reverses at one step.** A rule committed before the runs, run on seed 1 of each arm, found autoregressive training ahead by 4.43× at h = 368; 3 seeds at 10,000 iterations extend it: the factor is 4.61×, and 2.58× at h = 100, on the held-out pair's 4 independent 400-step trajectories (§5). At one step a second pre-registered rule, on 60 independent 33-row units, finds a gap of -0.0194 [-0.0310, -0.0093] in favour of **teacher forcing** (§5).

- **Two more of the base paper's claims, tested under rules committed before the runs.** RWM is ahead at h = 368 of MLP, RSSM and transformer baselines built to our reading of its specification and trained with RWM's settings, whether teacher-forced as the original trains them or autoregressively, on the held-out pair's 4 independent trajectories at 2,500 iterations, though every baseline is worse there than predicting no change, and neither reading our RSSM's forecast from its prior's expected or sampled latent nor retraining it with two other settings on one seed removes its open-loop collapse (rule X1, exploratory: **NOT RESCUED BY THE SETTINGS TRIED**; §5.3). On accuracy alone at our budget, four of the eight one-factor neighbours of the original's (M, N) = (32, 8) beat it, two shorter histories at under half its cost per iteration and both longer training forecasts at more (the rule's verdict holds on every reading it reports but held-out nRMSE at h = 1, which is unresolved), but trained longer the centre passes each of them on at least one reading (post hoc), so the ranking depends on the training budget; the original chose the centre as a trade-off with training time, which its own printed hours are consistent with and we do not test (§5.2).

- **The $\sigma = 0$ optimum of the implemented objective.** The implemented state loss is minimised at $\sigma = 0$, so the per-member σ the method discards collapses by construction: derived, and demonstrated against known noise (§6.3).

- **Trunk-sharing, tested.** The five members share one trunk, one recurrent state and 89.15% of each member's parameters, so their spread can express only uncertainty the trunk already carries (§6.4). Under a rule committed before the runs, on the held-out pair's 4 independent trajectories, 5 independently initialised full models are 2.03× better calibrated than the shared-trunk arms, against a pre-registered minimum detectable effect of 1.45×, and still 5.2× overconfident at h = 100; they also differ in capacity and data order, so this bounds the sharing effect rather than isolating it (§6.8).

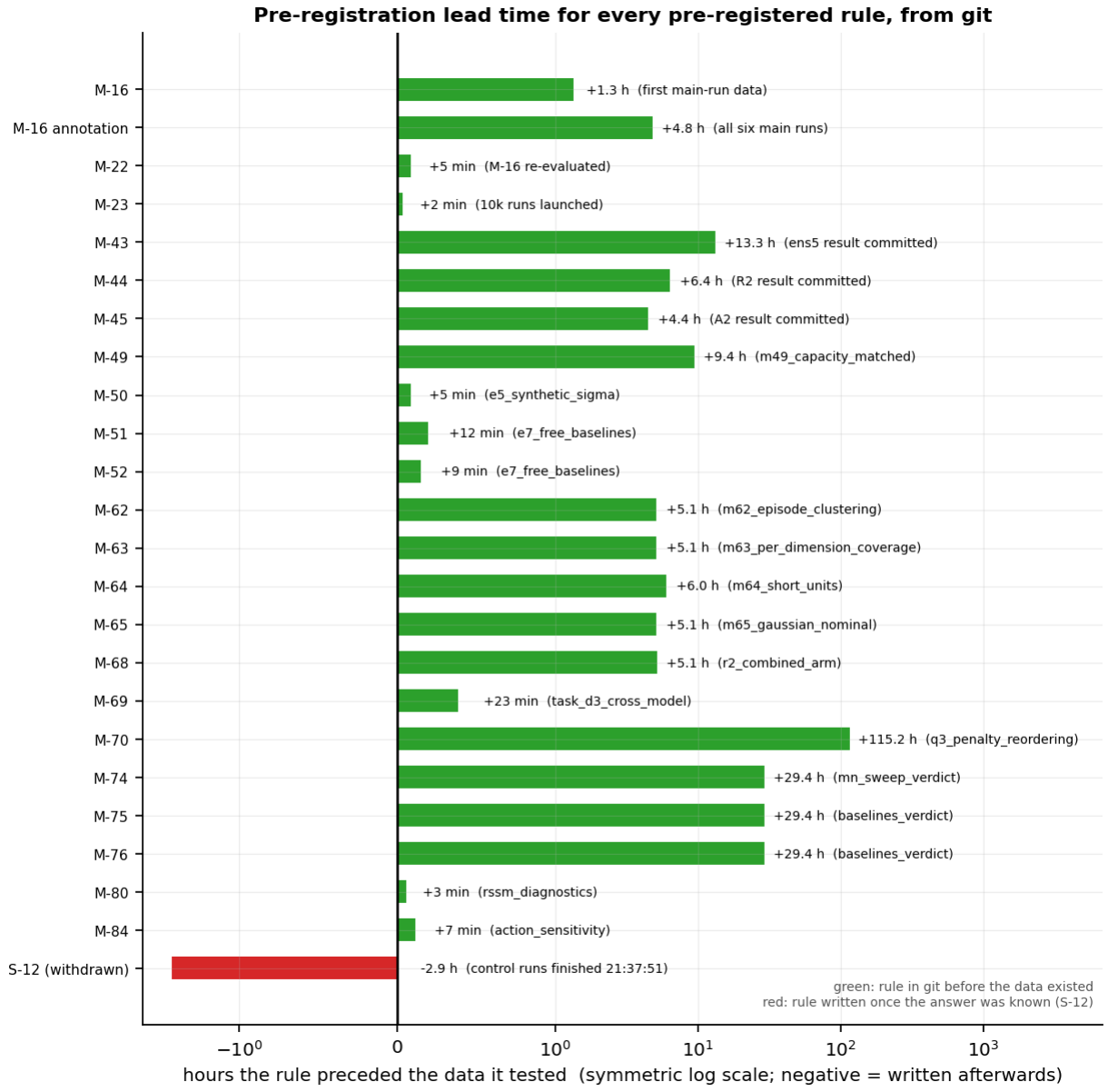


Figure 1: Pre-registration lead time for every pre-registered rule (Appendix E), with M-16's annotation beside its rule, from git commit timestamps on a symmetric-logarithmic axis. Positive is a rule committed before the data that tested it existed; negative is a rule written afterwards. The one negative bar is the Task 3 duplication rule (S-12), retracted as a pre-registration in this paper.

- **Per-horizon recalibration, with mixed evidence.** One multiplier per horizon, fitted on one episode and scored on the other, brings every released-checkpoint coverage estimate near nominal where a global one does not, though no single cell is resolvable and the cells are unseen only by the multiplier, since the checkpoint trained on both episodes (§6.7); on Arm A, which never saw them, its own multipliers manage 17 of 36 disagreement cells.

- **The released evaluation is misaligned by one step; over all ten episodes it raises the checkpoint's relative-L1 error up to h = 32, and from h = 100 the change is not resolved.** Evaluation feeds the action from $t-1$ where training pairs states and actions index-for-index, and

shifting its action index by one step fixes it. On all ten episodes' 20 independent trajectories, all of them training data for this checkpoint, the stale action raises its relative-L1 error by 55.0% [26.3, 92.7] at h = 1 and 37.4% [21.3, 65.4] at h = 32, and changes it by 8.2% [-10.4, 24.9] at h = 100 and -4.6% [-13.4, 3.2] at h = 368; on the held-out pair's 4 it raises it at every horizon reported, by 7.9% [3.1, 13.0] at h = 368 (§7.2).

- **A from-scratch reimplementation verified at the gradient level.** Outputs match the released module bitwise, and losses and gradients match to 0.000e+00 across 7 loss terms and 106 parameter tensors, before any training (Appendix A).

2 Related work

This paper asks whether an ensemble-disagreement penalty is calibrated, and separates its use as a *ranking* from its use as a *scale*. Both questions have a literature, and one paper asked almost exactly ours four years earlier; below we say which parts of what follows are new.

The direct precedent. Lu, Ball, Parker-Holder, Osborne and Roberts (*Revisiting Design Choices in Offline Model-Based Reinforcement Learning*, ICLR 2022) compare uncertainty heuristics in offline model-based RL under protocols built, in their words, to "capture the specific covariate shift induced by model-based RL", in order to assess calibration. They report Spearman rank and Pearson bivariate correlation against true model error **separately**, and observe that "despite the similar rank correlations ρ, the bivariate correlations r can vary considerably": a configuration can keep the ordering while the penalty's size relative to the error changes. That is our ranking-versus-scale distinction, on the same family of penalties. They also find the ensemble standard deviation, the exact quantity `system_dynamics.py:126` computes and `envs/base.py:166` applies, to track model error *better* than the penalties of MOPO and MOReL, being "strikingly similar" to the latter but better behaved; §6.6 finds the same quantity a usable ranking signal.

What is new here is four things. Lu et al. measure models they train themselves, on simulated benchmarks, and their statistics describe ordering and shape, not coverage. We measure a **released checkpoint** from a pipeline its authors deployed on hardware, where reading the penalty as an interval has consequences; we measure **coverage** against a nominal; we verify our reimplementation against the reference at the gradient level before any training (Appendix A); and §6.3 derives the $\sigma = 0$ optimum for the objective the released code *substitutes*, squared error on a reparameterised sample, rather than for the likelihood the parameterisation was built around.

Where the parameterisation comes from. The clamp of the bounded log-σ head whose optimum §6.3 shows is $\sigma = 0$ is inherited, line for line, from the probabilistic ensembles of Chua, Calandra, McAllister and Levine (PETS, NeurIPS 2018), but the objective is not: the released code replaces PETS's likelihood with squared error on a sampled prediction and ties the upper bound to the floor, so **a descendant of this lineage that made the same substitution, and left nothing pushing its variance floor back up, would inherit the same optimum**, a hypothesis about mechanism untested in any other descendant (§11). Of 13 public repositories the survey looked at, 10 carry the construction, the ones its protocol counts as examined, and 1 of those trains it against a sampled squared error, only in an optional mode, so on this sample of convenience the substitution is rare (Appendix I).

The method's family. MOPO (Yu, Thomas, Yu, Ermon, Zou, Levine, Finn and Ma, NeurIPS 2020) penalises the reward by an ensemble uncertainty estimate to solve a pessimistic MDP; MOReL (Kidambi, Rajeswaran, Netrapalli and Joachims, NeurIPS 2020) builds an unknown-state detector from pairwise ensemble disagreement instead; and both branch short model rollouts from real states in the manner of MBPO (Janner, Fu, Zhang and Levine, NeurIPS 2019). The follow-up we reproduce adapts MOPO's penalty into MBPO's loop, which is what "MOPO-PPO" names, and its 100-step imagination horizon sits inside MBPO's trade between rollout length and model error (§6.2).

What "ensemble" is supposed to mean. Deep ensembles (Lakshminarayanan, Pritzel and Blundell, NeurIPS 2017) are several networks trained from *different random initialisations and different data orderings*, and their spread is the uncertainty estimate. The released checkpoint's five members share one GRU trunk,

one recurrent hidden state, and 89.15% of each member's state-prediction parameters, so whatever their spread measures, it is not what a deep ensemble measures (§6.4).

Three sources of uncertainty, and the one nobody here estimates. Abbas, Sokota, Talvitie and White (ICML 2020) separate predictive uncertainty in model-based RL into aleatoric noise, parameter uncertainty, and **model inadequacy**, and observe that selective-planning work attends almost entirely to the second. The checkpoint we measure emits an aleatoric term and a parameter-uncertainty term, discards the first before use (§6.1), and has no estimate of the third, which is precisely the component that compounds with rollout depth: the shape §6.5 reports.

Why miscalibration is the expected finding. Modern networks are systematically miscalibrated (Guo, Pleiss, Sun and Weinberger, ICML 2017), and calibration degrades further under covariate shift, worsening with distance from the training distribution (Ovadia, Fertig, Ren, Nado, Sculley, Nowozin, Dillon, Lakshminarayanan and Snoek, NeurIPS 2019). An autoregressive rollout manufactures its own covariate shift, increasing with depth, so horizon-dependent calibration failure is what one should expect; our contribution on this axis is the *magnitude* and the *mechanism*, not the direction. §6.7's per-horizon multiplier is a coarse instance of calibrated regression (Kuleshov, Fenner and Ermon, ICML 2018), a post-hoc map fitted on one episode and scored on another, applied to a horizon index; we do not present it as a new idea.

The closest published work to our one constructive result. Malik, Kuleshov, Song, Nemer, Seymour and Ermon (ICML 2019) recalibrate a dynamics model's uncertainty inside model-based RL, and argue that "good uncertainties must be calibrated" rather than merely well ranked, the distinction §6.2 and §6.6 draw. §6.7 is a horizon-indexed instance: a single global multiplier **fails**, its fitted value varying across horizons, where a per-horizon one puts every released-checkpoint estimate within its tolerance band, though those cells are unseen by the multiplier but not by the model and no single cell is resolvable (§6.7). An open-loop rollout's error accumulates with depth, so a horizon-blind recalibration cannot follow it.

§6.4's mechanism is known, and we say so. That heads sharing a trunk under-report disagreement relative to independently initialised networks is established: Lee, Purushwalkam, Cogswell, Crandall and Batra (arXiv:1511.06314) treat ensemble diversity as something to engineer rather than assume; Fort, Hu and Lakshminarayanan (arXiv:1912.02757) show that independent initialisation buys a decorrelation subspace methods do not match; and BatchEnsemble (Wen, Tran and Ba, ICLR 2020) and MIMO (Havasi, Jenatton, Fort, Liu, Snoek, Lakshminarayanan, Dai and Tran, ICLR 2021) share deliberately and state what they trade away. **What is ours is finding it in a released robotics checkpoint, with the sharing quantified at 89.15% of each member and its cost bounded by the 2.03× an independent ensemble gains (§6.8).** The problem is not sharing; it is sharing and then reading the spread as though the members were independent.

And the objective in §6.3 has a neighbour. Seitzer, Tavakoli, Antic and Martius (ICLR 2022) identify failure modes of heteroscedastic σ heads trained by maximising log-likelihood. The released model's state loss is not a log-likelihood at all (a *sample* enters a squared error), so the failure §6.3 derives is more basic than theirs and does not depend on the optimiser. §6.3 derives it and demonstrates it against known noise.

3 Setup

Data. The released dataset is 10,000 rows of ANYmal D proprioceptive state and policy actions at 50 Hz: ten concatenated 20-second episodes, with a termination column that is identically zero, so nothing in the file marks the boundaries.

The segments are not all the same length. The first episode is 999 rows and the other nine are 1,000 each, with a reset at rows 999, 1,999 … 9,999 and 1 orphan row at the end of the file that begins an eleventh episode and ends immediately: $999 + 9 \times 1{,}000 + 1$. We recover the boundaries from the data: at every reset row the twelve joint velocities, the four HAA joint positions and all twelve actions are exactly zero, and no other row has that fingerprint. Ten equal segments of 1,000 would give one fewer crossing window and one more usable one, so the counts below differ by one from that reading; `results/step0_regimes.json` re-derives the crossing count from the segment lengths alone, and the two agree.

A window is 40 rows, 32 of history and 8 of forecast, and the reference window builder marks all 9,961 windows valid, including 352 that splice one episode's end onto the next one's start. The usable, episode-respecting count is 9,609: 10,000 rows, less 39 that cannot start a full window, less 352 that cross a boundary. The contamination rate is 3.53%.

Model. A GRU-based ensemble predicting the next proprioceptive state, with a mean head and a bounded log-σ head, plus auxiliary heads for contact and termination. The paper describes two loss terms; the implementation has 7. Before any training, our rebuild's outputs match the released module's bitwise, and its losses and gradients match to 0.000e+00 across those 7 terms and 106 parameter tensors (Appendix A).

Evaluation. Two arenas, kept separate throughout: *out-of-sample*, the two episodes withheld from training, and *in-sample*, the eight used for it. The released evaluation draws its trajectories from training data, and the original does not distinguish the two. **The released checkpoint trained on all ten episodes, so it has no held-out arena in this dataset**, and every figure for it is in-sample; later sections refer back to this as the in-sample caveat of §3. More data cannot be generated from either repository this reproduction pins (Appendix C).

Effective sample size. Trajectory count is not sample size. Two 400-step trajectories whose spans overlap are not independent evidence, and the out-of-sample arena contains only 4 mutually non-overlapping 400-step trajectories. A bootstrap over whole trajectories at that size has 256 distinct resamples, so its intervals are quantised at that resolution. Every long-horizon verdict in this paper is computed with a bootstrap over independent trajectories, every table reports that count, and §8 reports both resampling units where they differ. Later sections refer back to this as the n = 4 caveat of §3. One trajectory can carry much of a long-horizon effect: a one-step shift of the action moves the released checkpoint's 368-step relative-L1 error on single trajectories of its own training episodes by anywhere from -47.4% to +35.2% (§7.2), which illustrates why 4 trajectories bound every long-horizon claim.

3.1 Metrics

Each metric is stated as implemented, with the `file:line` of its implementation, because a reader who cannot see the denominator cannot check the headline.

Relative-L1 is the reference's own metric, reproduced in behaviour (`model_training.py:203`) so that our numbers are comparable to the upstream's printed one. On config-normalised states, per forecast step,

$$r_t \;=\; \frac{\sum_{d=1}^{45} \left|\hat{s}_{t,d} - s_{t,d}\right|}{\sum_{d=1}^{45} \left|s_{t,d}\right|}$$

and the reported figure is the flat mean over trajectories and steps,

$$e \;=\; \frac{1}{B\,(T - t_0)} \sum_{b=1}^{B} \sum_{t=t_0+1}^{T} r_t^{(b)}$$

with $t_0 =$ `history_horizon` $= 32$: the first 32 steps are teacher-forced and excluded. The denominator is recomputed at every step as a 45-term sum in normalised space, so it can pass through zero, which is why this metric goes non-finite on low-dimensional state groups and why a second one exists.

Normalised RMSE fixes the denominator once, over the training episodes only:

$$\mathrm{nRMSE}_t \;=\; \frac{\sqrt{\dfrac{1}{45}\sum_{d=1}^{45} \mathrm{MSE}_{t,d}}}{\dfrac{1}{45}\sum_{d=1}^{45} \sigma_d^{\mathrm{tr}}} \quad \text{with} \quad \mathrm{MSE}_{t,d} = \frac{1}{B}\sum_{b=1}^{B} \left(\hat{s}_{t,d}^{(b)} - s_{t,d}^{(b)}\right)^2$$

where the scale constant is

$$\sigma_d^{\mathrm{tr}} \;=\; \mathrm{sd}\big(\{\,\tilde{s}_{i,d} \;:\; \mathrm{episode}(i) \in \mathcal{E}_{\mathrm{train}}\,\}\big)$$

computed once, stored in `results/step4_0a_results.json`, and never recomputed per step or derived from held-out data. A value of 1.0 means no better than predicting the training mean. **The aggregation is form 1**: pool the per-dimension mean squared errors, then divide, a ratio of means. A mean of per-dimension ratios gives whichever dimension has the smallest scale unbounded leverage; Appendix G gives both forms and the comparison the second one inverts.

Coverage at ±kσ is the fraction of scalar (trajectory, forecast step, state dimension) triples whose absolute realised error falls within k times the σ predicted for that same triple:

$$\mathrm{cov}_{\pm k\sigma}(h) \;=\; \frac{1}{B\,h\,45}\sum_{b=1}^{B}\sum_{t=t_0+1}^{t_0+h}\sum_{d=1}^{45}\mathbf{1}\left[\;\frac{\big|\hat{s}_{t,d}^{(b)} - s_{t,d}^{(b)}\big|}{\sigma_{t,d}^{(b)}} \;\le\; k\;\right]$$

It is pooled over all three axes with equal weight per triple. It is **cumulative** over steps 1..h: coverage "at h" averages the whole rollout up to h and is not the value at step h, and every horizon-indexed quantity in this paper follows the same convention. Because it is built from an *absolute* error, $z \le k$ is the two-sided event, so the calibrated targets are $\mathrm{erf}(k/\sqrt{2})$: **68.27%** at $\pm1\sigma$ and **95.45%** at $\pm2\sigma$. **That nominal is checked rather than assumed.** Rescaling each model's σ by the single constant that makes mean|error| / mean σ equal a calibrated Gaussian's 0.7979 lands coverage at or above 68.27% in every one of 48 model × horizon cells, with 0 below it (`M-65`), so the shortfalls reported below are a property of the scale and not of the tail.

The overconfidence factor is how many times larger the typical realised error is than the typical predicted σ:

$$\rho(h) \;=\; \frac{\mathrm{mean}_{b,t\le h,d}\big|\hat{s}_{t,d}^{(b)} - s_{t,d}^{(b)}\big|}{\mathrm{mean}_{b,t\le h,d}\;\;\sigma_{t,d}^{(b)}}$$

It too is a **ratio of means**, because a mean of ratios is unbounded whenever a single σ approaches zero, which is exactly the regime §6.3 puts these models in. $\rho = 1$ is *not* calibration: a calibrated Gaussian has mean|error| / $\sigma = \sqrt{2/\pi} = 0.7979$. So ρ is reported as a magnitude of miscalibration and coverage as the calibrated reading, and both appear everywhere.

Which metric each headline uses. The A/B training claim (§5) is relative-L1, because the claim is about reproducing the upstream's comparison and that is the upstream's metric. The calibration claims (§6.2) are the overconfidence factor and coverage, because neither error metric involves σ. The ranking claims (§6.6) are Pearson correlations between the applied scalar penalty and total absolute error, because a ranking claim is about order rather than scale. Every headline number in the abstract names its metric. §7.2's alignment defect is given in both metrics, over all ten episodes and on the held-out pair. Over all ten episodes the two metrics agree that the stale action raises error at h = 8 and 32 and that from h = 100 the change is not resolved, and they differ at h = 1, where relative-L1 resolves the rise and nRMSE does not.

Horizons. Curves are reported at $h \in \{1, 8, 32, 100, 128, 368\}$. Two of those are load-bearing and the rest are landmarks. **h = 100** is the method's own imagination rollout length, the horizon over which the uncertainty-penalised policy loop actually runs this model (arXiv:2504.16680 Table S9 in v1 and Table S11 in v3, with the same value). **h = 368** is the upstream's open-loop diagnostic length: `len_eval_trajectory` = 400 minus the 32-step teacher-forced prefix, the curve the follow-up plots as its uncertainty figure. It is 3.68× the method's own rollout length and is not a deployment horizon. Every table keeps both: h = 368 makes our numbers comparable to the original's *figure*, and h = 100 to its *method*.

3.2 What each claim rests on

Every headline claim in this paper is measured on the out-of-sample or in-sample arena above, or on all ten episodes together, at a stated number of independent units (400-step trajectories unless the row says

otherwise), and at a stated checkpoint: the released one, or ours at a stated number of training iterations. Appendix D's second table, "What each tested claim rests on", gives each claim's arena, sample size, checkpoint and verdict. It is generated from the artifacts each claim is computed from, so no arena label, sample size or checkpoint in it is typed by hand.

4 What the original papers claim, and which claims we test

A reproduction that does not say what it left alone invites the reader to assume it tested everything. It did not.

We tested six claims and left six untested. The six are the base paper's autoregressive-versus-teacher-forcing comparison and its claim that teacher forcing generalises poorly (§5), its configuration claim (§5.2) and its claim against MLP, RSSM and transformer baselines (§5.3), and the follow-up's two claims about what its uncertainty outputs report (§6). **Every one of the six we did not test needs a simulator or hardware we do not have**: zero-shot transfer, the sample-efficiency result, the comparisons against SHAC and Dreamer, generality across robot morphologies, offline MBRL on real robots and the core claim that penalising rewards by disagreement improves the learned policy. **Five of them are claims about policy learning or hardware** — every one but generality across robot morphologies, which needs a simulator and recorded data from other robots but is a claim about the model rather than about a policy. This work trains no policy and runs on two CPU cores. The counts and lists are generated from the classification tags in the verdict column of Appendix D's first table, and they replace a withdrawn claim that every untested claim concerns policy learning or hardware (`S-17`). §11 states what the untested claims bound, and Appendix C what testing them would take.

For four of the six claims we tested, the original reports no quantitative figure. Each is asserted qualitatively and shown in a plot, with no number in text, caption or table; §6.6's coefficient is the first figure attached to the follow-up's "strong correlation" between disagreement and error. The exceptions are the configuration claim and the teacher-forcing claim, whose values one heatmap prints in every cell (§5.2). For the teacher-forced (32, 1) it prints 3.99 against 0.47 at the centre, 8.5× worse, on evaluation data and at a horizon it does not state; our sweep's (32, 1) is 6.35× worse on relative-L1 at h = 368 and 2,500 iterations. Our 4.61×, at 10,000, uses a different definition of teacher forcing (`docs/presubmission/ORIGINAL_SPECS.md` §2); §5 relates the two. Where a magnitude is legible only from a plotted curve we say so rather than estimating it from the axis.

Appendix D's first table gives it in full, claim by claim, with what the original states, where it states it, and our verdict.

5 The base paper's central claim reproduces

Claim under test. Training the dynamics model on its own autoregressive rollouts beats training it with teacher forcing, at long forecast horizons.

Rule, committed in advance (rule M-23, Appendix E; commit `C016`, in the submitted paper an anonymised commit label; see Data and code), naming conditions rather than outcomes. Three conditions, all required: the out-of-sample gap at h = 368 excludes zero under a bootstrap over independent trajectories; the sign is consistent across episodes; and the effect survives at 10,000 iterations rather than only at the paper's 2,500. The rule was run on seed 1 of each arm: autoregressive 0.3509 against teacher forcing 1.5540 at h = 368, 4.43×, gap interval [0.56, 2.05]. The 10,000-iteration runs of seeds 0 and 2 were trained after the verdict (ledger R-60, entered 2026-08-22), so the three-seed figures below extend it and carry none of its weight. The rule is anchored at h = 368, the upstream's **open-loop diagnostic** length and not a deployment horizon (§3.1), and its verdict is returned there; we do not re-anchor a discharged rule. The method's own horizon is h = 100, so the comparison is reported there too, and the two differ in size.

Result. Every condition holds. We give the evidence in order of how little it depends on the small held-out sample.

The sign test, which does not depend on n. At h = 368 the per-episode gap favours autoregressive training on **10 of 10** episodes, an exact two-sided binomial test with p = **0.0020**. At h = 100 it is **10 of 10**, p = **0.0020**, and at h = 1 it is 3 of 10, the same story the interval tells. It uses no bootstrap or multiplicity correction, and episodes, unlike §6's state dimensions, are separable units, so a binomial null is admissible. **Its scope is narrower than the arena labels suggest**: 8 of the 10 episodes are training data for *both* arms. The test is a valid **paired** comparison, since both arms saw identical data and an episode-level difference is due to the training rule rather than to memorisation, but it is not ten out-of-sample episodes and does not measure generalisation. The out-of-sample effect size below carries that burden, on 4 independent trajectories.

The in-sample arena, where the sample is larger. The same comparison on the eight training episodes has 16 independent 400-step trajectories against the held-out arena's 4, 4× more, and gives the same direction at every horizon and checkpoint but one: at h = 8 after 500 iterations teacher forcing leads in-sample, with an interval that excludes zero at both trajectory lengths (`results/review_bootstrap_unit.json`).

The out-of-sample effect size, at every horizon. At h = 368, the rule's horizon, the three-seed extension puts autoregressive training at **0.3582** ± **0.0283** against teacher forcing's **1.6497** ± **0.2858** (standard deviation over seeds, `ddof=1`), a factor of **4.61**×. Arm B predicts each of the window's 8 forecast targets from true inputs, where the original's teacher forcing is N = 1; the sweep's (32, 1), trained that way, is 6.35× worse than the centre at h = 368 at 2,500 iterations (§5.2), so the direction holds under both definitions. At h = 100, the method's own imagination rollout length and the horizon everything in §6 is anchored to, the same three seeds give **2.58**×. Quoting one and not the other would be a choice, so we report the curve (Figure 2): same rollouts, same 3 seeds at 10,000 training iterations, same held-out arena, n_independent = 4, with a cluster bootstrap over whole trajectories; the two floor columns are each arm's error divided by the hold-last floor's, so a value below one beats the floor:

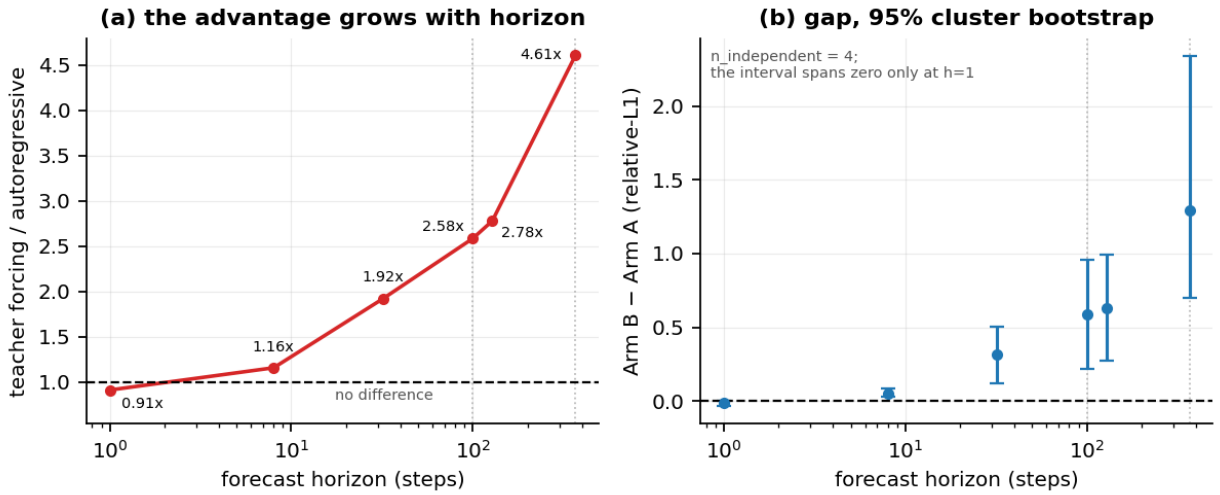


Figure 2: The autoregressive-versus-teacher-forcing advantage as a function of forecast horizon, out-of-sample over three seeds at 10,000 training iterations, on the held-out pair's 4 independent 400-step trajectories. (a) the ratio, which grows monotonically with depth: h = 368 is the end of a trend rather than a selected point, and the method's own rollout length of h = 100 sits partway along it. (b) the same comparison as a gap with its 95% cluster-bootstrap interval over whole trajectories; the interval spans zero only at h = 1, where teacher forcing is ahead – a lead a shorter evaluation unit resolves as real, not nominal (§5, M-64). Rule M-23 was run at h = 368 on seed 1 of each arm; these three-seed values, and every other horizon, were computed afterwards.

h	autoregressive	teacher forcing	ratio	gap [95% CI]	excludes 0	hold-last floor	A ÷ floor	B ÷ floor	episodes A leads
1	0.1447 ± 0.0023	0.1319 ± 0.0172	0.91×	-0.0128 [-0.0329, +0.0013]	**no**	0.0796	1.82×	1.66×	3/10
8	0.3087 ± 0.0318	0.3572 ± 0.0088	1.16×	+0.0485 [+0.0271, +0.0844]	yes	0.3298	0.94×	1.08×	10/10
32	0.3415 ± 0.0491	0.6555 ± 0.0699	1.92×	+0.3140 [+0.1233, +0.5047]	yes	0.5950	0.57×	1.10×	10/10
100	**0.3700 ± 0.0290**	**0.9561 ± 0.0211**	**2.58×**	**+0.5861 [+0.2174, +0.9547]**	**yes**	0.7558	**0.49×**	**1.27×**	**10/10**
128	0.3558 ± 0.0231	0.9881 ± 0.0376	2.78×	+0.6324 [+0.2716, +0.9931]	yes	0.7999	0.44×	1.24×	10/10
368 *(the rule's horizon)*	**0.3582 ± 0.0283**	**1.6497 ± 0.2858**	**4.61×**	**+1.2915 [+0.7004, +2.3390]**	**yes**	0.9930	**0.36×**	**1.66×**	**10/10**

The advantage does grow monotonically with forecast depth. Over 400-step trajectories the gap excludes zero at 5 of 6 horizons and spans it at h=1: h = 368 is the end of a trend rather than a point we picked, h = 100 sits partway along it, and the claim is weakest inside the horizon the model is trained on. **Only the h = 368 row is the rule's horizon, and the rule ran on seed 1 alone.** Every value in the table is a three-seed mean computed after the data existed, so by this paper's own standard (§8) none carries a pre-registration's weight, the same treatment §6.6 gives the expectation we held about the counter-baseline, and nothing in the table discharges or re-opens the rule.

At h = 1 the table understates the evidence, and the correction runs against us. The row rests on 4 independent 400-step trajectories, a unit only the longest horizon needs. Under a rule committed before the index was built (rule M-64, Appendix E), we rebuilt it at 33 rows, 32 of history and one forecast step, non-overlapping within an episode: 60 units on the same two episodes. The gap is -0.0194 [-0.0310, -0.0093]: it **excludes zero, in favour of teacher forcing** (0.85×). Both readings are reported: the rule above was discharged on the 400-step unit, and the short unit resolves the sign. At one step **autoregressive training is worse**, the direction the sign test and, on the 400-step unit, the hold-last floor already pointed.

Against a baseline, because neither number means anything without one. The hold-last floor, predicting that nothing changes, scores **0.9930** in the h = 368 cell, and autoregressive training beats it by **2.8**× there and by 2.0× at h = 100. **Teacher forcing is 1.66× worse than assuming nothing changes at all** at h = 368, and 1.27× worse at h = 100: the arm that reaches a lower training loss predicts the future worse than a model that makes no prediction, at every horizon we measured on the 400-step unit, the sharper statement of what exposure bias costs here; but on M-64's shorter units, each 32 + h rows long and non-overlapping (60 at h = 1, 50 at h = 8, 30 at h = 32, 14 at h = 100 and 12 at h = 128 on these episodes), at the same checkpoint and on the same two episodes, it beats the floor at h = 1, 8, 32 and 100 and loses to it only at the longest horizon those units reach, so that statement holds on the 400-step unit and not on the shorter one. **The floor is not a weak baseline everywhere**: at h=1 it beats the autoregressive arm as well, scoring 0.0796 at h = 1 against autoregressive training's 0.1447, the only horizon where the autoregressive arm loses to predicting no change on the 400-step unit; on the 33-row unit both arms beat the floor at h = 1 (0.1315 and 0.1121 against 0.1687). At 50 Hz one step is 20 ms and the state barely moves, so that is what one should expect.

Seeds, and what four trajectories can show. Seed spread is not symmetric between the arms: Arm A ranges 0.3341–0.3894 across seeds (7.9% relative), Arm B 1.4241–1.9710 (17.3%). Teacher forcing is more than twice as variable across seeds as autoregressive training at this horizon, so a single-seed comparison of these two arms is unreliable. For a single seed the bootstrap over trajectories gives the 95% interval [0.56, 2.05] on n = 4 independent trajectories; by the n = 4 caveat of §3 its tails move in steps of 0.39%, so it corroborates the sign test rather than being the primary evidence. The four per-trajectory gaps behind the three-seed table (Arm B minus Arm A, three seeds pooled, at h = 368) are **+2.8705, +0.8949, +0.7445, +0.6562**. All 4 of 4 are positive, the direction of the sign test's 10 of 10 episodes, but one trajectory carries +2.8705 against a smallest of +0.6562, which no interval on four units shows. At h = 100 the four are +1.0925, +0.8170, +0.2800, +0.1549; Appendix L notes where other sections store theirs.

What is small, and where it resolves. At h = 8, the horizon the model is trained on, the advantage is small, and it resolves only with the longer training and all 3 seeds. The table above, at 10,000 iterations with 3 seeds pooled, gives an h = 8 gap of +0.0485 [+0.0271, +0.0844], which excludes zero, a factor of 1.16×. The single seed the rule was run on (seed 1) gives an h = 8 gap of 0.008 at the same 10,000-iteration checkpoint, and its interval includes zero. At the 500 and 2,500-iteration checkpoints, with all 3 seeds pooled, the out-of-sample gap excludes zero in **0 of 4** h = 8 cells (both trajectory lengths crossed with both checkpoints). An earlier rule of ours, anchored at h = 8 and evaluated at those same checkpoints (rule M-16,

Appendix E), returned **CANNOT BE SETTLED AT THIS BUDGET**. **The advantage is small at the training horizon and large beyond it.**

At long horizons the out-of-sample gap excludes zero in **4 of 4** cells, both trajectory lengths crossed with the 500 and 2,500-iteration checkpoints, and each still excludes zero at a Bonferroni level of 0.05/8 over the family of 8 out-of-sample comparisons, where Holm–Bonferroni over the 4 long-horizon cells rejects **4 of 4** (Appendix L). Beside the artifact it reimplements, on 4 independent trajectories with our arms at 2,500 iterations over 3 seeds, relative-L1 and nRMSE both put the released checkpoint first at h = 1 and h = 8 and an Arm A variant ahead of it at h = 368, a reading that flatters the checkpoint, since the arena is out-of-sample for our arms and in-sample for it (Appendix L).

5.1 The data budget, which is the one part of the sample-efficiency claim we can measure

The base paper's headline is a sample-efficiency result: policies transfer to hardware from 6,000,000 state transitions of world-model pretraining against ~250M for the model-free baseline (Table I). That is a claim about policy learning and hardware, which we cannot test, but its *world-model* half is a quantity we can count exactly.

Our arms consume 7,991 distinct state transitions: the 7,999 rows of the eight training episodes less one per episode (8 of them), since an episode's first row has no predecessor, a transition being a consecutive pair of rows inside one episode. It is deliberately not the 7,687 training windows, which overlap almost completely (consecutive 40-row windows start one row apart), nor the 640,000 window draws a 2,500-iteration run makes, which resample the same data with replacement. Against the reference's 6,000,000, that is **751× less data, 0.133% of its world-model budget**.

A dynamics model trained on 0.133% of the reference's data still reproduces the training result, 4.61× at h = 368 and 2.58× at h = 100 over 3 seeds (the rule ran on seed 1), and still beats the hold-last floor, by 2.8× and 2.0× at those two horizons. That is what this paper can add to the sample-efficiency question without training a policy.

Three limits. It is not a reproduction of the 6,000,000-against-250M comparison, which is about policy learning. It says nothing about whether a policy trained inside our model would transfer to hardware, or anywhere. And our model is evaluated on the narrow distribution it trained on (one robot, one gait, one terrain, velocity commands from a single bounded box), where the reference's 6,000,000 transitions span considerably more; a smaller data budget buys less than it appears to when the evaluation distribution shrinks with it.

5.2 The configuration claim: history and forecast horizons

Claim under test. The model reads a history of M past steps and, in training, forecasts the next N steps from its own predictions. The base paper sweeps both and says that moderate values give an optimal trade-off between accuracy and training time, with (M, N) = (32, 8) as the instance (§IV-C). Its heatmap prints the error of every cell, and the centre's is the tied-lowest, level with its longest-forecast neighbour (`docs/presubmission/ORIGINAL_SPECS.md` a.5). Its second heatmap prints training hours: 1.07 for the centre and 2.27 for (32, 32), 2.12×, so on the original's own figures the centre matches that neighbour's printed error, 0.47, in under half the time. Our measured cost per iteration for the same pair is 1.81× (the table's last column).

Rule, committed in advance (rule M-74, Appendix E). It tests the accuracy half only: whether the centre has the lowest error among its eight one-factor neighbours in the original's grid, M of 1, 2, 8 and 16 at the centre's N and N of 1, 2, 16 and 32 at its M. Each is trained as Arm A is, for 2,500 iterations on three seeds. The statistic is a configuration's relative-L1 at h = 368 minus the centre's, averaged over trajectories and tested by an exact bootstrap over trajectories, with Holm's correction holding the chance of any false "better" in the family at $\alpha = 0.05$. Training time is reported and governs nothing: hours on two CPU cores are not what the original timed on a GPU.

(M, N)	relative-L1, h = 100	relative-L1, h = 368	difference from the centre [95% interval]	result	cost per iteration, relative to the centre
(32, 8), the centre	0.4798	**0.5856**	—	—	1.00
(1, 8)	0.3796	0.5340	-0.0516 [-0.1835, +0.0400]	not resolved	0.23
(2, 8)	0.3524	0.4717	-0.1139 [-0.2988, -0.0088]	**configuration better**	0.26
(8, 8)	0.4043	0.4852	-0.1004 [-0.2449, -0.0167]	**configuration better**	0.40
(16, 8)	0.4467	0.6161	+0.0305 [-0.0008, +0.0720]	not resolved	0.60
(32, 1)	1.0903	3.7198	+3.1342 [+1.3012, +6.5096]	centre better	0.81
(32, 2)	0.8886	2.0374	+1.4518 [+0.6250, +2.6933]	centre better	0.95
(32, 16)	0.3544	0.3347	-0.2509 [-0.5218, -0.0956]	**configuration better**	1.35
(32, 32)	0.3348	0.2987	-0.2869 [-0.5951, -0.1005]	**configuration better**	1.81

Arena: out-of-sample held-out pair, episodes 1 and 8, 4 non-overlapping 400-step trajectories, n_independent = 4. Each row is the mean over three seeds at 2,500 iterations (`results/mn_sweep_eval.json`). A difference is positive when the centre is better, and "result" is after Holm (`results/mn_sweep_verdict.json`). The last column is a configuration's steady training time per iteration over the centre's, timed without contention (`results/mn_compute_matched.json`); hours per run, and the 8 of 24 sweep runs that overlapped other logged CPU work, are in Appendix B. The hold-last floor is 0.7558 at h = 100 and 0.9930 at h = 368. The rules' evaluator averaged nRMSE per trajectory, where §3.1 pools it. The nRMSE readings quoted alongside the rules here and in §5.3 are pooled, recomputed afterwards (post hoc; ledger R-77, `results/pooled_nrmse_alongside.json`), and, for this section's rule (M-74), return the same verdicts as the averaged ones (§5.3 gives the two readings of its rules that differ).

Result: NOT OPTIMAL AT OUR BUDGET. (32, 32), (32, 16), (8, 8) and (2, 8) beat the centre (two shorter histories and both longer training forecasts), (32, 2) and (32, 1) are worse, and (1, 8) and (16, 8) cannot be told apart from it. At h = 368 the lowest error is (32, 32)'s, 0.2987 against the centre's 0.5856, a difference of -0.2869 [-0.5951, -0.1005]. The verdict does not rest on the anchor: on the in-sample arena's 16 independent 400-step trajectories, all four readings there return the same verdict, and among the held-out readings the rule reports alongside, every one agrees except nRMSE at h = 1, which returns CANNOT BE DISTINGUISHED.

At our budget, shorter histories win. The original's error falls steeply from M = 1 to M = 8 and then flattens (`ORIGINAL_SPECS.md` a.5). On the governing reading ours does not fall steeply: (2, 8) and (8, 8), histories shorter than the centre's, beat it, and no shorter history is resolvably worse, though other readings the rule reports put shorter histories behind it: (1, 8) on in-sample relative-L1 and nRMSE at h = 368, and (16, 8) on held-out nRMSE at h = 368. That is a statement about learning speed at this budget, not about converged accuracy: trained four times as long, the centre passes both winning shorter histories on three of the four readings (below), so it does not contradict the original's steep fall to M = 8, whose training budget the original does not state. The original's direction on N holds: the shortest forecasts are far worse, which is §5's teacher-forcing result again, and the longest are better, so the centre's tie with its longest-forecast neighbour becomes a loss.

Accuracy at equal compute (post hoc; ledger R-78). A longer training forecast costs more per iteration, 1.35× the centre's for (32, 16) and 1.81× for (32, 32); the shorter histories that win cost less, 0.40× for (8, 8) and 0.26× for (2, 8) (`results/mn_compute_matched.json`; Appendix U gives every reading, signed as in the table). The shorter histories win on all four readings at 2,500 iterations while using well under half the centre's computation, so that part of the verdict needs no compute correction. The longer forecasts' wins do. Trained to 5,000 iterations, the centre has had 1.11× the computation of (32, 32), and the readings split: (32, 32) is still ahead on the held-out pair at h = 368 (-0.2031 [-0.5033, -0.0275]), the centre is ahead on the in-sample arena at h = 100 (+0.0454 [+0.0356, +0.0563]), and the other two are not resolved. At 10,000 iterations, 2.21× the computation, the centre is ahead on both in-sample readings and neither held-out difference is resolved. Trained that long, it also passes both shorter histories on three of the four readings: on the held-out pair at h = 368, (2, 8) by +0.1135 [+0.0983, +0.1357] and (8, 8) by +0.1270 [+0.0987, +0.1589], with 15× and 10× their computation, and on both in-sample readings; on the held-out pair at h = 100 neither difference is resolved. Which setting is most accurate therefore depends on how long each is trained. None of this re-opens rule M-74.

Limits. One factor is varied at a time, so no interaction between M and N is tested. Our data budget is 0.133% of the reference's (§5.1), and the original states neither the ablation's budget, its evaluation data nor the horizon behind its error, so the verdict holds at our budget and no further: a larger one

may favour a longer history. All 48 runs at 2,500 iterations, the sweep's, the baselines' and both arms', are still lowering their training loss at the end, with slopes from -1.99 to -0.09 per thousand iterations (`results/training_tail_slopes.json`, post hoc), so the ranking is at this budget, not at convergence, and it changes with training length (Appendix U). With 4 independent trajectories, the rule's minimum detectable effect at h = 368, the difference its first Holm step would usually detect, is about 14.5% of the centre's error.

5.3 The architecture claim: MLP, RSSM and transformer baselines

Claim under test. The base paper compares RWM with three **architecture baselines**: other network designs trained on the same data to make the same predictions. They are a multilayer perceptron (MLP) over a flattened window of history; a recurrent state-space model (RSSM; Hafner et al., ICML 2019), in the form DreamerV2 refined (Hafner et al., ICLR 2021); and a decoder-only transformer. It says RWM "consistently achieves the lowest prediction errors across all environments" (§IV-D), gives no number (Fig. 7), and trains the baselines teacher-forced.

What we built. The pinned upstream code has an MLP but no RSSM or transformer, so all three are ours, the MLP following the upstream one. They follow the shapes in the original's Table S7 as we read it, with 610,484 (MLP), 3,180,468 (RSSM) and 132,148 (transformer) parameters against RWM's 714,164. Where the table is silent they take RWM's data, windows, objective, optimiser, iterations and seeds, and the RSSM takes its latent, activation and KL settings from DreamerV2. Each choice is a row of `docs/presubmission/BASELINE_SPECS.md`. Only the predicted state is compared.

Two rules, committed in advance (M-75 and M-76, Appendix E). M-75 trains the baselines teacher-forced, as the original does. M-76 trains them autoregressively, as RWM is, so that training rule and architecture are not confounded. Both take Arm A at 2,500 iterations as RWM and test each baseline's relative-L1 at h = 368 against it, on §5.2's arena with §5.2's bootstrap and Holm correction.

model	parameters	relative-L1, h = 100	relative-L1, h = 368	difference from RWM [95% interval]	result	cost per iteration, relative to RWM
RWM (Arm A, autoregressive)	714,164	0.4798	**0.5856**	—	—	1.00
MLP, teacher-forced †	610,484	1.2057	145.1650	+144.5794 [+14.8420, +373.9553]	RWM better	0.07
RSSM, teacher-forced	3,180,468	2.2128	6.3718	+5.7862 [+2.6251, +10.8616]	RWM better	2.37
transformer, teacher-forced †	132,148	2.0607	15.1548	+14.5692 [+8.7931, +24.0102]	RWM better	0.91
MLP, autoregressive	610,484	0.5400	1.4628	+0.8772 [+0.4248, +1.5662]	RWM better	0.08
RSSM, autoregressive †	3,180,468	3.2660	10.6405	+10.0549 [+5.7436, +18.1659]	RWM better	1.99
transformer, autoregressive	132,148	0.5821	1.8771	+1.2915 [+0.7258, +1.8079]	RWM better	0.97

Arena as in §5.2: out-of-sample held-out pair, 4 non-overlapping 400-step trajectories, n_independent = 4. Each row is the mean over three seeds at 2,500 iterations (`results/baselines_eval.json`). A difference is positive when RWM is better, and "result" is after Holm (`results/baselines_verdict.json`). The last column is timed beside RWM in one sitting (`results/mn_compute_matched.json`); hours per run are in Appendix B. † marks a diverged row, one where any seed's mean relative-L1 at h = 368 exceeds 10× the hold-last floor's (`results/pooled_nrmse_rescore.json`, post hoc). Run-away rollouts dominate those three rows' means; per seed they are MLP, teacher-forced: 182, 16.0 and 238; transformer, teacher-forced: 11.2, 10.4 and 23.9; RSSM, autoregressive: 9.87, 10.8 and 11.3. No verdict depends on that magnitude, only on the sign: each such row's 4 per-trajectory differences from RWM are all positive, so every bootstrap resample favours RWM whatever their size.

Result: REPRODUCES with the baselines teacher-forced, and RWM AHEAD OF ALL THREE with them trained autoregressively. RWM is ahead of baselines built to our reading of Table S7 and trained with RWM's settings for 2,500 iterations: all six comparisons favour it, with intervals that exclude zero. That says less than it seems at h = 368, where every baseline row, in both regimes, is above the hold-last floor and RWM is below it: no baseline beats predicting no change. The second verdict compares architectures at one training regime and is not a verdict on the original's claim, which the first carries. The lead is a long-horizon one, and the rules' relative-L1 readings at other horizons on the held-out pair say where it starts. Teacher-forced, the baselines fall resolvably behind from h = 32 (the transformer from h = 8). Trained autoregressively, the RSSM falls behind from h = 8, but the MLP and the transformer only from h = 100, the method's own horizon, where they trail by 0.060 [0.024, 0.096], about 12.5% of RWM's

0.4798, and 0.102 [0.034, 0.185]. Before those horizons, on the held-out pair, a baseline cannot be told apart from RWM: at h = 1 both rules return CANNOT BE SETTLED, and at h = 8 both return PARTIAL. On the in-sample arena's 16 trajectories, with nRMSE pooled as §3.1 defines it (computed afterwards, post hoc; ledger R-77), two readings differ from the per-trajectory average the rules' evaluator used. Teacher-forced, at h = 1 all three baselines are ahead of RWM: D, signed as the rules sign it (positive favours RWM), is -0.160 [-0.286, -0.002] for the MLP, -0.186 [-0.328, -0.013] for the RSSM and -0.052 [-0.089, -0.005] for the transformer, so M-75's reading there is DOES NOT REPRODUCE where the average gave CANNOT BE SETTLED: the same one-step lead for teacher forcing as §5's. Trained autoregressively, the MLP's deficit at h = 100 is no longer resolved (+0.017 [-0.017, +0.044]), so M-76's reading there is PARTIAL where the average gave RWM AHEAD OF ALL THREE. Neither changes a verdict; both rules govern at h = 368 on relative-L1.

Our RSSM is not an informative comparison. Teacher-forced, it is the most accurate of this section's models one step ahead, 0.0761 against RWM's 0.1232 and the floor's 0.0796, though not resolvably, and it has collapsed open-loop by h = 32, where its 1.0182 is above the floor's 0.5950. The original adds that an RSSM trained autoregressively performs comparably to RWM; ours does not, 10.6405 against RWM's 0.5856 at h = 368. A diagnostic committed before its readings existed (rule X1, ledger M-80; exploratory, it re-opens neither rule) asks why. Its Part A reads the forecast differently, feeding the prior's expected or sampled latent in place of its most likely one: the error at h = 32 falls to 0.7165 and 0.6914, still above the floor, so it returns **NOT EVALUATION-LIMITED** (M-81). Its Part B is descriptive: over the history, the teacher-forced RSSM's one-step error from its prior is 1.16 to 1.27× its error from its posterior at the same recurrent state (the autoregressive one's at most 1.02×), with 16.4 to 20.0 nats of KL divergence between them per step. Its Part C retrains the teacher-forced RSSM at seed 0 for 2,500 iterations, once with PlaNet's KL settings and once with DreamerV2's layer-normalised recurrent cell. Neither rescues it: read from its most likely latent at h = 32, they score 1.2588 and 1.4685 against the floor's 0.5950, with 3 and 2 of the 4 trajectories above their own floor value; a rescue needs the mean below the floor's and every trajectory below its own. So rule X1 returns **NOT RESCUED BY THE SETTINGS TRIED** (ledger M-83). The failure may still be our RSSM rather than the architecture: Table S7's latent is ambiguous, we read it in DreamerV2's naming (`BASELINE_SPECS.md`, the RSSM rows), and X1 tried only the two settings above, on one seed, neither of them PlaNet's latent overshooting (Hafner et al., ICML 2019), which trains the prior's multi-step predictions in latent space and which PlaNet's own final RSSM agent did not need. So the architecture claim rests on the MLP and the transformer.

Limits. One robot on flat ground; the original has several environments. The baselines are our reading of a table that fixes their shapes and nothing else. Their loss, optimiser and budget are RWM's, not tuned for them. Their sizes are Table S7's, not matched to RWM's; parameter-matched variants were specified but not run. With 4 independent trajectories, the minimum detectable effect at h = 368 is about 12.0% of RWM's error, well below every difference here.

6 Neither of the checkpoint's uncertainty outputs is usable as an interval

How this section runs. §6.1–§6.2 settle which quantity the method uses and measure it; §6.3–§6.4 examine why each output fails; §6.5–§6.6 separate the magnitude of the failure from its ordering, and test the ordering against free baselines; §6.7–§6.8 ask what would fix it.

6.1 Which quantity the method actually uses

The released checkpoint emits **two** uncertainty quantities, and the method consumes only one of them. This has to be settled before any calibration number means anything.

`system_dynamics.py:125` computes an **aleatoric** term, the mean over ensemble members of each member's predicted σ. `system_dynamics.py:126` computes an **epistemic** term, the standard deviation across the members' mean predictions. In `envs/base.py:142` the aleatoric term is bound to a local variable that is never read again; the epistemic term is stored, returned to the policy loop at `:158`, and applied at `:166` as a reward penalty with weight -1.0.

The paper agrees with its code. arXiv:2504.16680**v1** Eq. 4 defines the penalised quantity as $u = \mathrm{Var}_b[\mu_b]$, the variance across ensemble members (the numbering is unchanged in the current v3), and Eq. 5 applies it as $\tilde{r} = r - \lambda u$. The per-member predicted variance enters the training objective and nothing downstream.

Eq. 4 specifies a **variance** while `system_dynamics.py:126` computes a **standard deviation**, which with $\lambda = 1$ differ by a square. We asked, and the first author confirms the code is operative: the penalty is applied to the standard deviation as intended, and Eq. 4 is "more of a high-level explanation" (personal communication, 21 August 2026, quoted with the first author's permission; the exchange is reproduced, anonymised, in the supplementary `SUPPLEMENTARY_CORRESPONDENCE.md`, and see Data and code). We measure the code's quantity throughout, which is the intended one.

The same correspondence confirms the discard directly: "the aleatoric term is not used in downstream training. It is reported in Fig. 3 (right) as an analysis of the model behavior." (The figure number follows arXiv:2504.16680**v1**; in the current v3 it is Fig. 4, and the crosswalk is in `results/original_paper_figures.json`.) So the discard is the intended design, not an implementation slip: the aleatoric head, the one the state loss and the bound loss shape and §6.3 explains, is computed on every imagination step to shape training and be inspected, and is never consumed. We report both quantities below. Our own arms are ensemble size 1, where the epistemic term is identically zero by construction, so the epistemic measurement is possible only on the released checkpoint.

What the follow-up does and does not claim. It does not claim its uncertainty is a calibrated interval. Its §5.1 claims the epistemic term "closely follows the trend of the prediction error" and that this "justifies its role as a trust metric", and of the aleatoric term it observes only that it "remains low, reflecting small stochasticity in the environment". Our measurement **supports the first claim**: the epistemic ordering is real (§6.6), though weaker per dimension than it looks (§6.5). What follows is therefore not a refutation of a calibration claim nobody made, but three things the papers do not address: that the aleatoric head is discarded before use, that neither quantity is usable as a scale, and that the low aleatoric value has a different cause than the one offered.

6.2 The measurement

For each model we compute the mean predicted σ, the mean absolute realised error, and the fraction of realised errors falling inside $\pm 1\sigma$. A calibrated Gaussian puts 68.27% inside $\pm 1\sigma$ (§3.1).

model	mean |error| / mean σ, whole 368-step rollout	coverage at $\pm 1\sigma$, h=1	coverage at $\pm 1\sigma$, h=100
faithful Arm A (sampled MSE)	52.2× [39.6, 69.7]	11.67% [7.96, 15.19]	2.17% [1.59, 2.75]
corrected Arm A (`gaussian_nll`)	10.9× [7.8, 14.7]	42.78% [24.44, 62.04]	9.8% [6.80, 12.81]
teacher-forced Arm B	315× [177, 509]	12.96% [7.04, 20.93]	1.22% [0.84, 1.61]
released checkpoint	7,878× [5,410, 9,934]	0.56% [0.00, 1.67]	0.08% [0.06, 0.11]

Our arms are at 2,500 training iterations; the released checkpoint is as released. Every cell carries a 95% interval from a cluster bootstrap over whole trajectories, n_independent = 4, quantised as the n = 4 caveat of §3 describes; where three seeds contribute, seeds are pooled inside each draw rather than resampled, because seeds are not trajectories (§8).

All four rows are the held-out pair — the two episodes withheld from our own arms, n_independent = 4 400-step trajectories — because that is the only arena on which our arms can be scored fairly. On the aleatoric head every model is overconfident by between one and four orders of magnitude (Figure 3); that is the quantity §6.1 shows the method discards. The released checkpoint's 7,878× is its whole 368-step rollout on those same 4 trajectories, for comparability with the arms; its best-sampled figure is the 11,683× below, cumulative to h = 100 at n_independent = 20. **Both are correct, on a different arena and a different horizon**, and neither is out-of-sample for that checkpoint (the in-sample caveat of §3).

The quantity the method does use is also uncalibrated. On the released 5-member checkpoint over all 10 episodes, n_independent = **20** non-overlapping 400-step trajectories. The checkpoint trained on all ten, so restricting it to the held-out pair would buy no independence and cost four fifths of the sample; that version, at n_independent = 4, is in the supplementary material (`results/task_b2_epistemic.json`). Its

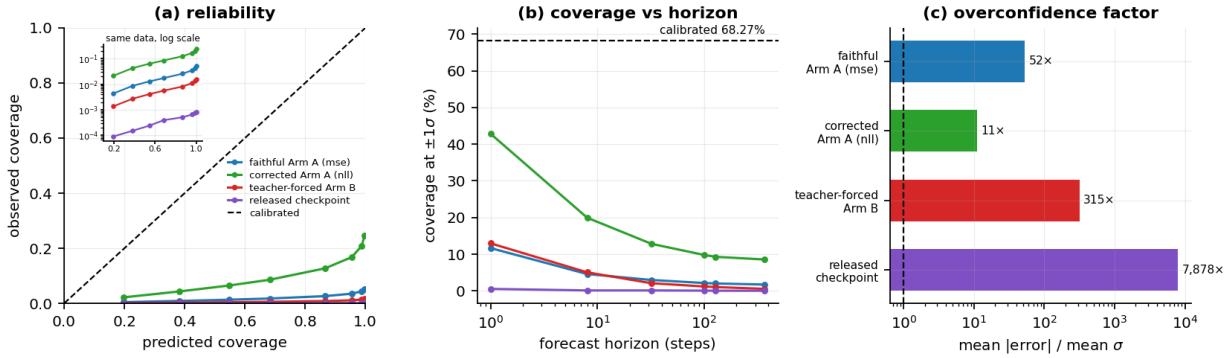


Figure 3: Calibration of all four models on the held-out pair's 4 independent 400-step trajectories, our arms at 2,500 training iterations. The released checkpoint trained on these episodes, so for it this arena is in-sample (§3). (a) reliability: observed against predicted coverage, with the calibrated diagonal; the inset repeats the same points with observed coverage on a log scale, where the four models separate. (b) coverage at $\pm 1\sigma$ against forecast horizon, log scale, against the 68.27% a calibrated Gaussian gives. Every curve sits far below the diagonal and falls further with horizon. (c) the overconfidence factor for each model: mean absolute error divided by mean predicted σ, log scale, with the dashed line where error equals σ.

epistemic column agrees in direction with this one at all 6 of 6 horizons; its aleatoric column agrees at 4 of 6, flipping sign at h=8 and h=100, where both readings sit close to chance and the aleatoric σ is in any case 1,827× (h = 1) to 20,669× (h = 368) too small for its ordering to be the interesting quantity.

h	aleatoric err/σ [95% CI]	aleatoric $\pm 1\sigma$	epistemic err/σ [95% CI]	epistemic $\pm 1\sigma$ [95% CI]	epistemic $\pm 2\sigma$	dims r>0	permutation P
1	1,827× [915, 3,110]	0.11%	**8.3**× [6.1, 9.7]	16.22% [13.44, 18.89]	30.11%	44/45	0.0056
8	3,034× [1,638, 4,849]	0.08%	15.1× [10.3, 19.2]	9.99% [8.60, 11.32]	19.76%	45/45	0.0069
32	4,525× [2,492, 7,196]	0.07%	22.6× [15.5, 28.7]	6.95% [5.99, 7.85]	13.68%	45/45	0.0344
100	11,683× [7,970, 15,846]	0.04%	**33.4**× [28.7, 39.0]	4.61% [4.12, 5.09]	9.27%	45/45	0.2769
128	14,934× [10,564, 19,700]	0.03%	34.2× [30.6, 38.2]	4.37% [3.96, 4.81]	8.75%	45/45	0.3762
368	20,669× [15,666, 25,688]	0.02%	34.4× [29.8, 40.3]	3.59% [3.27, 3.92]	7.19%	45/45	0.0804

Read the h = 1 row first. At one step the disagreement the method penalises rewards with is already **8.3**× **[6.1, 9.7]** smaller than realised error, with $\pm 1\sigma$ coverage of 16.22% [13.44, 18.89] against a calibrated 68.27%. The discarded per-member σ is 1,827× out at h = 1 as well. Everything further down the table is deterioration from a starting point that is already broken.

Why that row, and not the deep ones. The obvious objection is that a per-step σ is a *conditional* quantity: it says how uncertain the next state is given this input, and in an open-loop rollout the input is the model's own previous output, wrong by an amount σ never claimed to describe. Comparing it with *accumulated* rollout error would then make the 368-step figure an artifact of that mismatch. (§6.5 answers a different objection, that a model trained on 8 steps cannot be expected to speak about 368.) **h = 1 answers it at no extra cost.** At one step there is no accumulation: the input *is* the true state, and σ is asked exactly the question it was trained to answer. At h = 1 it is out by 8.3×, and 16.22% of outcomes fall inside an interval that should hold 68.27%. Whatever compounding does at depth, it did not do that.

And that row is measured on data the checkpoint trained on: all ten episodes, 10 of 10 of them (`results/insample_framing.json`). In-sample measurement biases *toward* better calibration, so the 8.3× at h = 1 is if anything flattering: an upper bound on how well this checkpoint is calibrated. So the horizon curve is not the claim; it is the shape of the deterioration, and the claim is the h = 1 row. We keep h = 100 because it is where the method actually deploys, and h = 368 because it is the upstream's own diagnostic length.

At h = 100, the method's own imagination rollout length, epistemic is 349× better than aleatoric and still wrong by **33.4**× **[28.7, 39.0]**, with $\pm 1\sigma$ coverage of 4.61% [4.12, 5.09] where a calibrated Gaussian gives 68.27%. At one step it is **8.3**× out and 220× better than aleatoric; at h = 368 the two-term ratio is 600×.

The gap between the two uncertainty terms is itself horizon-dependent, which is why each figure above names its horizon. At the open-loop diagnostic horizon of h = 368 it is 34.4× [29.8, 40.3] with 3.59% coverage, barely different from h = 100. **Total** uncertainty, `sqrt(aleatoric² + epistemic²)`, equals the epistemic value to four significant figures at every horizon, because the aleatoric term is too small to move it.

A constant scale error would not matter, and this one is not constant. The penalty enters as $\tilde{r} = r - \lambda u$, so a u uniformly c times too small is arithmetically identical to running with λ/c, and λ is tuned. Rescaling σ by the single constant that best calibrates it, fitted and scored on the same data and so an upper bound on what *any* constant achieves, needs 10.4 at h = 1 and 43.2 at h = 368, a factor of 4 across the rollout (rule M-65, Appendix E). No single λ is both of those. The mechanism is §6.5's: σ grows far more slowly than error, which grows by an order of magnitude, so the ratio runs 8.3× at h = 1, 33.4× at h = 100 and 34.4× at h = 368. The penalty is applied at every step of a 100-step imagination rollout, so its *profile across the rollout* is wrong in a way no rescaling can correct: deep-rollout states are under-penalised relative to their realised risk. This bounds what the quantity reports across depth, not what the distortion costs a trained policy (the policy caveat of §11).

The larger sample changes one thing materially. At n_independent = 4 the epistemic ordering looked like chance at short horizon, 23 of 45 dimensions at h=1. At n_independent = 20 it is 44 of 45 at h=1, with mean r = +0.662, the *strongest* mean correlation of any horizon, and the in-sample permutation test agrees (Appendix K). The short-horizon "chance" result was an artifact of four trajectories, not a property of the model.

Two rules committed in advance check how these numbers are read (Appendix M). Resampling whole episodes rather than 400-step trajectories changes no verdict in the one arena with the power to show it, the out-of-sample cells being uninformative by design (rule M-62: **NO MOVE**), and the epistemic term's one-step coverage failure is spread across the 45 state dimensions rather than carried by a few (rule M-63: **UNIFORM**).

The released checkpoint is no longer the only ensemble measured. Three Arm A arms at ensemble size 5 (§6.6, 3 seeds, out-of-sample, n_independent = 4 400-step trajectories, 2,500 training iterations) give, averaged over seeds:

h	epistemic err/σ [95% CI]	$\pm1\sigma$ [95% CI]	$\pm2\sigma$	dims r>0
1	2.1× [1.7, 2.5]	35.93% [29.63, 42.22]	60.37%	35.0/45
8	6.3× [4.4, 7.5]	14.10% [11.02, 17.18]	26.62%	37.3/45
32	8.0× [5.9, 9.8]	10.79% [8.53, 13.71]	20.50%	34.3/45
100	**10.5**× [9.0, 11.5]	8.19% [6.80, 9.59]	16.11%	44.0/45
128	11.0× [9.5, 11.9]	7.79% [6.81, 8.77]	15.34%	44.0/45
368	**13.0**× [11.9, 13.9]	6.30% [5.84, 6.77]	12.51%	40.7/45

Our arms are **better calibrated than the released checkpoint and fail the same way**: 10.5× overconfident at h = 100 against its 25.7× on the same held-out pair (33.4× over all ten episodes), with 8.19% coverage where a calibrated Gaussian gives 68.27%. §6.4 establishes that the two are the same architecture in the respect that matters here, so this is a comparison of like with like. Being closer to calibrated is not being calibrated.

The last column of the released checkpoint's table gives permutation P-values over whole trajectories; none survives Holm–Bonferroni across the arena's 30 cells, so it is a consistency check on direction (Appendix M).

The scalar penalty as actually applied, `means.std(0).sum(-1)` at `envs/base.py:166`, correlates **+0.605** with total absolute error over the rollout, 95% CI [+0.545, +0.694] from a bootstrap over whole trajectories, n_independent = 20 (7,360 pooled trajectory-step points). The interval resamples whole trajectories, not trajectory-step pairs, which would narrow it by about the square root of the rollout length.

6.3 Why the aleatoric head collapses: the optimum is $\sigma = 0$

This subsection explains the aleatoric column and only that column, and leaves the epistemic one open: ensemble disagreement is not shaped by the mechanism below, and why *it* is miscalibrated is not established here. It also supplies the alternative explanation promised in §6.1. The follow-up reads the low aleatoric value as reflecting "small stochasticity in the environment". The observation is correct and the reading is not: σ is low because $\sigma = 0$ is the optimum of the loss that trains it, and it would be low on any dataset, stochastic or not.

The state loss is squared error on a *sample* drawn from the predicted Gaussian, not a likelihood:

$$\mathcal{L} \;=\; \mathbb{E}\big[(\mu + \sigma\varepsilon - y)^2\big] \;=\; (\mu - y)^2 + \sigma^2$$

which is minimised at $\sigma = 0$ for any μ. There is no log-σ term to oppose it. The bound term that appears to oppose it does not, because `max_logstd` is not an independent parameter — it is constructed as `min_logstd + exp(log_delta_logstd)`, so

$$\overline{\log \sigma_{\max}} - \overline{\log \sigma_{\min}} \;=\; \overline{\exp(\log \Delta_{\log \sigma})}$$

and `min_logstd` cancels algebraically, taking no gradient from that term. The floor the interval closes onto therefore freezes while the interval closes: a one-way ratchet.

The derivation above covers two terms, and the objective has 7. Each term, back-propagated alone on one real batch, confirms it: 4 are live under the released configuration, exactly 2 reach σ (`state` and `bound`), and the other 5 give a gradient of exactly zero (`results/e4_sigma_gradients.json`, Appendix J).

The derivation says the collapse happens on any dataset, and that is testable. Under a rule committed before the runs (rule M-50, Appendix E) we trained the released head, unmodified but over a one-dimensional state with no recurrent trunk, on synthetic data whose known noise varies 25× across the input range: **under the implemented objective σ sits 21.7× below the true noise and does not track it at all**, while under the authors' unused likelihood branch it recovers the true level to a median ratio of 0.9779, seed-variably. The rule returns **OBJECTIVE-DRIVEN**, which establishes the contrast and not the size of the recovery (Appendix J).

We predicted the collapse from this algebra before training, then observed it: across all 28 runs at the released width outside §5.2's sweep the log-σ range moves linearly in iteration count at a nearly identical rate within each objective, falling under sampled MSE and rising under the corrected one (Figure 4a). Appendix J lists the runs, the capacity-matched arm excluded from every rate, and the 22 the rate is fitted on.

Two different things are being explained here, and §6.5 separates them. *Magnitude collapse is objective-driven.* It occurs in all 17 sampled-MSE runs at a rate of -9.3857e-05 per iteration with a standard deviation of 6.0e-07, **including the teacher-forced arm**, which shares the objective, and reverses to +3.2296e-05 in the 5 runs that change it. *Input-independence is not.* It varies by a factor of 15.6 between two arms trained under the same objective, so the objective cannot be what produces it.

Under the corrected objective the sign flips (Figure 4b), the strongest evidence that the mechanism is the objective and not the optimiser, the data or the architecture.

The correction fails differently rather than succeeding. The reference contains an unused `gaussian_nll` branch. Running it reverses the collapse and improves the magnitude from 52.2× to 10.9× overconfident. It does not produce a usable estimate, and it destroys something the faithful arm had: the σ-versus-error ordering falls from 39/45 dimensions positively correlated to 21/45, which is chance. Under the trajectory permutation test of §6.5, at h = 368, those counts give P = 0.0417 and 1.0000 out of sample, 0.0240 and 0.8746 in sample. The faithful arm's ordering is the one result in this family that points the same way in both arenas; it is also the weakest effect of the three, and it does not survive multiplicity correction either.

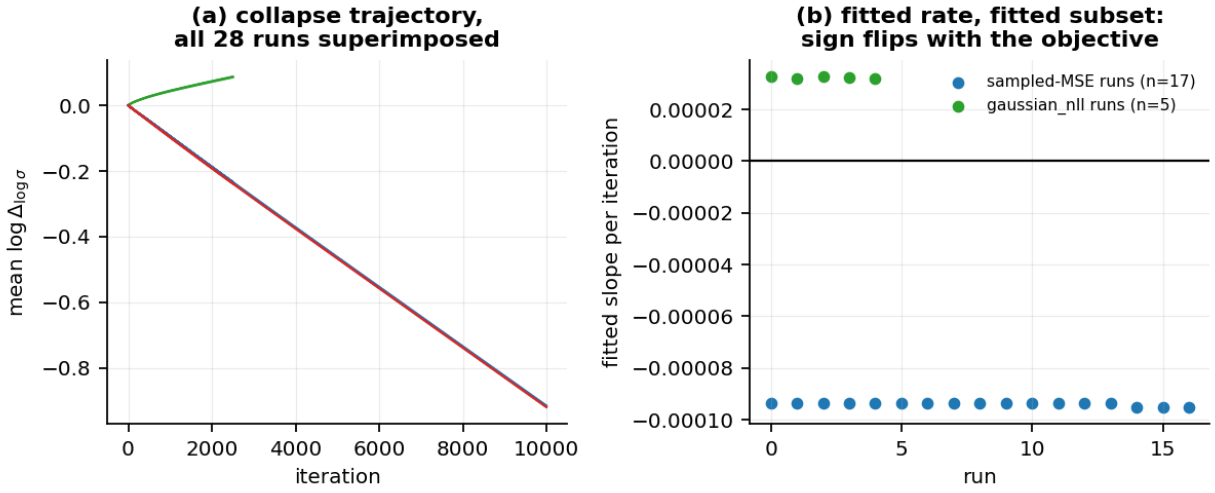


Figure 4: The variance collapse is objective-driven. (a) mean $\log \Delta_{\log \sigma}$ against training iteration for each of the 28 runs of the collapse family, which is every run of §5–§7 at the released width outside §5.2's sweep (Appendix J). The runs are drawn individually but are visually coincident within each objective, so the 28 read as two lines, one falling and one rising – which is the point: the trajectory does not vary visibly from run to run. (b) the fitted per-iteration slope for each run, grouped by objective: negative and tightly clustered under sampled MSE, positive under `gaussian_nll`. The sign flip is the evidence that the objective, not the optimiser or the data, produces it.

6.4 Why the epistemic term may be miscalibrated: the members are not independent models

§6.3 explains the aleatoric column and leaves the epistemic one open. This subsection supplies a candidate, structurally symmetric to §6.3's, from source and from the checkpoint's own tensors; nothing here is trained and nothing is inferred from a measurement. The effect is known (§2). What is ours is finding it in a released robotics checkpoint, with the sharing quantified at 89.15% of each member and its cost bounded by the 2.03× an independent ensemble gains on the overconfidence factor (§6.8).

The released five-member ensemble is not five models. `system_dynamics.py:34` builds **one** `state_base`. `system_dynamics.py:35-41` replicates the *heads* `ensemble_size` times, and only the heads. In the forward pass `system_dynamics.py:87` evaluates the trunk **once** and `:90` hands the identical feature vector to every head; `system_dynamics.py:126` then computes the epistemic term as the standard deviation across those heads.

The parameter counts make the scale of the sharing concrete. The state pathway is a 636,672-parameter two-layer GRU trunk plus five heads of 77,492 parameters each. Per member, 636,672 of 714,164 parameters — **89.15%** — are numerically identical to every other member's. Only 10.85% differ. Across the whole released object, the two shared trunks are 63.81% of 1,995,569 parameters.

The sharing is stronger than the parameter count suggests. The trunk owns a *single* recurrent hidden state (`rnn.py:40`), and an autoregressive rollout feeds the ensemble **mean** back into it (`system_dynamics.py:115`; `src/rwm_model.py:223` in our reimplementation). So the five members do not roll out independently at all. There is exactly one hidden-state trajectory between them, and disagreement at step t is the spread of five two-layer MLPs read off a single 256-dimensional vector.

Members which share a feature extractor have correlated errors by construction, and their spread cannot express uncertainty the shared trunk does not already carry. Where a deep ensemble varies initialisation *and* data ordering across whole models, here only the output heads differ, so the quantity the method penalises rewards with is a lower bound on epistemic uncertainty by construction. It is the same shape of finding as §6.3: not a training failure, a structural one.

This applies to our own arms identically, which is why §6.2's comparison is fair. Our ensemble-5 arms build one trunk the same way (`src/rwm_model.py:164-167`), evaluate it once (`:182`), hand the same vector to every head (`:185`) and compute the same spread (`:200`). Their tensor names and parameter counts match the released checkpoint exactly — 636,672 shared, 77,492 per head, 1,995,569 in total, on all 3 arms checked (`results/v1_ensemble_topology.json`). So §6.2's "our arms fail the same way at 10.5× at h = 100" compares two instances of one architecture, not two architectures.

What this is and is not. It is a *candidate* mechanism, established structurally. It does not show that trunk-sharing is *the* explanation for §6.2's miscalibration; architecture could be a minor contributor to a failure dominated by something else. That needs an ensemble which shares nothing, pre-registered as rule M-44 (Appendix E) and reported in §6.8. The topology is a fact about the released artifact; the mechanism is a hypothesis about that fact.

6.5 The failure is one of magnitude; the ordering is weaker than it looks

Adding the teacher-forced arm, trained for §5, sharpens the finding (our arms at 2,500 training iterations; the released checkpoint as released):

model	σ variation across inputs (CoV)	dims with r(σ, error) > 0 at h=368, held-out pair	perm P at h=368, held-out pair	perm P at h=368, training episodes
faithful Arm A	0.0076	39/45	0.0417	0.0240
corrected Arm A	0.0059	21/45	1.0000	0.8746
teacher-forced Arm B	**0.1188**	**45/45**	**0.2609**	**0.5565**
released checkpoint	0.0177	20/45	0.6957	0.9753

The CoV column is the aleatoric σ in every row, the only σ the ensemble-size-1 arms have. Our ensemble-5 arms (§6.6) have both: their aleatoric CoV is comparable to the other arms', and their *epistemic* term is far more input-dependent than any aleatoric head here, at 0.379–0.395 against the released checkpoint's 0.0177. **The count column is the out-of-sample arena** (n_independent = 4), so that all four models are compared on trajectories none of our arms trained on. For the released checkpoint's aleatoric head it is not the most informative arena: at h = 368 and n_independent = 20 over all ten episodes that head is 0/45, negatively correlated with error on *every* dimension, against 20/45 here; Appendix S quotes the larger arena.

Arm B's σ is 15.6× more input-dependent than the faithful arm's, and it has the largest mean correlation of the four (r = 0.257). It is still 315× overconfident.

The P column is a permutation P, which pairs each trajectory's σ with a different trajectory's realised error and so keeps the coupling between dimensions and the growth of error with depth that every trajectory shares. The binomial P-values an earlier draft attached to these counts are withdrawn (`S-15`), and the correction is large (Appendix K).

So σ *collapsing in magnitude* is objective-driven, and σ *becoming input-independent* is not. The teacher-forced arm collapses in magnitude exactly like the autoregressive ones — same objective, same rate — while retaining 15.6× more variation across inputs. Input-independence is a property of the autoregressive arms and the released checkpoint, and input-dependence and correct ranking are both achievable without the interval becoming meaningful. One candidate mechanism, stated as an untested hypothesis: autoregressive feedback narrows the input distribution toward the model's own manifold, leaving a heteroscedastic head less variation to key on.

The same pattern holds for the quantity the method uses, and this is where the correction bites hardest. At h=128 and h=368 the epistemic term correlates positively with realised error on **45 of 45** dimensions, matching the best aleatoric head here on the sign count, while being 39.7× overconfident at h = 368. **Figures in this paragraph are the held-out arena (n_independent = 4) unless labelled otherwise**, so the epistemic term and the four aleatoric heads are compared on identical trajectories; §6.2 quotes 34.4× for the same ratio at h = 368 and n_independent = 20, and the abstract and §12 use 33.4× at h = 100 on those 20 trajectories. Nor does it beat Arm B's head on strength: its mean correlation at h=368 is +0.151 against 0.257. The two rank comparably, and neither is close to an interval. Under the permutation null that count gives P = 0.0435 on the held-out pair and 0.0775 on the training episodes, both

in-sample for the checkpoint, against 5.68e-14 from the independent-trials test. It still fails the horizon test the same way: σ grows 1.59× from h=1 to h=368 while error grows 13.33×.

Across arenas, and after correction for multiplicity. The two larger arenas, which are nested (every in-sample trajectory is among the all-ten ones), put the epistemic ordering's strength at *short* horizon, because at long horizon the forecast-depth trend every trajectory shares lifts the null until a full count is close to chance, which is what motivates §6.6's index control. Nothing here survives Holm–Bonferroni in any of the three arenas, and out of sample at the 400-step unit nothing could, since its smallest attainable P already exceeds the smallest Holm threshold (Appendix K).

So this section's claim is: **the magnitude failure is established and large; the ordering points the right way for the epistemic term and the faithful and teacher-forced arms, not for the corrected arm or the released aleatoric head (Appendix S), and is not established at conventional significance once the dependence between dimensions is respected. The failure is specifically magnitude calibration, in both components.**

The structural excuse does not survive. One could argue that a model trained on an 8-step horizon cannot be expected to report calibrated uncertainty about step 368. It cannot report it about step 8 either. Inside the trained horizon, σ is flat while error grows (Figure 5; out-of-sample held-out pair, n_independent = 4 400-step trajectories, in-sample for the released checkpoint; our arms at 2,500 training iterations, the released checkpoint as released):

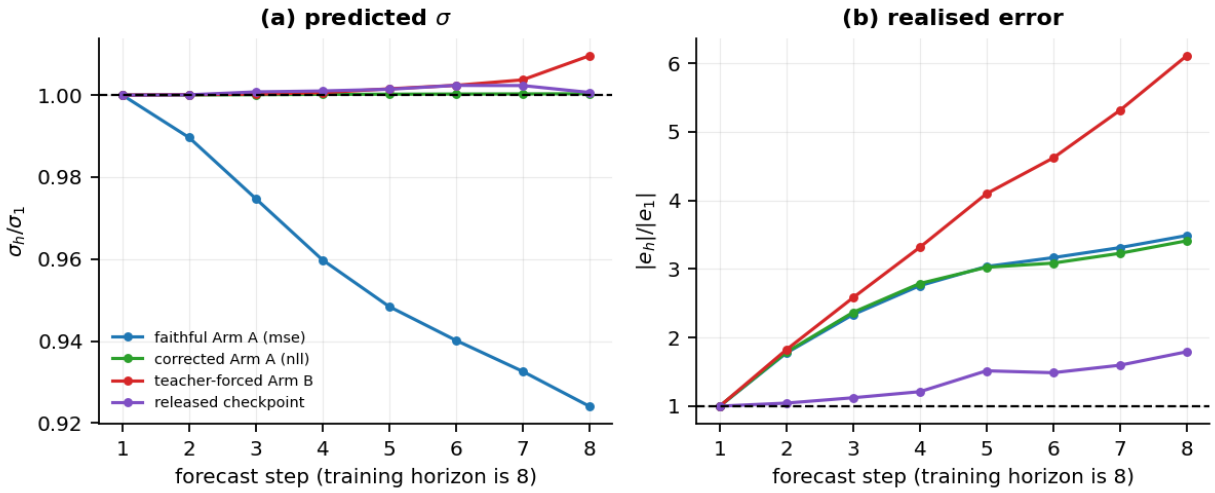


Figure 5: Why the coverage collapse is a horizon effect. Both panels are normalised to forecast step 1, on the out-of-sample held-out pair, n_independent = 4 400-step trajectories; our arms at 2,500 training iterations, the released checkpoint as released; it trained on these episodes, so for it this arena is in-sample (§3). (a) predicted σ barely moves, and for the faithful arm it declines. (b) realised error grows 1.79× to 6.11× over the same steps, across the four models. The gap between the panels is the collapse.

model	σ growth, step 1 → 8	error growth, step 1 → 8
faithful Arm A	0.9241×	3.49×
corrected Arm A	1.0003×	3.41×
teacher-forced Arm B	1.0096×	6.11×
released checkpoint	1.0007×	1.79×

The faithful arm's σ *declines* (0.9241×) while its error grows 3.49×. The coverage collapse in Figure 3(b) is therefore driven entirely by growing error against a fixed σ.

6.6 Ensemble disagreement beats the trivial baseline

The follow-up justifies ensemble disagreement as a trust metric on the grounds that it "closely follows the trend of the prediction error". §6.5 shows the per-dimension version of that claim is weaker than it looks; this section finds that the *scalar* the method applies tracks error well, which coexists with §6.5 without contradiction, and asks the question that matters more to a practitioner: **does disagreement beat something free?** The quantity is a usable ranking signal and is still not an interval.

Error in an autoregressive rollout grows with depth, so the trivial competitor to any trust metric is the forecast step index — a counter. It needs no ensemble, no second forward pass and no model. If a counter ranks error as well as disagreement does, the ensemble is not earning its cost. Neither paper runs this comparison, so we do. All three correlations below are on the scalar the method applies, `means.std(0).sum(-1)` at `envs/base.py:166`, against total absolute error, on the released checkpoint over all ten episodes, n_independent = 20 trajectories, with 95% intervals from a bootstrap over whole trajectories.

h	r(step index, |error|)	r(disagreement, |error|)	partial r(disagreement, |error| · index)	paired difference, disagreement − index
1	—	**+0.994** [+0.918, +0.999]	—	—
8	+0.181 [+0.108, +0.349]	**+0.738** [+0.537, +0.807]	+0.757 [+0.540, +0.822]	+0.556 [+0.241, +0.679]
32	+0.174 [+0.131, +0.336]	**+0.735** [+0.577, +0.823]	+0.756 [+0.604, +0.841]	+0.561 [+0.298, +0.674]
100	+0.490 [+0.394, +0.604]	**+0.628** [+0.574, +0.839]	+0.584 [+0.516, +0.815]	+0.138 [+0.034, +0.362]
128	+0.526 [+0.421, +0.643]	**+0.671** [+0.604, +0.846]	+0.617 [+0.534, +0.812]	+0.145 [+0.011, +0.324]
368	+0.269 [+0.106, +0.431]	**+0.605** [+0.545, +0.694]	+0.596 [+0.526, +0.687]	+0.337 [+0.147, +0.544]

(h=1 has a single forecast step, so the index is constant and its correlation — and therefore the partial and the difference — undefined. The epistemic correlation is not, and it is the largest anywhere in this work: at one step, ensemble disagreement is very nearly a perfect ranking of realised error.)

Disagreement wins at every horizon tested. Over the full h = 368 rollout the counter reaches +0.269 against disagreement's +0.605, and the index leads in 0 of 5 horizons.

The last column is the test of whether disagreement beats the counter. Comparing the two marginal intervals for overlap is the wrong comparison: both correlations are measured on the *same* trajectories, so their sampling errors move together and the marginal intervals are needlessly conservative. The paired difference, resampling whole trajectories and recomputing *both* correlations inside each draw, is the appropriate and more powerful test. It excludes zero at **5 of 5** horizons. The distinction matters at 2 of them: at h=100 and h=128 the marginal intervals *do* overlap. h=128 is where the counter is strongest (+0.526) and the margin narrowest: the paired difference there is +0.145 [+0.011, +0.324], which excludes zero only just: +0.011 is the smallest lower bound in the table, at the 400-step unit. At the shorter unit of rule M-64 (Appendix E), 60 non-overlapping units over the same ten episodes give [+0.148, +0.351] for the same paired difference, so that horizon carries more weight than it did. At h = 100 the paired difference is +0.138 [+0.034, +0.362], which also excludes zero.

The third column asks whether disagreement merely re-encodes the clock. Partialling the step index out of both variables *lowers* disagreement's correlation by 0.010, from +0.605 to +0.596. Almost none of what disagreement knows is explained by how deep into the rollout you are: it carries real information about *this* rollout.

What survives removing each confound (Appendix N). Across 5 models of how far into the rollout a step is, the weakest figure is +0.582, so disagreement is not re-encoding the clock; but the pooled figure is in large part a between-rollout effect, so the statistic that matters holds both the rollout and the depth constant. Preregistered as rule M-45, it gives +0.419 [+0.318, +0.576] at n_independent = 20 400-step trajectories, and the rule returns **DISAGREEMENT CARRIES WITHIN-ROLLOUT INFORMATION**: disagreement still tracks error rather than merely reporting which episode is hard.

Per horizon, on the same 20 trajectories (the released checkpoint over all ten episodes) and the same kind of cluster bootstrap: it is a separate run, so its h = 368 interval differs from the one above in the third decimal, as Appendix N's pooled interval does from §6.2's [+0.545, +0.694].

h	r_dd, double-demeaned	95% CI	excludes zero
1	—	—	—
8	+0.306	[-0.387, +0.713]	no
32	+0.186	[-0.032, +0.636]	no
100	**+0.385**	**[+0.283, +0.701]**	**yes**
128	+0.453	[+0.359, +0.713]	yes
368	+0.419	[+0.315, +0.573]	yes

(h = 1 has one forecast step per trajectory, so there is nothing within a rollout to demean against and r_dd is undefined rather than zero.)

The within-rollout effect is materially smaller than the pooled figure and not established at short horizon; at h = 1 the correlation ranks whole rollouts, on 20 trajectory-level points (Appendix N).

Does it hold on a model we trained? Three Arm A arms at ensemble size 5, under a rule committed before the runs (rule M-43, Appendix E), lead the index in **12 of 12** seed-horizon cells, but the paired difference excludes zero at only 1 of 4 horizons, so the rule returns **DOES NOT GENERALISE**. At n_independent = 4 400-step trajectories it was under-powered, which we measured after the fact and should have checked before committing it (Appendix N).

We ran the baseline test expecting it to go the other way. A counter matching disagreement would have been the more consequential result, making the trust metric close to vacuous, since a counter is free. We record that as an expectation only: it was not committed to git before the data existed, so by this paper's own standard (§8) it is not a pre-registration. It did not go that way against the counter. It went that way against something else.

One adversary is one, so we added two more, and the ranking claim survives only one of them. A claim that beats exactly one competitor is a claim about that competitor. Under a rule committed before either was computed (rule M-51, Appendix E, corrected by rule M-52), we added two baselines needing no ensemble and no second model: `step-size`, the magnitude of the model's own predicted state change $\|\mu_t - \mu_{t-1}\|$, which costs nothing because the rollout has already made those predictions; and `entry-res`, its one-step error at the step *before* the forecast window opens, which costs **one extra rollout in this harness** and nothing in deployment, where a model consumes the history to build its recurrent state anyway. The first rule called both free without distinguishing those, and the second records the correction.

baseline	r(baseline, error)	margin	partial r(disagreement given baseline)	beaten?
forecast step index *(the existing adversary)*	+0.2686	+0.3368	+0.5957	—
`step-size` — $\|\mu_t - \mu_{t-1}\|$	**+0.4697**	+0.1357	+0.5430	**no**
`entry-res` — one-step error before the window	+0.1471	+0.4582	+0.5948	yes

r(disagreement, error) = +0.6053 on the released checkpoint over all ten episodes, n_independent = 20 400-step trajectories. Minimum detectable effect, estimated before either baseline existed: 0.2891 on a margin, 0.1131 on a partial.

The verdict is SURVIVES entry-res ONLY, and the reason is the row a reader should look at twice. Disagreement beats 1 of the 2 free baselines added here. The magnitude of the model's own predicted state change ranks realised error at +0.4697, against +0.6053 for the five-member ensemble disagreement the method is built on. The margin between them, +0.1357, is **below** the 0.2891 this sample size can resolve, so at n_independent = 20 400-step trajectories we cannot say the ensemble ranks better than a subtraction. If the observed margin is the true one, settling it needs 25 of them, an estimate under an assumed effect (§11, Appendix R).

Disagreement does still carry information the subtraction does not. With `step-size` partialled out it retains +0.5430, far above the 0.1131 MDE for that test, and holding the forecast step exactly fixed it keeps +0.7392 against `step-size`'s +0.5196. It is not re-encoding predicted step size. What is not established is that it adds enough to be worth five models. The rule fixed this reading in advance: the partial is the load-bearing test and the margin is corroboration, so a baseline passing the partial while failing

the margin does not refute the ranking claim. **It does refute a framing.** "Disagreement ranks error well" is supported. "You need the ensemble to rank error" is not.

On this axis the follow-up's claim survives adversarial testing against a real baseline, and that is the strongest form of support this paper offers any claim of either original work — now with the qualification that a free baseline comes closer to it than the counter did.

6.7 One constant scalar does not fix it; a per-horizon one lands within the band, though no single cell is resolvable at this arena

Recalibrating a dynamics model's uncertainty inside model-based RL is not new (§2). What is new here is the conditioning variable, the *forecast horizon*, and one negative result: a single global multiplier **fails** where a per-horizon one shows no sign of failing. An open-loop rollout's error accumulates with depth while its predicted σ does not (§6.5), so a horizon-blind recalibration cannot follow the thing it is trying to correct, and this section measures that.

If σ had the right shape and the wrong scale, a single multiplier would repair it, and the finding would be a units problem with a one-line remedy. We tested that. A scalar was fitted on **one** held-out episode and evaluated on the **other**, in both directions, so it is never fitted on its own test set.

Two senses of "unseen", kept apart from here on. A cell is *unseen by the multiplier* when the multiplier scoring it was fitted on the other episode, and *unseen by the model* when the model never trained on that episode. The two held-out episodes are unseen by our arms' models, which trained on the other 8, but not by the released checkpoint, which trained on all ten (§3). **Every released-checkpoint cell in this section is therefore unseen by the multiplier only.**

Fitting at one step works at one step and nowhere else. On the epistemic term — the quantity the method uses — a scalar of 5.08–5.82 brings h=1 coverage to 63–74%, essentially calibrated against the 68.27% target, and leaves h=368 at 17–21%. On the aleatoric term a scalar of 593–611 gives 64–70% at h=1 and 11–15% at h=368. Fitting over the whole rollout instead drives one-step coverage to 100%, an interval wide enough to be vacuous where the model is accurate, while still falling short at the far end. A constant multiplier cannot track an error that grows while σ does not (§6.5), so for a *constant* scale "right shape, wrong scale" does not survive.

On the released checkpoint a per-horizon scalar lands near target, and this is the one concrete remedy in this paper; on a model that has not seen the test episodes the evidence is mixed. One multiplier per horizon is fitted on one held-out episode and evaluated on the other, in both directions, so no multiplier is ever scored on the episode that produced it. The two held-out episodes contribute 4 non-overlapping 400-step trajectories between them, so each direction fits on n_independent = 2 and is scored on the other 2. The last column repeats the test on Arm A at ensemble size 5 and 2,500 iterations (3 seeds), whose model never saw either episode:

quantity	released checkpoint: cells unseen by the multiplier within 10 points of 68.27%, per-horizon c (unpowered)	same, constant c	per-horizon c fitted on the *other* model, both models scored (unpowered)	range of fitted c	Arm A, its own per-horizon c: cells within the band, unseen by the model too
aleatoric	**12 / 12**	2 / 12	0 / 72	592.6 – 7782 (13.1×)	10 / 36
epistemic	**12 / 12**	2 / 12	12 / 72	5.082 – 47.33 (9.31×)	17 / 36

On the released checkpoint every point estimate lands within 10 points of the 68.27% target for both quantities. **That absolute test is UNPOWERED here, as in the *different model* column:**

`results/p4_transfer_power.json` scored this model under these very multipliers and found the 10-point band resolvable at 0 of 12 quantity-by-horizon pairs, with a binding minimum detectable effect of 12.55–40.62 points. So a cell inside the band is compatible with a true coverage well outside it, and nothing here shows any cell is outside it either. The largest deviation over all 24 held-out cells is aleatoric at h=100, fitted on episode 8 and scored on the other, at 77.48%, 9.21 points off target. The two largest deviations are both on the aleatoric term and both above target — 77.48% at h=100 and 76.55% at h=128 — so the fitted multiplier is mildly **conservative** at the long horizons rather than unstable in both directions. The constant scalar manages 2 of 12, and those are the h=1 cells it was fitted at. **On Arm A, where the episodes are unseen by the model as well, its own multipliers manage 17 of 36 epistemic and 10 of 36 aleatoric cells**, so the recipe that lands every released-checkpoint cell does not carry over intact to a model that has not seen the test episodes.

Three cautions. The per-horizon scalar has one free parameter per horizon against the constant one's one, so it *must* fit better in sample; only cells unseen by the multiplier are evidence, and those are the cells reported. **That column is thinner than its count suggests:** the 12 cells are 6 horizons × two fold directions on the same 4 trajectories, each multiplier fitted on n_independent = 2 and scored on the other 2. They are not 12 independent successes and no P-value attaches to the count, which is reported so a reader can see how thin the evidence is; later sections refer back to this as the §6.7 caution. And the correction is a calibration patch, not a fix: it leaves σ carrying no more information than before and rescales it by how far ahead you are looking. On the released checkpoint it nevertheless brings every estimate unseen by the multiplier near what the interval claims, which is what a downstream user needs, and it costs one lookup table.

The fourth column: the table is a property of the model, not of the horizon. Under rule M-69 (Appendix E), multipliers fitted on Arm A and scored on the released checkpoint, and the reverse, on the same 4 held-out trajectories, return **DOES NOT TRANSFER — A PROPERTY OF THE MODEL** on the paired change in coverage the arena can resolve: 52 of 72 governing cells, which are not independent tests, have an interval wholly outside the ±10-point band, and Arm A's multipliers are 0.0069× to 1.36× the released checkpoint's at the same horizon (Appendix T). The *different model* column is the absolute test, unpowered for the reason above, so it is reported and cannot move that verdict.

So the accurate form of this section is: **a constant scalar does not repair the interval; on the released checkpoint a per-horizon one brings every estimate unseen by the multiplier within the band, though no single cell is resolvable at this arena, and does so across episodes but not across models. On episodes unseen by the model as well, Arm A's own multipliers manage 17 of 36 epistemic cells, so there the evidence is mixed.**

6.8 Testing the mechanism: an ensemble that shares nothing, alone and with the corrected objective

§6.4 establishes the topology as a fact and the mechanism as a hypothesis. Two rules committed to git before any of their artifacts existed test it, each with a minimum detectable effect estimated in advance (Appendix E). Rule M-44 scores 5 independently initialised full models, Arm A at ensemble size 1 trained at 5 seeds, together as an ensemble at evaluation time: each member keeps its own recurrent hidden state and the ensemble mean is fed back to all of them, so the rollout differs from the shared-trunk one only in what is under test. Rule M-68 repeats it with every member trained under `gaussian_nll`, so the two fixes are measured on the same models. Both are scored on the held-out pair against each of the shared-trunk seeds separately.

h	independent err/σ	shared-trunk err/σ	independent $\pm1\sigma$	shared-trunk $\pm1\sigma$
1	1.4×	2.1×	50.00%	35.93%
8	3.4×	6.3×	22.64%	14.10%
32	4.4×	8.0×	17.92%	10.79%
100	**5.2**×	**10.5**×	**15.31%**	**8.19%**
128	5.3×	11.0×	15.03%	7.79%
368	5.1×	13.0×	15.00%	6.30%

Same trajectories (the held-out pair), same harness, n_independent = 4, every model at 2,500 training iterations. The shared-trunk column is the mean over 3 seeds; the comparison below is paired against each of them separately.

Rule M-44 returns MECHANISM SUPPORTED. All 6 of its 6 conditions hold against every one of the 3 shared-trunk seeds: at h = 100 the overconfidence factor improves **2.03**× against a minimum detectable effect of 1.45×, coverage rises a mean of +7.12 points against 2.26, and every paired interval excludes zero. σ itself grows 1.65× there, **71%** of the improvement, so the trunk-sharing mechanism is the larger part of the effect where the method operates; at h = 368, 57% is the ensemble simply predicting better (Appendix O).

h	combined err/σ	shared-trunk err/σ	combined $\pm1\sigma$	shared $\pm1\sigma$	combined $\pm2\sigma$	shared $\pm2\sigma$
1	1.4×	2.1×	51.11%	35.93%	78.89%	60.37%
8	3.5×	6.3×	20.62%	14.10%	39.72%	26.62%
32	4.4×	8.0×	17.97%	10.79%	32.66%	20.50%
100	**5.4**×	**10.5**×	**15.53%**	**8.19%**	28.27%	16.11%
128	5.5×	11.0×	15.39%	7.79%	27.97%	15.34%
368	5.4×	13.0×	14.44%	6.30%	27.63%	12.51%

Same trajectories (the held-out pair, n_independent = 4 400-step trajectories), same harness, same bootstrap unit as the table above, every model at 2,500 training iterations. The shared-trunk columns are the mean over 3 seeds; the comparison below is paired against each of them separately.

Rule M-68 returns THE COMBINATION IMPROVES CALIBRATION, branch 2 of the four it names: all 5 of its 5 conditions hold, a **1.94**× improvement against a minimum detectable effect of 1.218×. That is essentially what independence gave alone. Against the independent arm, which differs in the loss and nothing else, switching to `gaussian_nll` multiplies the overconfidence factor by 1.044× [1.029, 1.054], slightly the wrong way and below what the design can resolve (Appendix P). **The two fixes do not add**: the objective's separate contribution is at most small, which the design can bound but not establish as zero. Once stated that is no surprise: the objective governs the aleatoric head, and the epistemic term is a spread across members the loss never sees.

What the arms bound, and what they do not repair. Independently seeded runs differ in data ordering as well as initialisation, and the independent members carry 3.49× the shared-trunk arms' state-pathway parameters, so these comparisons bound the architectural effect rather than isolating it; at matched capacity the improvement falls to 1.79× against a minimum detectable effect of 2.00×, so that rule returns **UNDER-POWERED — favours the matched ensemble by less than the MDE** (rule M-49, §11). Neither arm is an interval: at h = 100 the independent ensemble is 5.2× overconfident with 15.31% coverage and the combined arm 5.4× with 15.53%, where a calibrated Gaussian gives 68.27%. §6.7's per-horizon multiplier remains the only correction in this paper whose estimates all land within the band, though only on the released checkpoint and with no single cell resolvable at this arena.

7 Defects in the released pipeline

Appendix H lists the ledger's other confirmed contributions that the body does not state, among them defects in the released data pipeline and departures of the released code from the paper's description.

7.1 Ten unmarked episode boundaries. §3. The window builder reads a termination column that is identically zero, so it marks all 9,961 windows valid.

7.2 Training and evaluation disagree on action alignment, and evaluation is the broken one. Row t holds the action that *produced* state t. The data say so directly: at every reset row all twelve actions are exactly zero (§3), and a policy network with biases cannot emit exact zeros, so those zeros mark the absence of a producing action — the reset produced that state, not a policy step. The training path pairs states and actions index-for-index, which is causally correct; the evaluation path feeds the action from $t-1$ to predict state t, stale by one step.

What the stale pairing costs depends on the horizon. All ten episodes are this checkpoint's training data, so the held-out pair is not more honest here, only smaller, and the larger arena leads. Over all ten episodes' 20 independent trajectories the stale action raises the released checkpoint's relative-L1 error by 55.0% [26.3, 92.7] at h = 1, 54.4% [37.5, 82.3] at h = 8 and 37.4% [21.3, 65.4] at h = 32, and from h = 100 the change is not resolved: 8.2% [-10.4, 24.9] there and -4.6% [-13.4, 3.2] at h = 368, where single trajectories run from -47.4% to +35.2%. In nRMSE the rise is resolved at h = 8 and 32 (31.5% [24.7, 68.2] and 17.4% [7.7, 47.4]) but not at h = 1 (7.4% [-14.8, 87.8]), and from h = 100 it is not resolved either (7.9% [-18.9, 32.9] there, -2.1% [-10.0, 3.9] at h = 368). Each is a 95% interval from a cluster bootstrap over whole trajectories, both pairings inside each draw (`results/alignment_by_horizon.json`, `results/alignment_defect_ci.json`). On the held-out pair's 4 independent trajectories the stale action raises the error at every horizon reported: by 34.2% [10.6, 75.0] at h = 1, by 22.5% [6.3, 41.7] at h = 100, where the larger arena resolves no change, and at h = 368 by 7.9% [3.1, 13.0] on relative-L1 and 6.6% [1.0, 8.0] in nRMSE, with per-trajectory values +0.8%, +9.8%, +14.8%, +6.5% and +2.4%, +5.8%, +8.3%, -0.3%. Shifting the evaluation loop's two action slices by one step (`model_training.py:129` and `:132`) aligns it with training; our harness does this with `action_offset = 1` (`src/score_reference.py:180-190`). Our own Arm A checkpoints at 10,000 iterations, trained under the causal pairing, change by -0.99% at h = 1 and +22.67% at h = 368 when fed the stale one (three-seed mean, relative-L1, held-out pair; ledger R-79, which replaces R-76's void figures). They respond to the action. Given another trajectory's actions, their error at h = 8 on their 16 in-sample trajectories rises by +676.3% [+454.8, +1060.0], and at 2,500 iterations by +227.8% [+131.6, +398.2], so rule X2, committed before its readings existed (ledger M-84), returns **RESPONDS TO THE ACTION** at both checkpoints (Appendix V). On those trajectories the stale action itself raises their error at h = 1 by +124.6% [+75.8, +201.9]; a one-step shift moves the action by 0.32 of its spread there, and a swap by 1.44.

7.3 No held-out evaluation. Evaluation trajectories are drawn from training data. For the released checkpoint, trained on the entire file, no held-out measurement is possible at all, and neither pinned repository can generate the data that would make one possible (§3).

7.4 What the spliced windows cost: nothing measurable in rollout. A contaminated arm trained with 195 spliced windows added, at 2.47% contamination (below the pipeline's 3.53%, by design, so that no held-out row leaks), raises training loss by 21.57% where a duplication control adding the same number of exact copies raises it by 0.90%, and in rollout, over 32 cells (the held-out pair and the training episodes, 200 and 400-step trajectories, the 500 and 2,500-iteration checkpoints, n_independent 4 to 39), it is hurt in 0 and helped in 9 (Figure 6a), the signature of regularisation. At this rate the splices do not harm rollout; the unmarked boundaries remain a real defect on leakage grounds (Appendix Q).

7.5 The released artifacts do not reproduce the released checkpoint's variance state. Read as a clock, §6.3's collapse rate puts the checkpoint's variance state out of reach of a constant-rate run from the released initialisation at the configured learning rate, at every iteration count the release, the paper and the checkpoint tag state. The first author's account, that the release is several revisions removed from the setup that trained it, supplies a mechanism, a warm start or a different initialisation, so this is a documentation gap between a release and a run rather than an inconsistency (Appendix F and the supplementary `docs/APPENDIX_G_VARIANCE_ARITHMETIC.md`).

8 Method

An append-only ledger. Every claim here has a permanent identifier, an evidence class (source, data, run, external, inference) and a status, in `FINDINGS_LEDGER.md` (285 entries). Claims are never edited in place: one that turns out to be wrong is marked superseded, pointed at what replaced it, and kept.

Pre-registration, and one failure of it. Decision rules were committed to git before the data that tested them, with one exception. Figure 1 and Appendix E give the lead time for all 23; 22 are positive and 1 is not. Every positive bar is a difference of two commit timestamps. **The negative one is not**: it is the duplication-control rule (§7.4), whose *data* side is the moment the control runs finished, a line in `results/control_driver.log` rather than a commit, dated from the commit that introduced that line. The rule was stated in conversation before the runs and reached git **2.9 hours after they finished**, and we found it only by auditing our own `git log`. The measurement stands, because the arm was built without

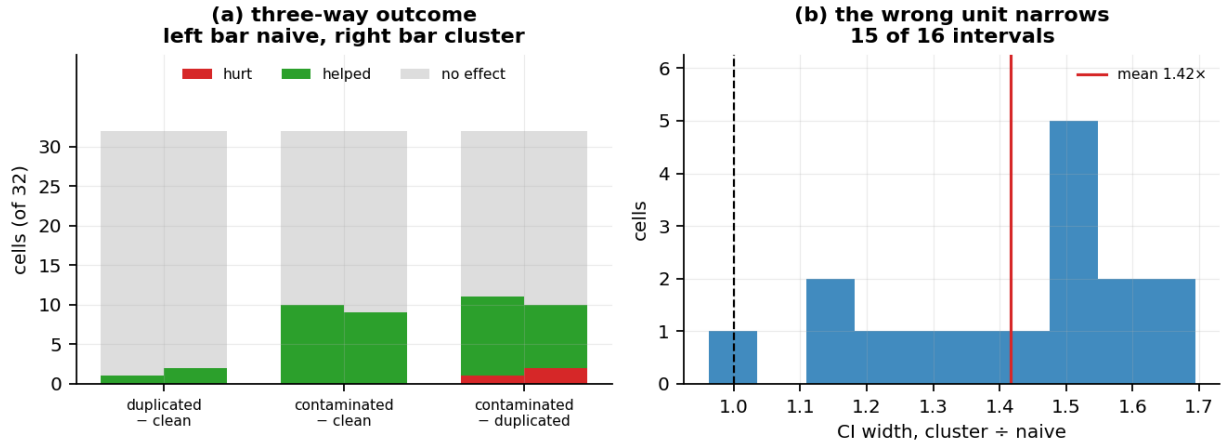


Figure 6: The contamination control. (a) outcome across 32 cells for each arm pair (the held-out pair and the training episodes, 200 and 400-step trajectories, the 500 and 2,500-iteration checkpoints, n_independent 4 to 39), naive bootstrap on the left of each position and cluster bootstrap on the right; the duplication control is inert. (b) a different family, the 16 cells of §5's A/B gap at the 500 and 2,500-iteration checkpoints (§8): distribution of the ratio of cluster to naive confidence-interval width, with the mean marked. Resampling pooled seed × trajectory values rather than whole trajectories narrows 15 of the 16 intervals.

reference to its outcome, but the claim that it was pre-registered does not, and it is withdrawn (`S-12`). A discipline that is only checked when it succeeds is not a discipline.

Eight claims withdrawn on evidence, and six framings withdrawn rather than numbers, out of 21 superseded entries kept in the record (the supplementary `docs/BUILD_CHECKS.md` lists them). The most consequential of the framings withdrawn is `S-15`: the inference from per-dimension sign counts to a binomial P-value, which assumed an independence the 45 state dimensions do not have (§6.5). Found by our own pre-submission audit, it withdraws the strength of evidence behind what an earlier draft called its strongest result.

A statistic that was resampling the wrong unit. Our bootstrap pooled three seeds over a shared trajectory set and resampled the pooled vector while reporting the independent-trajectory count, so each trajectory appeared three times. Resampling trajectories instead widens intervals by a mean 1.42× and changes 1 of 16 verdicts, in the out-of-sample h = 8 cell at 200-step trajectories and 2,500 iterations, already recorded as unresolvable. Every long-horizon verdict survives; both units are reported.

Reproducibility, and a build that checks its own prose. Every measured number in this paper is substituted from an artifact on each build, and a quick run on a clean clone (`./reproduce.sh --quick --force`, which skips training) rewrites 0.92% of the numeric values under `results/` that the comparison counts; the rest are carried in and prove nothing about reproduction. **The number of regenerated values that differ and are themselves a measurement, a statistic or the verdict of a test is 0.** One of the build's own gates, the clean-clone check in `part_f_gate`, requires that no regenerated value differ at all; 3 do, so it fails, and it is published as failing rather than given a tolerance. The accounting behind these figures, the registry of checks the build runs on this paper's own prose, and the paper's record of verifying its own claims are in `docs/BUILD_CHECKS.md`, shipped as supplementary.

9 Actionable lessons

Six things a practitioner can apply without reading the rest of this paper.

Use ensemble disagreement as a ranking signal, but price it against the free alternatives first. Do not read it as a distance. And expect it to degrade with horizon. On the released checkpoint's

own training episodes, at one forecast step it ranks whole rollouts almost perfectly, +0.994 [+0.918, +0.999] across the 20 trajectories, a ranking of *rollouts* rather than of moments within one, because at h=1 there is only one moment. Over the full rollout it falls to +0.605 [+0.545, +0.694]: excellent where you can check it cheaply and merely good where you most need it. It beats the forecast step index at every horizon we tested, on a paired test that excludes zero at 5 of the 5 horizons where the index is defined, and retains +0.596 once that index is partialled out (§6.6). **What it does not clearly beat is the model's own predicted step size**, which costs nothing and needs no ensemble: that ranks error at +0.4697 against disagreement's +0.6053, a margin of +0.1357, below the 0.2891 this sample can resolve. Disagreement carries information the subtraction does not, retaining +0.5430 once step size is partialled out, and with both the forecast depth and the rollout held constant it still correlates +0.419 [+0.318, +0.576] with error (§6.6); but a practitioner about to pay for five members should price the subtraction first. And it is too small to be an interval by a wide margin: at h = 100, the horizon the method rolls out over, 33.4× [28.7, 39.0] on the released checkpoint over all ten episodes, and on the held-out pair 10.5× [9.0, 11.5] on the ensemble-5 arms we trained, against the released checkpoint's 25.7× there. A risk gate or safety margin that reads σ as a distance is not supported at any horizon, on either.

If you need the interval, rescale per horizon, not globally, and refit on your own model. One multiplier per forecast horizon, fitted on one episode and scored on another, brings every released-checkpoint coverage estimate within 10 points of nominal, though this arena cannot resolve any single cell to that band; a single global multiplier manages 2 of the 12 epistemic ones (§6.7). That checkpoint trained on both episodes; on Arm A, whose model never saw them, its own multipliers manage 17 of 36 epistemic cells. The cells are not independent trials (the §6.7 caution), so read the sweep as consistency and the per-cell deviations as the evidence. The fitted epistemic multipliers span 9.31× across horizons, which is precisely why one number cannot serve.

Do not convert per-dimension sign counts into P-values. State dimensions in a robot are physically coupled and share a forecast-depth trend, so an independent-trials null is badly wrong, in our tables by up to $10^{13}\times$ (§6.5, Appendix K). Permute whole trajectories instead. An earlier draft of this paper used the binomial version, and it made our weakest evidence look like our strongest (`S-15`).

Count independent trajectories, not trajectories. Two 400-step windows that overlap at all are one piece of evidence, not two. The held-out arena here contains 4 independent 400-step trajectories however many windows are drawn from it, and that number bounds every long-horizon claim (§3); reporting an interval beside a trajectory count rather than an independent-trajectory count overstates precision. Resampling pooled seed × trajectory values instead of trajectories narrows intervals by a further 1.42× (§8).

Anchor a decision rule to the horizon the claim is about. Our first pre-registered rule was anchored at h = 8, the training forecast horizon, and returned **CANNOT BE SETTLED AT THIS BUDGET**; the claim was about deployment horizons. The rule was correct in form and pointed at the wrong regime, a failure mode pre-registration does not protect against on its own.

Check that the implemented loss is the described loss before reproducing any number from it. The paper describes two loss terms; the implementation has 7. The predicted variance has an optimum at zero under the implemented one, which is why the released checkpoint's σ is 11,683× smaller than its own error at h = 100, and 20,669× at h = 368. Reading the loss took an afternoon and explained a result that would otherwise have looked like a training bug.

10 Broader impact

This is a reproduction of a dynamics model on public simulation data. It creates no new capability, uses no personal data and deploys nothing.

The findings bear on one practice, and it is not the one the original method uses. The follow-up applies ensemble disagreement as a reward penalty. For that use, our measurements support its ordering (§6.6), with the qualification that a free signal ranks error nearly as well. A different use is reading the same number as an error bar, a safety margin, or a trigger for handing control to a fallback controller. The original papers neither make nor recommend that use, and it is the use our measurements rule out. At the method's own

100-step horizon, the disagreement is 33.4× smaller than realised error and covers 4.61% of outcomes where 68.27% is expected. We say this because the released checkpoint exposes the quantity, and it is easy to read as an interval.

We do not claim the original method is unsafe. No policy is trained here. The one policy-free test we ran found that correcting the scale per horizon leaves every pairwise ordering of accumulated penalty unchanged on the available trajectories, a test whose null is partly structural (§11).

11 Limitations

Limits that belong to one result are stated beside it: the budget and our reading of Table S7 (§5.2, §5.3), the mechanism of §6.4 as a hypothesis, and the recalibration's two episodes (§6.7). Appendix R gives every limitation below in full.

Effective sample size bounds every long-horizon claim. The out-of-sample arena has 4 independent 400-step trajectories (the n = 4 caveat of §3), the binding constraint on §5, and a larger dataset cannot be generated here (Appendix C), so the released checkpoint's lack of a held-out arena is a constraint, not a choice.

One dataset, one gait, one terrain. All commands are drawn from one bounded box and the gait is a single trot, so "generalisation" here means across velocity commands only.

Single seeds. The long-horizon trend fit and the per-dimension matched comparison rest on seed 1 alone (Appendix H), and the headline A/B verdict on seed 1, the one its rule ran on; the magnitudes beside it are three-seed means with per-seed values (§5).

Ensemble size: supported, not established. Our ensemble-5 arms reproduce the direction of §6.6's ranking finding and the calibration failure, but rule M-43 returns **DOES NOT GENERALISE** on their 4 held-out 400-step trajectories, where it was under-powered (§6.6). So that finding is established on the released checkpoint and supported but not established on a model we trained.

The ranking claim is not established as needing an ensemble. The model's own predicted step size ranks error nearly as well, by a margin this sample cannot resolve (§6.6). If the observed margin is the true one, settling it needs 25 independent 400-step trajectories where all ten episodes provide 20, a required-sample-size estimate under an assumed effect (Appendix R).

We did not measure what the miscalibration costs. The penalty the follow-up applies is miscalibrated as a scale, 33.4× overconfident at h = 100, the horizon its own imagination rollouts run to, but the method's only use of that quantity is to shape policy learning, and we did not train a policy. **The finding bounds what the quantity reports, not what it costs**, and the ratio is not a measure of harm. Other sections refer back to this as the policy caveat of §11. A proxy that needs no policy, the ordering of the penalty accumulated along whole rollouts before and after the per-horizon correction, finds none of the 6 pairs among the held-out pair's 4 trajectories reordered (in-sample for the checkpoint; rule M-70: **DOES NOT REORDER**), but it orders the penalty alone, not the penalised return, its null is partly structural, and its interval is degenerate (Appendix R).

The per-dimension ordering tests are underpowered at every sample size we can reach at h = 368 (§6.5, Appendix K); at h = 128 and below a shorter unit would raise the count, a rerun we did not do. This limits the per-dimension evidence only; the aggregate scalar the method applies is one test, not forty-five coupled ones (§6.6).

No family-wide correction is applied across our own pre-registered rules. There are 23 of them with per-rule verdicts (Appendix E), each reported against the thresholds it was committed with (one, `S-12`, is withdrawn as a pre-registration because it reached git after its data, §8); a reader who prefers a corrected family threshold can apply it from that count.

The independent-ensemble comparison bounds the trunk-sharing effect rather than isolating it. Independent members differ from shared-trunk heads in data ordering and in capacity as well as in sharing. Rule M-49 (Appendix E) holds capacity fixed, at 1,023,880 state-pathway parameters, a ratio of

0.9998: the independent ensemble is still better calibrated on every shared-trunk seed, every paired interval excludes zero and its coverage gain of +6.42 points clears its own MDE, but the overconfidence improvement falls from 2.03× to **1.79**× against an MDE of 2.00×, so the rule returns **UNDER-POWERED — favours the matched ensemble by less than the MDE**. Capacity does not explain the effect away; how much of it capacity accounts for, this design cannot say (Appendix R).

Deliberately out of scope. No policy-learning result of either paper is reproduced or tested, the sample-efficiency comparison is not tested for the same reason, and nothing here uses a GPU. We did not test whether the $\sigma = 0$ optimum affects other descendants of the PETS parameterisation: the hypothesis is well-founded only for one that makes the same substitution and leaves nothing pushing its floor back up, and is untested for any (§2, Appendix I).

12 Conclusion

The Robotic World Model's central training claim reproduces at long horizons, and the margin is large there, though teacher forcing leads at one step on the short windows built for it. RWM is ahead at h = 368 of the baselines we built to our reading of it and trained with its settings, though there none of them beats predicting no change, and on accuracy alone four other history and forecast lengths beat its chosen ones at our budget, a verdict every reading the rule reports returns but one, which is unresolved, and a ranking that changes when the chosen setting trains longer; it chose them as a trade-off with training time (§5.2, §5.3). Neither uncertainty output of the follow-up that adds them reports what a reader would take it to report. At h = 100, the horizon the method's own imagination rollouts run to, the aleatoric σ is 11,683× smaller than its own error, because the objective's optimum is $\sigma = 0$ and the term that should prevent this cancels out of the gradient. The epistemic term the method actually penalises with is better by a factor of 349 and still 33.4× [28.7, 39.0] overconfident, both figures at h = 100 on the 20 trajectories of the checkpoint's own training episodes.

The ranking use the follow-up claims survives a real test. On the released checkpoint, ensemble disagreement beats the forecast step index at every horizon and, with both the rollout and the depth held constant, still correlates +0.419 [+0.318, +0.576] with realised error (§6.6), though a free subtraction, the model's own predicted step size, ranks error nearly as well, +0.470 against disagreement's +0.605, by a margin this sample cannot resolve (SURVIVES entry-res ONLY: disagreement beats the one-step error before the window, but not the model's predicted step size). The scale may be repairable per horizon on the released checkpoint, but on a model that never saw the test episodes the evidence is mixed (§6.7), and per dimension no ordering reaches significance after multiplicity correction once the coupling between dimensions is respected (§6.5). As the scalar the method applies, ensemble disagreement gets the order right and the size wrong: it should not be read as a scale, and a ranking use deserves its own validation on the deployment distribution (Appendix S).

What should travel from this paper, and what should not. The findings are of three kinds. *Properties of the objective and of how its bounds are built* are the ones to expect elsewhere, and only where both are present together: §6.3 derives the $\sigma = 0$ optimum from squared error on a sampled prediction through a bounded head whose floor nothing pushes back up, so it should hold wherever both are present, the hypothesis §2 states and leaves untested outside this codebase. On §2's count the substitution is present in 1 of the 10 descendants examined, and there only in a non-default mode; the survey did not examine the floor, so that is an upper bound on how often both are. *Properties of this released artifact* should be checked rather than assumed in any other: the one-step misalignment in its evaluation harness (§7.2), the trunk its five members share (§6.4), and its specific overconfidence factors, 11,683× for the aleatoric σ and 33.4× for the epistemic term at h = 100 (§6.2), are measurements of one checkpoint. *Properties of this dataset and its arenas* are limits on what could be resolved, not findings in either direction, and neither pinned repository can lift them (§3): our arms' held-out arena holds 4 independent 400-step trajectories and all ten episodes hold 20, which is why §11 leaves the replication at ensemble size 5, the free-baseline margin and capacity's share of the trunk-sharing effect open rather than settled. §5.2's configuration verdict should not travel either: it is resolved, but at our data budget, and says nothing about the original's.

13 Data and code

The full repository — code, every artifact under `results/`, and `FINDINGS_LEDGER.md` with the complete claim record including the retractions — accompanies this submission as anonymised supplementary material, and will be released under a permanent archival identifier on acceptance. Neither upstream repository is redistributed; `setup.sh` fetches both at pinned commits and verifies two SHA-256 hashes.

The repository's history was rewritten once, and §8 depends on that history, so we say what changed. A supplementary file quoting private correspondence was committed and briefly published before consent to quote it had been given; it was purged from the history rather than merely deleted, because a deletion commit leaves the content recoverable from a public repository indefinitely. Purging a path rewrites every commit from the one that introduced it onward, so **33 of the 35 commits Figure 1 cites keep their identifiers and two do not**: the two made between that file's introduction and the purge. Timestamps, content and ordering are unchanged; only the hashes moved, and for that reason Figure 1 finds its first rules' commits by subject and the later ones by the commit that introduced their ledger heading, never by a stored hash alone. The transcript itself reaches reviewers in the anonymised supplementary archive, which is not published.

The pre-registration argument in §8 rests on commit timestamps, and those are author-settable via `git commit --date`. That matters, because §8 is load-bearing. Two things address it. The supplementary material includes an anonymised `git log` covering every commit cited here, so the ordering in Figure 1 is checkable at review time. In the submitted paper and bundle, commit identifiers are replaced by stable anonymous labels, with ordering, timestamps and subjects unchanged, and the map from label to identifier is disclosed on acceptance. And **the repository was archived by a third-party archive before submission**, under a permanent identifier whose visit timestamp is not author-controllable. Neither the identifier nor the date of that visit appears here: both resolve to a named repository, and a date is a one-field lookup away from an origin. They are disclosed on acceptance.

What that archive establishes should be stated precisely, because it is easy to overclaim. It does **not** prove any individual commit date is genuine. It proves that the repository, with the whole pre-registration history in the form this paper cites, existed no later than that archival moment, as recorded by a third party with no interest in the claim — so nothing in the record can have been back-dated afterwards. That bounds §8 rather than proving it, and a reviewer should read it as such.

On anonymity. The code and data for this work are public, as they are for most reproducibility work, and the submission is anonymised: its bundle is scrubbed, asserted clean of a deny-list, and excludes the files that carry identity. Why the repository stays public, which §8's checkability depends on, is recorded in `docs/DOUBLE_BLIND_DECISION.md`.

14 References

1. C. Li, A. Krause, M. Hutter. *Robotic World Model: A Neural Network Simulator for Robust Policy Optimization in Robotics.* arXiv:2501.10100**v1**, 17 January 2025. *Read at v1, whose Roman-numeral sectioning our references follow; v2 (23 April 2025) renumbered to Arabic and moved IV-C into Appendix A.4.1.*

2. C. Li, A. Krause, M. Hutter. *Uncertainty-Aware Robotic World Model Makes Offline Model-Based Reinforcement Learning Work on Real Robots.* arXiv:2504.16680**v1**, 23 April 2025. *Read at v1; now at v3, last revised 8 Jan 2026. §5.1 and Eq. 4–5 keep their numbers there; every figure and appendix table has moved, and the model is renamed RWM-O to RWM-U — the same model, with the letter re-expanded from "Offline Robotic World Model" to "Uncertainty-Aware Robotic World Model". The two names never co-occur: v1 uses RWM-O 39 times and no RWM-U, v3 the reverse. The crosswalk is in* `results/original_paper_figures.json`.

3. Z. Abbas, S. Sokota, E. J. Talvitie, M. White. *Selective Dyna-style Planning Under Limited Model Capacity.* ICML 2020. arXiv:2007.02418, 2020.

4. K. Chua, R. Calandra, R. McAllister, S. Levine. *Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models.* NeurIPS 2018. arXiv:1805.12114, 2018.

5. S. Fort, H. Hu, B. Lakshminarayanan. *Deep Ensembles: A Loss Landscape Perspective.* arXiv:1912.02757, 2019.

6. C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. *On Calibration of Modern Neural Networks.* ICML 2017. arXiv:1706.04599, 2017.

7. D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, J. Davidson. *Learning Latent Dynamics for Planning from Pixels.* ICML 2019 (PMLR 97). arXiv:1811.04551, 2019.

8. D. Hafner, T. Lillicrap, M. Norouzi, J. Ba. *Mastering Atari with Discrete World Models.* ICLR 2021. arXiv:2010.02193, 2021.

9. M. Havasi, R. Jenatton, S. Fort, J. Z. Liu, J. Snoek, B. Lakshminarayanan, A. M. Dai, D. Tran. *Training independent subnetworks for robust prediction.* ICLR 2021. arXiv:2010.06610, 2021.

10. M. Janner, J. Fu, M. Zhang, S. Levine. *When to Trust Your Model: Model-Based Policy Optimization.* NeurIPS 2019. arXiv:1906.08253, 2019.

11. R. Kidambi, A. Rajeswaran, P. Netrapalli, T. Joachims. *MOReL : Model-Based Offline Reinforcement Learning.* NeurIPS 2020. arXiv:2005.05951, 2020.

12. V. Kuleshov, N. Fenner, S. Ermon. *Accurate Uncertainties for Deep Learning Using Calibrated Regression.* ICML 2018. arXiv:1807.00263, 2018.

13. B. Lakshminarayanan, A. Pritzel, C. Blundell. *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles.* NeurIPS 2017. arXiv:1612.01474, 2017.

14. S. Lee, S. Purushwalkam, M. Cogswell, D. Crandall, D. Batra. *Why M Heads are Better than One: Training a Diverse Ensemble of Deep Networks.* arXiv:1511.06314, 2015.

15. C. Lu, P. J. Ball, J. Parker-Holder, M. A. Osborne, S. J. Roberts. *Revisiting Design Choices in Offline Model-Based Reinforcement Learning.* ICLR 2022 (Spotlight). arXiv:2110.04135, 2022.

16. A. Malik, V. Kuleshov, J. Song, D. Nemer, H. Seymour, S. Ermon. *Calibrated Model-Based Deep Reinforcement Learning.* ICML 2019 (PMLR 97:4314-4323). arXiv:1906.08312, 2019.

17. Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. V. Dillon, B. Lakshminarayanan, J. Snoek. *Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift.* NeurIPS 2019. arXiv:1906.02530, 2019.

18. M. Seitzer, A. Tavakoli, D. Antic, G. Martius. *On the Pitfalls of Heteroscedastic Uncertainty Estimation with Probabilistic Neural Networks.* ICLR 2022. arXiv:2203.09168, 2022.

19. Y. Wen, D. Tran, J. Ba. *BatchEnsemble: An Alternative Approach to Efficient Ensemble and Lifelong Learning.* ICLR 2020. arXiv:2002.06715, 2020.

20. T. Yu, G. Thomas, L. Yu, S. Ermon, J. Zou, S. Levine, C. Finn, T. Ma. *MOPO: Model-based Offline Policy Optimization.* NeurIPS 2020. arXiv:2005.13239, 2020.

Entries 3–20 are the bibliography of §2 and §5.3, generated from `results/t1_bibliography_verified.json`*. Each was checked against the paper itself: title and full author list from the arXiv record, venue from the record or the paper's first page, and every sentence this paper attributes matched verbatim against that paper's own text: 18 of 18 entries and 19 of 19 attributed fragments, 3 of them single common words whose match verifies nothing about the attribution (ledger* `D-35`*). No entry was added that was not verified.*

A Verification chain

What every downstream number rests on. Each level was passed before the next was attempted.

level	claim	result
shapes	parameter counts match the reference	exact
wiring	inference outputs match the reference module	**0.000e+00**, bitwise
indexing	the harness feeds the actions it claims	bitwise against the raw CSV
residual	the zero-delta model is the hold-last floor	1.192e-07
objective	**losses and gradients match**	**0.000e+00 across 7 terms, 106 tensors**
trainer	can memorise a single batch	1,506× loss reduction

B Reproducing

```
./setup.sh                      # clone upstreams at pinned commits
python3.11 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
./reproduce.sh --quick --force # everything except training
```

`--force` matters: a clean clone already contains each stage's declared output, so without it every stage skips.

Runtime. Training stages are excluded by `--quick`, which is what makes the quick path practical. Training all 33 runs takes **49.8 hours** of recorded wall clock on two CPU cores: 19.7 hours for the 6 runs at 10,000 iterations and 30.1 for the remaining 27 at 2,500. The longest single run is 4.4 hours. Every figure here is read from the `wall_clock_s` field of the run artifacts rather than estimated.

5 of those 33 runs, 1.9 hours, are `M-49`'s capacity-matched arm at `rnn_hidden_size 124` rather than the released 256. They are part of this project's CPU spend and are counted in the total above — the other 47.9 hours are the 28 runs at the released width, and the two parts are asserted to make the total rather than stated beside it. They are **not** part of the 28 runs of §6.3's σ-collapse family, because §6.3's rate is a property of one architecture and mixing widths into it would make "nearly identical across runs" a claim about two different models. Every run artifact records the width it trained at, and `paper_numbers.py` selects the collapse family by that field rather than by filename; `docs/BUILD_CHECKS.md`, shipped as supplementary, records how that selection came to be made.

The pre-submission runs are counted separately. §5.2's sweep and §5.3's baselines added 42 runs and **28.3 hours**: 24 sweep runs (13.4 h), then 9 teacher-forced and 9 autoregressive baseline runs (7.9 h and 7.0 h), read from `results/presubmission_runtime.json`. They are outside the total above, whose remainder that paragraph describes as released-width runs, which the baselines are not. Of these, 8 overlapped other CPU work a session logged, which inflates their wall clock and changes no weight; the artifact lists each overlap. Rule X1's Part C (§5.3) ran 2 more teacher-forced RSSM runs from a second queue, 4.2 hours, which are in neither total; 1 of them overlapped CPU work a session logged. They are the table's last rows. Per run family, every run at 2,500 iterations (the sweep's centre is §5's Arm A, whose runs are counted above):

run family	runs	hours per run, mean	runs that overlapped other CPU work
(1, 8) (§5.2)	3	0.15	0
(2, 8) (§5.2)	3	0.16	0
(8, 8) (§5.2)	3	0.30	1
(16, 8) (§5.2)	3	0.51	2
(32, 1) (§5.2)	3	0.51	0
(32, 2) (§5.2)	3	0.74	3
(32, 16) (§5.2)	3	0.78	1
(32, 32) (§5.2)	3	1.32	1
MLP, teacher-forced (§5.3)	3	0.04	0
RSSM, teacher-forced (§5.3)	3	1.88	0
transformer, teacher-forced (§5.3)	3	0.71	0
MLP, autoregressive (§5.3)	3	0.06	0
RSSM, autoregressive (§5.3)	3	1.53	0
transformer, autoregressive (§5.3)	3	0.74	0
RSSM, teacher-forced, PlaNet's KL settings (rule X1, §5.3)	1	2.11	1
RSSM, teacher-forced, DreamerV2's layer-normalised cell (rule X1, §5.3)	1	2.10	0

C What testing the untested claims would require

Appendix D's first table marks 6 claims tested and the rest not. "Not tested" is an apology unless it comes with a price, so here is what each would cost. We give compute orders where we can estimate them honestly from this project's own measurements and say so where we cannot.

Every row needs what this reproduction did not have: interaction. Our arms train on the released CSV, which is a recording. Those claims need a policy acting in an environment, simulated or real, and the environment responding. In simulation that means Isaac Lab, which needs an RTX-class NVIDIA GPU, and no amount of CPU substitutes: the reference's data generation is GPU-parallel simulation, not a data-loading problem. For the hardware-transfer and real-robot rows the binding constraint is a robot rather than compute. **A GPU would not be enough on its own.** The code that generates data is in neither repository this reproduction pins: the only code that touches the file reads it (`train.py:44` in the lite release), the lite release's environment rolls the learned model forward rather than physics, and its readme places collection in a third repository, the authors' Isaac Lab extension (`readme.md:13`), which this reproduction does not pin and which would need Isaac Lab and an RTX-class GPU (ledger `D-36`).

untested claim	what it needs	order
Sample efficiency, 6,000,000 against ~250M transitions (§IV-E)	Isaac Lab, an RTX-class GPU, the MBPO-PPO loop, and a PPO baseline run to convergence for the comparison	the reference reports 6,000,000 pretraining transitions and 50 min of RWM training on their hardware; the PPO baseline's 250M is the dominant cost
MBPO-PPO beats SHAC and Dreamer (§IV-E)	the above, plus SHAC and Dreamer implementations at matched budgets	three policy-learning stacks, each tuned enough that the comparison is fair — the largest engineering item here
Zero-shot hardware transfer (§IV-E)	all of the above, plus an ANYmal, a safe test area, and the sim-to-real stack	not estimable in compute; the binding constraint is hardware access, not GPU hours
Generality across quadruped, humanoid, manipulation (§IV-D)	recorded state-action data from a humanoid and a manipulator, which means Isaac Lab and a policy in each environment to generate it — the released CSV is one robot on one terrain	one data-generation run per morphology, plus a world-model training campaign each on the scale of ours, 33 runs in 49.8 h; the model training is the cheap half and the data is not
Offline MBRL on real robots (2504.16680v1)	a real robot, a logged dataset from it, and the offline MBRL loop	not estimable in compute; hardware access again, and a claim the follow-up itself states as prospective

untested claim	what it needs	order
Whether the penalty improves the learned policy (2504.16680v1 §5)	Isaac Lab, the MOPO-PPO loop, and at minimum an ablation with the penalty weight at zero	one policy-learning stack; the cheapest of the rows that need a simulator, and the one that would bound §11's open question about what the miscalibration costs

Two claims this table once priced as within reach of this setup have since been run: the configuration claim (§5.2) and the architecture claim (§5.3), 42 runs and 28.3 h of CPU training between them (Appendix B). The six that remain all need a simulator or hardware, and no amount of care with the released CSV changes that.

What we would do first. The penalty ablation. It is the cheapest of the simulator-requiring items, it bears directly on a limitation §11 states our measurements cannot settle — whether the miscalibration we document costs anything downstream, where §11's policy-free proxy tests only whether the per-horizon correction reorders the penalty component and measures no cost — and it needs no robot.

D Every claim of the originals, and what we did with it

The body's §4 summarises the first table. It is here in full because the third column — what the original actually reports — is the answer to a question a reader of any reproduction should ask, and because "no quantitative figure" is itself a finding that deserves to be checkable row by row. Its verdict cells quote figures from the sections they cite, each on that section's arena, n_independent and checkpoint; the second table gives them claim by claim.

*Section references follow arXiv:2501.10100**v1**, which uses Roman-numeral sectioning. v2 renumbered to Arabic and moved IV-C's material into Appendix A.4.1. References to arXiv:2504.16680 follow **v1**, which is the version we read; it is now at v3 (8 Jan 2026), where §5.1 and Eq. 4–5 keep their numbers but every figure and appendix table has moved — Figure 2 (right) became Figure 3 (right), and the model was renamed RWM-O to RWM-U, which is a re-expansion of the letter ("Offline Robotic World Model" to "Uncertainty-Aware Robotic World Model") rather than a second variant: no version of the paper contains both names. All locations, and the occurrence counts that establish that, are recorded in `results/original_paper_figures.json`.*

claim, and where	tested	what the original reports	verdict
RWM-AR consistently outperforms RWM-TF (2501.10100 §IV-D)	**yes**	**no quantitative figure.** "significantly outperforms"; the gap is plotted in Fig. 7 and stated nowhere in text, caption or table	**reproduces** at long horizon (§5)

claim, and where	tested	what the original reports	verdict
Teacher forcing gives "poor autoregressive performance" (§IV-C)	**yes**	**quantitative**: Fig. 6 prints e for the teacher-forced N=1 row, 3.99 at (32, 1) against 0.47 at the centre, on unstated data and horizon; the passage itself gives no number	reproduces, and more strongly: on the 400-step unit Arm B is worse than the hold-last floor at every horizon (on M-64's shorter units only at the longest they reach), and §5.2's (32, 1) is 6.35× the centre's error at h = 368
M=32, N=8 gives the optimal trade-off between accuracy and training time (§IV-C)	**yes**	**quantitative**: Fig. 6 prints the error and the training hours of every cell of its grid, and the centre's error is the tied-lowest (`docs/presubmission/ORIGINAL_SPECS.md` a.5)	**NOT OPTIMAL AT OUR BUDGET** on accuracy alone (§5.2): (32, 32), (32, 16), (8, 8) and (2, 8) beat it at our budget (every reading its rule reports returns that verdict but held-out nRMSE at h = 1, which is unresolved), but the ranking depends on training length: trained longer, the centre passes each of them on at least one reading (post hoc, §5.2, Appendix U). The trade-off with training time is not tested; the original's printed hours are consistent with it
Beats MLP, RSSM and transformer baselines (§IV-D)	**yes**	**no quantitative figure.** "consistently achieves the lowest prediction errors across all environments"; plotted in Fig. 7, with no number in text, caption or table	**REPRODUCES** with the baselines teacher-forced, as the original trains them, and **RWM AHEAD OF ALL THREE** with them trained autoregressively, which compares architectures at one training regime rather than testing this claim (§5.3). Both are the rules' verdicts at h = 368; from h = 32 and h = 128 respectively, every reading the rules report gives them, and at shorter horizons they split. The baselines are built to our reading of Table S7 and trained with RWM's settings; at h = 368 every one is worse than predicting no change, and rule X1's other read-outs and retraining variants did not rescue our RSSM (**NOT RESCUED BY THE SETTINGS TRIED**), so the RSSM comparison stays uninformative and the claim rests on the MLP and the transformer.
Zero-shot hardware transfer (§IV-E)	no	—	One robot where the original has several `[hardware: zero-shot transfer]` no hardware; this is a dynamics-model reproduction
Policies transfer to hardware from ~6M state transitions against ~250M for the model-free baseline (§IV-E) — the paper's headline sample-efficiency result	no	**6M against 250M state transitions** at equal real tracking reward (0.90 +- 0.04 against 0.90 +- 0.03), Table I — the only table of numbers in either paper	`[policy, hardware: the sample-efficiency result]` **not tested.** It is a claim about policy learning and hardware deployment, and requires the RL loop, a simulator and an ANYmal. We reproduce the dynamics model only; no policy is trained anywhere in this work, so no transition count of ours is comparable
MBPO-PPO beats SHAC and Dreamer (§IV-E)	no	—	`[policy: the comparisons against SHAC and Dreamer]` no policy learning reproduced

claim, and where	tested	what the original reports	verdict
Generality across quadruped, humanoid, manipulation (§IV-D)	no	plotted in Fig. 7; no numbers in text	`[model: generality across robot morphologies]` one released dataset, ANYmal D flat
Epistemic "closely follows the trend of the prediction error", justifying "its role as a trust metric" (2504.16680v1 §5.1)	**yes**	**no quantitative figure.** A "strong correlation" is asserted with no coefficient, interval or sample size; plotted in Fig. 2 (right)	**supported as a scalar ranking, against a real baseline** — the applied scalar correlates +0.605 [+0.545, +0.694] with realised error at n_independent = 20, beats the forecast-index counter at every horizon — though **not** the free `step-size` counter, +0.4697 against +0.6053, a margin below the 0.2891 minimum detectable effect (SURVIVES entry-res ONLY: disagreement beats the one-step error before the window, but not the model's predicted step size) — and survives 5 controls on forecast depth — the linear partial that keeps +0.596, and four harder ones — plus a sixth on trajectory difficulty — the last giving +0.419 [+0.318, +0.576] with both the rollout and the depth held constant (§6.6, M-45). **Weaker per-dimension**: at h = 368 the 45-of-45 sign count gives a permutation P of 0.0435 (out-of-sample) and 0.0775 (in-sample), and no cell survives multiplicity correction (§6.5). **Not supported as a scale**: 33.4× overconfident at h = 100, the method's own rollout length, and 34.4× at the h = 368 diagnostic horizon; a per-horizon rescaling brings the released checkpoint near nominal, but on a model that has not seen the test episodes it does so in only 17 of 36 cells (§6.7)
Aleatoric "remains low, reflecting small stochasticity" (2504.16680v1 §5.1)	**yes**	**no quantitative figure.** "Low" is relative to the epistemic curve on the same axes of Fig. 2 (right); no absolute value, and no comparison against realised error	the observation holds; the explanation does not (§6.3)
Offline MBRL on real robots (2504.16680v1)	no	—	`[policy, hardware: offline MBRL on real robots]` not tested
Penalising rewards by ensemble disagreement improves the learned policy (2504.16680v1 Eq. 4–5, §5) — the follow-up's core method claim	no	Fig. 3 (right) plots epistemic uncertainty under three penalty weights during training; no numbers	`[policy: the core claim that penalising rewards by disagreement improves the learned policy]` **not tested.** We measure the penalty quantity itself — what it is (§6.1), how well it ranks error (§6.6), whether it is calibrated (§6.2) — but never train a policy with or without it. Our findings bound what the quantity *reports*, not what it *costs* (§11)

What each tested claim rests on. The second table is §3.2's: one row per headline claim of this paper, with the section that owns it. Each row gives its own arena and number of independent units, its checkpoint, whether that arena is in-sample for the model measured, the verdict as its rule or analysis returned it, and

whether it survives multiplicity correction. It is generated from the artifacts each claim is computed from (`results/evidence_summary.json`).

claim (§)	arena (n_ independent)	checkpoint	in-sample for the model measured?	verdict	survives multiplicity correction?
Autoregressive training beats teacher forcing at h = 368 (§5)	out-of-sample (4)	10,000 iterations	no	gap excludes zero, favouring autoregressive training	not applicable at 10,000; the 500- and 2,500-iteration cells survive Holm (Appendix L)
The same comparison reverses at h = 1, at the short unit M-64 built (§5)	out-of-sample (60)	10,000 iterations	no	gap excludes zero, favouring teacher forcing	not applicable
(M, N) = (32, 8) is the optimal configuration on accuracy alone (the accuracy half of the trade-off) (§5.2)	out-of-sample (4)	2,500 iterations	no	NOT OPTIMAL AT OUR BUDGET; 4 of 8 neighbours beat the centre; every other reading the rule reports returns the same verdict but held-out nRMSE at h = 1, which is unresolved; the ranking depends on training length (Appendix U)	yes
RWM beats MLP, RSSM and transformer baselines trained with RWM's settings, at h = 368 (teacher-forced, as the original trains them; the RSSM comparison uninformative, rule X1) (§5.3)	out-of-sample (4)	2,500 iterations	no	REPRODUCES; every reading the rule reports returns it from h = 32, and below that the readings split; at h = 368 every baseline is worse than predicting no change	yes

claim (§)	arena (n_ independent)	checkpoint	in-sample for the model measured?	verdict	survives multiplicity correction?
RWM beats MLP, RSSM and transformer baselines trained with RWM's settings, at h = 368 (trained autoregressively: architecture at one training regime, not the original's claim; the RSSM comparison uninformative, rule X1) (§5.3)	out-of-sample (4)	2,500 iterations	no	RWM AHEAD OF ALL THREE; every reading the rule reports returns it from h = 128, and below that the readings split; at h = 368 every baseline is worse than predicting no change	yes
Ensemble disagreement is smaller than realised error, at h = 1 (§6.2)	all ten episodes (20)	released	yes	overconfident; the ratio interval excludes 1	not applicable
Ensemble disagreement is smaller than realised error, at h = 100 (§6.2)	all ten episodes (20)	released	yes	overconfident; the ratio interval excludes 1	not applicable
The aleatoric σ head has collapsed and is orders of magnitude smaller than realised error, at h = 1 (§6.2)	all ten episodes (20)	released	yes	overconfident; the ratio interval excludes 1	not applicable
Disagreement ranks realised error better than the forecast step index, at h = 100 (§6.6)	all ten episodes (20)	released	yes	paired difference excludes zero	not applicable
Disagreement ranks realised error better than the model's own predicted step size (§6.6)	all ten episodes (20)	released	yes	SURVIVES entry-res ONLY (disagreement beats the one-step error before the window, but not the model's predicted step size): step size not beaten, the margin below the minimum detectable effect; the partial survives	could not at this n
With both the rollout and the depth held constant, disagreement still tracks error (§6.6)	all ten episodes (20)	released	yes	interval excludes zero and clears the minimum detectable effect	not applicable

claim (§)	arena (n_ independent)	checkpoint	in-sample for the model measured?	verdict	survives multiplicity correction?
A per-horizon multiplier brings coverage near nominal where a constant one does not (§6.7)	held-out pair (4)	released	yes	every released-checkpoint point estimate unseen by the multiplier within tolerance; on Arm A at 2,500 iterations, unseen by its model too, 17 of 36 epistemic cells; tolerance not resolvable at this arena	could not at this n
An ensemble that shares no trunk is better calibrated than the released topology (§6.8)	out-of-sample (4)	2,500 iterations	no	MECHANISM SUPPORTED	not applicable
The same contrast at matched capacity (§11)	out-of-sample (4)	2,500 iterations	no	UNDER-POWERED — favours the matched ensemble by less than the MDE	could not at this n
Independence and the corrected objective together improve on the released topology (§6.8)	out-of-sample (4)	2,500 iterations	no	THE COMBINATION IMPROVES CALIBRATION	not applicable
The released evaluation pairs states and actions one step stale (§7.2)	all ten episodes (20)	released	yes	defect confirmed in the code; raises error up to h = 32 on relative-L1; not resolved from h = 100	not applicable

E Every pre-registered rule, its lead time and its verdict

§8's argument rests on decision rules committed to git before the data that tested them, and the body names those rules by identifier. An identifier with no table behind it is either decoration or an instruction to open a 651 KB ledger, so here is the table. It is generated from `FINDINGS_LEDGER.md` and `results/appendix_g_rules.json`; nothing in it is typed.

Lead time is the rule's commit timestamp subtracted from the commit that first held the data it tested, never by a stored hash: the first rules' commits are found by *subject*, because the history was rewritten once and two of the commits Figure 1 cites changed hash (Data and code), and the later rules' by the commit that introduced their ledger heading. Positive means the rule was in git before the data existed. This is the same computation Figure 1 plots.

rule	what it governs	commit	lead time	tested by	verdict
`M-16`	The Arm A / Arm B comparison	`C004` Step 5: pre-register the decision rule before launching any main run	+1.3 h	first main-run data	CANNOT BE SETTLED AT THIS BUDGET

rule	what it governs	commit	lead time	tested by	verdict
M-22	Whether episode difficulty biases the A/B comparison	C013 Pre-register the Task 4b difficulty-bias rule, and the two-arena convention	+5 min	M-16 re-evaluated	RESOLVED — branch 1, 4c not run
M-23	The 10,000-iteration comparison	C016 5.1: pre-register M-23, the long-horizon decision rule	+2 min	10k runs launched	RESOLVED — reproduces at long horizon
M-43	The ensemble-5 replication	C081 PRE-REGISTER the ensemble-5 replication rule, before the runs exist	+13.3 h	ens5 result committed	DOES NOT GENERALISE
M-44	The trunk-sharing mechanism	C090 PRE-REGISTER M-44 and M-45, with the power check M-43 was committed without	+6.4 h	R2 result committed	MECHANISM SUPPORTED
M-45	The within-trajectory control	C090 PRE-REGISTER M-44 and M-45, with the power check M-43 was committed without	+4.4 h	A2 result committed	DISAGREEMENT CARRIES WITHIN-ROLLOUT INFORMATION
M-49	Pre-registered: does trunk-sharing survive capacity matching?	C123 M-49: pre-register the capacity-matched test, with the MDE that bounds it	+9.4 h	results/m49_ capacity_ matched.json	UNDER-POWERED — favours the matched ensemble by less than the MDE
M-50	Pre-registered: is the sigma collapse driven by the objective, on data whose noise is known?	C127 M-50 pre-registered, and a different-architecture run stopped contaminating §6.3	+5 min	results/e5_ synthetic_ sigma.json	OBJECTIVE-DRIVEN
M-51	Pre-registered: does the ranking claim survive more than one free adversary?	C128 M-51 pre-registered: two more free adversaries for the ranking claim	+12 min	results/e7_free_ baselines.json	SURVIVES entry-res ONLY
M-52	M-51 named a quantity that does not exist, and what replaced it	C129 M-52: M-51 named a quantity that does not exist, corrected before it ran	+9 min	results/e7_free_ baselines.json	SURVIVES entry-res ONLY
M-62	Pre-registered: does any headline verdict change when the bootstrap resamples episodes rather than 400-step trajectories?	C154 M-62 to M-65: four rules committed before the data that will test them	+5.1 h	results/m62_ episode_ clustering.json	NO MOVE
M-63	Pre-registered: is the h=1 coverage failure uniform across the 45 state dimensions, or carried by a few?	C154 M-62 to M-65: four rules committed before the data that will test them	+5.1 h	results/m63_per_ dimension_ coverage.json	UNIFORM

rule	what it governs	commit	lead time	tested by	verdict
M-64	Pre-registered: does the horizon-scoped power increase from shorter evaluation units change any verdict at h $\leq$ 128?	C154 M-62 to M-65: four rules committed before the data that will test them	+6.0 h	results/m64_ short_units.json	MOVES, at one cell
M-65	Pre-registered: is the Gaussian nominal of 68.27% defensible, or is the error distribution heavy-tailed?	C154 M-62 to M-65: four rules committed before the data that will test them	+5.1 h	results/m65_ gaussian_ nominal.json	GAUSSIAN NOMINAL ADEQUATE
M-68	The combined arm: independence and the corrected objective together	C180 Pre-register M-68: the combined arm's rule, its verdict branches and its minimum detectable effect	+5.1 h	results/r2_ combined_ arm.json	THE COMBINATION IMPROVES CALIBRATION
M-69	The cross-model transfer of the per-horizon multiplier table	C182 Pre-register M-69: the cross-model transfer rule, its criterion and its minimum detectable effect	+23 min	results/task_d3_ cross_model.json	DOES NOT TRANSFER — A PROPERTY OF THE MODEL
M-70	Whether the per-horizon correction reorders cumulative penalties	C206 Pre-register M-70: whether the per-horizon correction reorders cumulative penalties	+115.2 h	results/q3_ penalty_ reordering.json	DOES NOT REORDER
M-74	The configuration claim: is (M, N) = (32, 8) optimal?	C225 PRE-REGISTER the M/N sweep and architecture-baseline rules, before the runs exist	+29.4 h	results/mn_ sweep_verdict.json	NOT OPTIMAL AT OUR BUDGET
M-75	The architecture claim, baselines teacher-forced, as the original trains them	C225 PRE-REGISTER the M/N sweep and architecture-baseline rules, before the runs exist	+29.4 h	results/baselines_ verdict.json	REPRODUCES
M-76	The architecture claim, baselines trained autoregressively, as RWM is	C225 PRE-REGISTER the M/N sweep and architecture-baseline rules, before the runs exist	+29.4 h	results/baselines_ verdict.json	RWM AHEAD OF ALL THREE
M-80	X1: is our RSSM's long-horizon failure a matter of how its forecast is read, of its training settings, or of neither?	C301 PRE-REGISTER RSSM diagnostic X1, before its readings exist	+3 min	results/rssm_ diagnostics.json	NOT RESCUED BY THE SETTINGS TRIED
M-84	X2: do our trained models condition their forecasts on the action they are given?	C409 PRE-REGISTER action-sensitivity rule X2, before its readings exist	+7 min	results/action_ sensitivity.json	RESPONDS TO THE ACTION at 2,500 and at 10,000 iterations
S-12	"Task 3's duplication rule was pre-registered"	C025 Task 3: the duplication control confirms R-47's mechanism and refutes its statistic	-2.9 h	control runs finished 21:37:51	RETRACTED

M-69's discharge commit was amended 2 minutes after it was created, so its lead time is +21 min by that commit's author time and +23 min by its committer time. Both are positive, so the rule reached git before its data on either reading.

23 rules, 23 with a computed lead time, of which 22 are positive and 1 negative. **The negative one is kept deliberately.** S-12 withdraws the claim that the Task 3 duplication rule was pre-registered; the control runs had finished before any threshold reached git. A table that dropped it would be asserting exactly what the ledger retracts.

M-52 is in the table and that is deliberate. It is the one mid-flight amendment to a pre-registration in this project: M-51 named a baseline that does not exist in the artifact it named — the residual on the last teacher-forced step of the history window, which the rollout helper never computes because it copies the history rather than predicting it — and M-52 names the replacement, committed before the replacement's statistic was computed. A table of pre-registrations that omitted the one amendment would be a highlights reel, so the table's rows are selected by each entry's Status line and their count is asserted against the set scripts/ledger_check.py reports.

What each rule says, in its own committed words is in the supplementary material, as docs/APPENDIX_G_RULES.md — every rule's text unabridged, generated from the ledger by the same script that generates this table. Quoting all 23 in full here would add pages to an appendix whose job is to be checkable at a glance, and quoting them in part would ship quotations ending mid-sentence. The table is the claim; the supplementary is the evidence.

F The variance-state arithmetic behind §7.5

§7.5 states the conclusion. The arithmetic and the five assumptions it rests on are in docs/APPENDIX_G_VARIANCE_ARITHMETIC.md, shipped as supplementary, kept out of the body because the numbered claim it once supported is retracted (S-19) and because a forensic case the section then defuses with the author's own reply is not what a reader needs in the body of a reproduction.

So the finding is not that the release is internally inconsistent. It is that **the released artifacts do not reproduce the released checkpoint's variance state, and the author's account is that the released repository is not the one that trained it.** That is a **documentation gap between a release and a run**: common, worth recording, and much less interesting than an inconsistency. The σ collapse is linear in iteration count and its rate is nearly identical across our runs (§6.3), which makes it a clock, and read as a clock it puts the checkpoint's variance state out of reach of a constant-rate run from the released initialisation at the configured learning rate, at every iteration count the release, the paper and the checkpoint tag state. The first author's account is that the released repository is several revisions removed from the setup that trained the checkpoint, which supplies a mechanism, a warm start or a different log_delta_logstd initialisation, that would explain it with no inconsistency at all. We report the arithmetic because it is what let us detect the gap at all, not as a charge against the work.

G The two nRMSE aggregations, and the comparison one of them inverted

§3.1 states that this paper uses **form 1**: pool the per-dimension mean squared errors across the 45 state dimensions, then divide by the pooled scale — a ratio of means. **Form 2** is the mean of per-dimension ratios.

The two differ because the state dimensions differ widely in scale. Form 2 gives whichever dimension has the smallest denominator unbounded leverage over the aggregate, and a dimension that is nearly constant in the training episodes has a very small denominator. Form 1 has no such lever: a dimension contributes in proportion to its share of the total squared error.

Why this appendix exists rather than a sentence. The choice between the two once inverted a published-model comparison in this project's own history — a result that favoured us under one aggregation and did not under the other. The claim was withdrawn on our own evidence and is kept in

the record (`FINDINGS_LEDGER.md`). Form 2 figures appear nowhere in the body; they are retained in `results/step4_0a_results.json` for continuity with figures this project published before the inversion was found. This appendix gives the two definitions and the inversion rather than those figures, so continuity rests on that committed file and not on a deleted number.

The definition in force throughout the paper is the one §3.1 gives. Nothing in §5, §6 or §7 uses form 2.

H Findings not in the main text

The ledger tags some of its entries as contributions of this project. These are the confirmed ones the body does not state, or uses without stating the finding itself, each checked against its ledger entry. Code facts are cited by file and line at the pinned upstream commits; every measurement is read from the artifact named. Measurements are at 2,500 training iterations unless a row says otherwise or concerns the released checkpoint, and each row names its arena.

finding	evidence (ledger ID, artifact)	bearing on the paper
Paper-versus-code gaps		
No forecast decay factor exists: the training loss averages every forecast step with equal weight, though the paper describes a decay	C-09; `system_dynamics.py:186-231` and both config files	Our training does the same, so the loss is undecayed in the released code and in ours
The auxiliary heads (contact and termination) are trained on true next states, the state head on its own samples	M-13; `system_dynamics.py:217, 261`	In a rollout the auxiliary heads read predicted states they never trained on; this paper compares states only (§5.3) and does not measure them
Training feeds back a reparameterised sample, inference the mean	C-05; `system_dynamics.py:115, 217`	Our arms train and roll out the same way; for the released checkpoint, whose per-member σ has collapsed (§6.3), the sample is numerically the mean
Actions enter the network raw (the config's action mean and standard deviation are zeros and ones), and no action scale is recorded in either repository; measured against joint motion, they are not joint targets in radians	C-07, D-07; `anymal_d_flat_cfg.py`	A perturbation of one size means different things for actions and for states; no reported result perturbs either
The release's configuration, the paper (Table S9, 2,500) and the checkpoint's own tag state three different training lengths; the first author believes the configuration's is a typo	C-13; `base_cfg.py:97`, the checkpoint's `iter` field	§7.5: none of them reaches the released variance state at a constant rate
Pipeline defects		
The reset guard tests index *values* for truthiness, so a reset at the first row would be missed	B-02; `train.py:143`	Latent twice over: this dataset has no reset at its first row and marks none at all (§7.1)
The train/test split draws windows at random, and adjacent windows share all but one of their 40 rows, so the released test loss is not held out	B-03; `model_training.py:33`	The held-out arena (§3) is built from whole episodes and does not use this split
Measurements		
The data hold 21 plateaus of achieved base velocity, two per episode with a third in episode 7; the command itself is not recorded, and the plateaus match the simulator's command resampling	D-10; `results/step0_regimes.json`	A held-out episode is not a near-duplicate of a training one: the held-out pair tests generalisation across velocity commands, within one gait and terrain

finding	evidence (ledger ID, artifact)	bearing on the paper
`state_min_logstd`, on the slower gradient path C-11 identifies, drifts 5.2× slower than `log_delta_logstd`; extrapolated from one Arm A run's rates, the two imply about 150,000 and 270,000 iterations for the released variance state, an order of magnitude rather than a fitted count	R-25; `results/step6_3_min_logstd.json`	§7.5's conclusion does not rest on one parameter
Over overlapping trajectories on the held-out pair, which the released checkpoint trained on, its worst twentieth carry 92.6% of its squared error at h = 368 as each dimension's own scale weights it (form 2, which `g_z` dominates; Appendix G), and they start within two short ranges of rows, one per episode; on the 4 non-overlapping trajectories the largest is 1.2× the median	R-30; `results/taskAB_gate_r27.json`	A tail measured on overlapping trajectories can be one short stretch of data, which a count of independent trajectories (§3) exposes
The A/B relative-L1 gap at h = 368 (teacher forcing's error minus autoregressive training's, over 2 independent trajectories per episode) is positive on all ten episodes, from +0.73 up; episode 1, one of the held-out pair, gives +6.97, 2.5× the next largest, so the pair's +3.98 is 3.0× the other eight episodes' +1.33	R-39; `results/task4_arenas.json`	The direction is robust across episodes; a magnitude read from the held-out pair overstates the typical episode
Per state dimension, over the whole 368-step forecast, the released checkpoint loses to the hold-last floor on 18 of 45 across all ten episodes (20 independent 400-step trajectories) and on 8 on the held-out pair (4), including all three components of the gravity vector; Arm A at 10,000 iterations loses on 1 in each, `g_z`	R-45, and R-29 on overlapping trajectories; `results/task2_3_matched_trend.json`	Per dimension, a model trained from scratch fails far less often, even on the pair the checkpoint trained on and Arm A did not. In aggregate it does not: on the held-out pair the released checkpoint is ahead at h = 8 and level at h = 368 (R-45). One seed (seed 1, §11)
Over five checkpoints up to 10,000 iterations, the absolute A/B gap at h = 368 narrows as both arms improve (held-out pair +3.33 to +1.20, in-sample +1.72 to +0.87), while the ratio does not shrink (3.34× to 4.43×, in-sample 2.33× to 9.70×). The held-out values are not monotone: both peak at the second checkpoint (+6.74, 12.38×), one anomalous Arm B value	R-46; `results/task2_3_matched_trend.json`	An absolute effect quoted early overstates what remains, a ratio does not, and §5 reports ratios at 10,000 iterations; one seed (seed 1, §11)
Our trained models respond to the action they are given: fed another trajectory's actions, Arm A's error at h = 8 on its in-sample arena's 16 trajectories rises by +227.8% at 2,500 iterations and +676.3% at 10,000 (rule X2, pre-registered: RESPONDS TO THE ACTION)	M-84, M-85; `results/action_sensitivity.json`	A world model must respond to actions before it can serve policy optimisation; ours do on this test, though no policy is trained here (§7.2, Appendix V)

I The PETS lineage of the bounded σ head, and how often its descendants substitute the objective

Where the parameterisation comes from. The bounded log-σ head that §6.3 shows has its optimum at $\sigma = 0$ is largely not this codebase's invention: its clamp is inherited, line for line, from the probabilistic ensembles of Chua, Calandra, McAllister and Levine (PETS, NeurIPS 2018). PETS's Appendix A.1 gives

```
logvar = max_logvar - softplus(max_logvar - logvar)
```

```
logvar = min_logvar + softplus(logvar - min_logvar)
```

and `architectures/mlp.py:92-93` is those two lines in log-standard-deviation rather than log-variance, with `system_dynamics.py:302` supplying PETS's regulariser on the bounds.

What is **not** inherited is the objective, and one line of the bounds. PETS uses "the negative log prediction probability as our loss function", and that likelihood's log-σ term opposes $\sigma \to 0$; `system_dynamics.py:283` substitutes squared error on a reparameterised sample, which has no such term. And `architectures/mlp.py:91` builds the upper bound as the floor plus a learned positive gap, where PETS keeps the two bounds independent. §6.3 needs both changes: the substitution removes the log-σ term, and the tie is why the bound regulariser does not take its place, since the floor cancels out of it. In a descendant that kept PETS's independent bounds, the same regulariser pushes the floor up while the objective pulls σ down onto it, a case §6.3 does not derive and we have not tested (ledger `M-72`). So **a descendant of this lineage that replaced the likelihood with a sampled squared error, and left nothing pushing its variance floor back up, would inherit the same optimum.** That is a hypothesis about mechanism, untested in any other descendant (§11).

How often the substitution is made, which narrows the hypothesis. Of 10 public repositories examined on 20 September 2026 under the protocol in `results/q1_search_protocol.md`, 10 carry the construction and 1 of those trains it against a sampled squared error, and that one only in an optional value-aware mode; the other 9 keep PETS's likelihood (`results/q1_pets_descendants.json`). The protocol counts a repository only if it has the bounded head, learnable bounds, and a loss that squares the error of a *sampled* prediction; squaring the error of the predicted *mean*, an option several of these repositories offer, leaves σ untrained rather than driving it to zero, and does not count. We read "learnable" as "trainable by the code, by default or not", a reading settled after the survey, because in `mbrl-lib` and in `va_mbpo`, the one repository that inherits, the bounds train only when a caller switches that on (ledger `M-73`). The survey did not check how each repository treats its variance floor, so 1 of 10 is an upper bound on how often both conditions hold. The survey looked at 13 in all; the other 3 lack the construction, so the protocol does not count them as examined, among them mainline `rsl_rl`: the bounded head exists in the fork this paper pins, not in the library it forks, and we do not count the fork, since counting what §6.3 measured as evidence that the result travels would be circular. The protocol capped the survey at 25 repositories; it stopped at 10 examined, 13 looked at, and its notes give no reason. We wrote the protocol before the search, but neither it nor its hash reached git before the results did, so that order rests on our own record; it is a search protocol, not one of Appendix E's decision rules. Search engines rank and truncate, so this is a sample of convenience, but on it the substitution is rare, and a reader who takes the hypothesis to reach widely infers more than the evidence supports.

J The $\sigma = 0$ optimum: the synthetic-noise test (rule M-50), the term-by-term gradient check, and the runs behind the collapse rate

§6.3's derivation says the collapse happens on any dataset, and that is testable. It matters because it is what answers the follow-up's own explanation: on the released CSV, "small stochasticity in the environment" and our reading are observationally identical, since the data may simply be nearly deterministic. So under a rule committed before the runs (rule M-50, Appendix E) we built data where it is not.

Synthetic data whose true noise level is **known** and varies by a factor of 25 across the input range, with a non-constant true mean; the **same** bounded log-σ head as the released model — `MLPStateHead` unmodified, including the double-softplus clamp, the learnable `state_min_logstd` and `state_log_delta_logstd`, and the bound loss at its configured weight — trained under each objective in turn on 4,000 points for 12,000 iterations at 3 seeds. Nothing else differs between the arms.

Two ways this is not the released setting. The head is built here over a **one-dimensional** state with no recurrent trunk in front of it, where the released one predicts 45 dimensions from a GRU. The trunk's absence is deliberate: the question is about the head's objective, and a GRU would add a confound. The dimensionality matters because the state loss sums over state dimensions, so at one dimension it is roughly

45× smaller relative to the bound term than in the released path. That makes this setting *more* favourable to σ surviving, and the collapse happens anyway.

objective	median $\hat{\sigma}$ / σ_true	$\hat{\sigma}$ spread across the input range	slope of log $\hat{\sigma}$ on log σ_true
`mse` — the implemented branch	**0.0460**	1.002–1.003×	+0.000493
`gaussian_nll` — the authors' unused branch	0.9779	1.02–3.67×	+0.1330

Ratios and slopes are means over 3 seeds; spreads are the range across them, because the mean of a spread hides which seeds recovered.

Under the implemented objective σ sits 21.7× below the true noise and does not track it at all: a spread of 1.003× where the truth spans 25×, and a slope below the 0.00309 the design can detect. Under the authors' own branch, same data and same head, σ recovers the true level to a median ratio of 0.9779, and every one of the 3 seeds the rule was discharged over clears the slope threshold. **Twenty seeds show that clearance is not general**: 11 of 20 clear it, so the all-seeds criterion would not have held at that sample. The rule's verdict stands as returned over its own 3 and is not re-opened by more seeds (§8); what twenty establish is that the hedge below was necessary. **The recovering arm is seed-variable, and the rule said so before the runs**: its slopes span 0.0078–0.3537, a factor of 45, and its 3 seeds recover σ spreads of only 1.02–3.67× against the truth's 25×. So what this experiment establishes is the **contrast**, that one objective tracks the noise at all and the other does not, and not the magnitude of the recovery, which this training budget does not pin down. **OBJECTIVE-DRIVEN**, which is the verdict the rule names for that pattern.

The statistic is the slope, not the correlation, because a correlation is scale-free: a $\hat{\sigma}$ that is essentially constant still returns a large one off its own numerical noise. Under a permutation null, the same data with the input-to-noise pairing destroyed, a head whose σ spanned 1.0004× returned correlations as large as ±0.24, while its slope was 2e-05. The detection threshold is set at the slope corresponding to a 1.01× spread rather than at that noise floor, and the measured false-positive rate at zero signal is 0%.

What this does that §6.3's derivation alone could not. It removes the competing explanation rather than arguing against it: the stochasticity here is large, known and input-dependent, and the collapse happens anyway. The design's limit, stated in the rule, holds: the dilution ladder detects the signal at full strength and at no dilution below it, so this establishes that σ does not track the noise **at all**, not the magnitude of how badly.

§6.3's derivation covers two terms, and the objective has 7. Its completeness rests on the other 5 being inert with respect to σ, so each term is computed alone on one real batch (64 windows from the training episodes, at freshly initialised weights of the released architecture) and back-propagated alone, and the gradient reaching the log-σ tower, `state_log_delta_logstd` and `state_min_logstd` is recorded. A term that cannot move σ produces exactly zero on all three.

loss term	live?	weight	where the reference computes it	∂/∂ log-σ tower	∂/∂ `log_delta_logstd`	∂/∂ `min_logstd`
`state`	live	1.00	`system_dynamics.py:270-289`	0.000325	0.0509	0.0703
`sequence`	**dead**	1.00	`system_dynamics.py:274-277`	0	0	0
`bound`	live	1.00	`system_dynamics.py:301-302`	0	0.2	0
`kl`	**dead**	0.10	`system_dynamics.py:223`	0	0	0
`extension`	**dead**	1.00	`system_dynamics.py:233-268`	0	0	0
`contact`	live	1.00	`system_dynamics.py:233-268`	0	0	0
`termination`	live	1.00	`system_dynamics.py:233-268`	0	0	0

4 of the 7 configured terms are live at all under the released configuration: `sequence_loss` is dead code, guarded by a `prediction_type` the reference sets to `"single"` on both paths, and `kl` and `extension` are zero because their dimensions are. Of the 7, exactly 2 reach σ: `state` and `bound`. The remaining 5 produce a gradient of exactly zero, not merely a term the code suggests is irrelevant (`results/e4_sigma_gradients.json`).

We predicted the collapse from §6.3's algebra before training, then observed it. Three run counts appear below and they are not the same set. This project trained 33 runs for §5–§7, besides the 42 of §5.2 and §5.3

(Appendix B), of which 28 are at the released `rnn_hidden_size` of 256 and form the collapse family; the remaining 5 are the capacity-matched arm of rule M-49 (Appendix E) at width 124, a different architecture, excluded from every rate §6.3 quotes (Appendix B). Across all 28 runs of that family the log-σ range moves linearly in iteration count at a nearly identical rate within each objective, falling under sampled MSE and rising under the corrected one (Figure 4a). Rates are fitted on 22 of those 28: the 6 10,000-iteration runs continue seeds already counted at 2,500 and would double-weight them. Figure 4(a) shows all 28 runs of the collapse family and Figure 4(b) only the 22 the rate is fitted on, so the scatter and §6.3's quoted statistic describe the same set. The 33 runs, with the width column separating the collapse family from the capacity-matched arm:

arm	iterations	ensemble	objective	dataset	width	seeds	seed ids
Arm A	2,500	1	gaussian_nll	clean	256	5	0, 1, 2, 3, 4
Arm A	2,500	1	mse	clean	124	5	0, 1, 2, 3, 4
Arm A	2,500	1	mse	clean	256	5	0, 1, 2, 3, 4
Arm A	2,500	1	mse	contaminated	256	3	0, 1, 2
Arm A	2,500	1	mse	duplicated	256	3	0, 1, 2
Arm A	2,500	5	mse	clean	256	3	0, 1, 2
Arm A	10,000	1	mse	clean	256	3	0, 1, 2
Arm B	2,500	1	mse	clean	256	3	0, 1, 2
Arm B	10,000	1	mse	clean	256	3	0, 1, 2

K The per-dimension permutation tests behind §6.5

§6.5's P column is a permutation P, not a binomial one, and the binomial P-values an earlier draft attached to its counts are withdrawn (`S-15`). A binomial null treats the 45 state dimensions as independent trials, and they are not: position, velocity and torque for the same joint are physically coupled, and base linear and angular velocity are coupled through the gait. More importantly, error grows with rollout depth in every trajectory, so *any* σ that also grows with depth correlates with *any* trajectory's error, including one it was never paired with.

We therefore permute whole trajectories. The null pairs each trajectory's σ with a different trajectory's realised error, which keeps both marginal distributions and the entire cross-dimension dependence structure and destroys only the association under test. The correction is large, and largest exactly where we leaned hardest. The worst-affected cell is teacher-forced armB at h=368, in the in-sample arena. At h = 368 it moves from 5.68e-14 to 0.5565, a factor of about 10^{13}, because under a null that keeps the dependence a random re-pairing already yields 43.8 of 45 dimensions positive on average, so observing 45 of 45 is close to unremarkable. A fair coin centres the count at 22.5 of 45; the dependence-preserving null centres it between 5.1 and 43.8 depending on model, horizon and arena.

The larger arenas agree with each other against the smallest. At n_independent = 4 400-step trajectories on the held-out pair, in-sample for the checkpoint, the epistemic ordering looks strongest at long horizon (0.0417 at h=128, 0.0435 at h=368) and unremarkable at short (0.4348 at h=1). Both larger arenas invert that. In sample (n_independent = 16): 0.0052 at h=1, 0.0070 at h=8, against 0.3794 at h=128. Over all ten episodes (n_independent = 20): 0.0056, 0.0069 and 0.3762, with h = 100 at 0.2769, between h=32's 0.0344 and h=128's 0.3762. Two larger arenas at four and five times the sample say the effect is strongest at *short* horizon, though they are not independent: all 16 in-sample trajectories are among the 20.

The null means explain why, and the explanation is the one that motivates §6.6. At long horizon the shared forecast-depth trend lifts the null to 41.6 of 45 at h = 128, so a count of 45 is close to what chance alone delivers; at h = 1 the null sits near 15.4 and the same count is genuinely surprising. The out-of-sample arena is not wrong so much as blind: at 4 trajectories its smallest attainable P-value is 0.04167, so it cannot tell a strong effect from a marginal one at any horizon. **That blindness belongs to the 400-step unit, not to the arena**: a 33-row unit gives 60 independent units on the same two episodes, and 14 at h = 100 (rule M-64, Appendix E). We did not recompute this permutation family at the shorter unit, so what the short units change is the scope of the design claim, not any verdict: the arena is underpowered at h = 368, which

needs the full 400 rows, and is no longer demonstrably so at h = 128 and below. Running it there is one pass over stored rollouts, and we did not do it. The small arena's numbers are reported because it is the only arena out-of-sample for our own arms, not because it is the better measurement.

Nothing here survives multiplicity correction, in any of the three arenas. Holm–Bonferroni over each arena's 30 model × horizon cells at $\alpha = 0.05$ rejects 0 out of sample, 0 in sample and 0 over all ten episodes. Out of sample that is a property of the design: with 4 independent trajectories the smallest attainable P-value is 0.04167, which already exceeds the smallest Holm threshold 0.001667, so no effect of any size could have been rejected there. In sample the miss is real: the smallest P in the family is teacher-forced armB h=128 at 0.0027 against a threshold of 0.001667.

L Section 5's long-horizon cells, multiplicity, and the reimplementation beside the released checkpoint

At long horizons §5's pattern is consistent across the design. Under the cluster bootstrap, the out-of-sample gap excludes zero in **4 of 4** long-horizon cells, both trajectory lengths crossed with the 500 and 2,500-iteration checkpoints. These figures are relative-L1; the nRMSE aggregation is reported separately and does not change the direction.

Multiplicity. Those 4 cells sit in a family of 8 out-of-sample comparisons. All 4 of 4 still exclude zero at a Bonferroni level of 0.05/8, and Holm–Bonferroni rejects **4 of 4**. §5's sign test is unaffected either way.

How good the reimplementation is as a model, next to the artifact it reimplements. §5's tables compare two training rules with each other and §6.2's compares calibration, so neither puts the released checkpoint and our arms side by side on absolute accuracy. Both aggregations, for these models and for §5.3's architecture baselines, on one arena:

model	nRMSE h = 1	rel-L1 h = 1	nRMSE h = 8	rel-L1 h = 8	nRMSE h = 100	rel-L1 h = 100	nRMSE h = 368	rel-L1 h = 368
released checkpoint	0.0544	0.0563	0.0697	0.0808	0.5028	0.3304	0.9051	0.6041
Arm A — autoregressive, faithful MSE	0.1262	0.1232	0.3029	0.3289	0.4913	0.4798	0.5425	0.5856
Arm A — autoregressive, `gaussian_nll`	0.1204	0.1195	0.2867	0.3154	0.4603	0.4413	0.5245	0.5608
Arm B — teacher-forced	0.0879	0.0929	0.3064	0.3392	1.0363	1.0849	3.7927	4.5684
MLP, teacher-forced (§5.3) †	0.1393	0.1539	0.3624	0.4086	1.1399	1.2057	112.5344	145.1650
MLP, autoregressive (§5.3)	0.1364	0.1470	0.3016	0.3492	0.5452	0.5400	1.4383	1.4628
RSSM, teacher-forced (§5.3)	0.0707	0.0761	0.3393	0.3686	2.0376	2.2128	5.3890	6.3718
RSSM, autoregressive (§5.3) †	0.1223	0.1354	0.6004	0.6211	3.1128	3.2660	9.2508	10.6405
transformer, teacher-forced (§5.3) †	0.1202	0.1239	0.3773	0.3969	2.0195	2.0607	13.4730	15.1548
transformer, autoregressive (§5.3)	0.1329	0.1384	0.2562	0.3025	0.5821	0.5821	2.0110	1.8771
hold-last floor	0.0989	0.0796	0.4117	0.3298	0.9537	0.7558	1.0897	0.9930

Arena, stated once for the whole table: out-of-sample held-out pair, episodes 1 and 8, 4 non-overlapping 400-step trajectories, n_independent = 4; in-sample for the released checkpoint. Arm rows are the mean over 3 seeds at 2,500 training iterations (the `weights_2500.pt` checkpoint), with per-seed values in `results/head_to_head_accuracy.json`; nRMSE is form 1 (§3.1), and both metrics are cumulative over forecast steps 1..h. Every RWM row is read from the stored rollouts behind §6.2's calibration tables, so no model is run to build it. The architecture-baseline rows (§5.3) come from their own evaluator on the same four trajectories, whose relative-L1 reproduces the Arm A row exactly; their nRMSE is pooled as §3.1 defines it, recomputed afterwards with the same evaluator (post hoc, `results/pooled_nrmse_rescore.json`), and † marks a diverged row (§5.3). **This table is at 2,500 iterations and §5's by-horizon table at 10,000**, which is why Arm A's relative-L1 at h = 368 reads 0.5856 here and 0.3582 there: the same arm, trained longer.

Both metrics put the released checkpoint first at h = 1 and h = 8, they name different leaders at h = 100, and at h = 368 both put an Arm A variant ahead of it: the reimplementation is behind the artifact it reimplements at short horizons and ahead of it at the longest horizon we measure. That reading flatters the released checkpoint, because the split is ours: the arena is out-of-sample for our arms and in-sample for it, which trained on all ten episodes (the in-sample caveat of §3).

Stored per-trajectory values elsewhere. §6.8's and §11's paired contrasts at §5's n = 4 store their four per-trajectory values in `results/r2_independent_ensemble.json` and

`results/m49_capacity_matched.json`; §6.2's two held-out tables, at n_independent = 4, give intervals only, coarse for the same reason (the n = 4 caveat of §3).

M Section 6.2's reading checks: the resampling unit, the spread across dimensions, and the permutation column

Two pre-registered checks on how §6.2's numbers are read. The first (rule M-62, Appendix E) asks whether any verdict depends on resampling 400-step trajectories rather than whole episodes, which two trajectories share. It returns **NO MOVE**: in the one arena with power at that level, all ten episodes (n = 20 falling to 10), the pooled correlation's interval widens by 27% and the double-demeaned one's width changes by a factor of 0.98, and neither crosses zero. The other 4 cells it names are out-of-sample, where an episode bootstrap has n = 2 and three distinct resamples, or, for the per-horizon multiplier cell, one episode per direction and no interval at all; the rule said so in advance, and they are reported as uninformative rather than as intervals.

The second (rule M-63, Appendix E) asks whether the one-step failure is a few bad channels or all of them, since a pooled coverage is the unweighted mean of 45 per-dimension ones. It returns **UNIFORM**: the interquartile range across dimensions is 10.0 points against a 15-point threshold committed in advance, and the five worst dimensions carry well under half of the shortfall. **No channel is exempt.** The reading is coarse by construction: at h = 1 on 20 trajectories a per-dimension coverage moves in 5-point steps, a limit the rule fixed before the run.

The last column of §6.2's released-checkpoint table gives permutation P-values over whole trajectories, not binomial ones, on the same 20 trajectories as the counts beside them; Appendix K explains why a binomial null is inadmissible here. h = 100 is tested too, because it carries the abstract's headline figure. These are six tests on one family and none survives Holm–Bonferroni across the arena's 30 cells: the smallest is faithful (mse) h=368 at 0.0037 against a threshold of 0.001667. Read the column as a consistency check on direction, not as six independent findings.

N Section 6.6's robustness checks: depth controls, the decomposition, rule M-43, and the within-step control

What survives removing each confound. A linear partial is not much of a control: error does not grow linearly with depth, and a control that under-fits the index leaves index-driven variance in the residual and flatters disagreement. The first five rows after the pooled baseline below are **depth controls**, each partialling out a different model of how far into the rollout you are; the last four **decompose** the pooled figure into its between-trajectory and within-trajectory parts, and the last of those was pre-registered before it was computed (rule M-45, Appendix E). **One row is not comparable to the others**: the rank partial is a correlation of ranks rather than of values, so its being larger than the pooled figure says nothing about how much depth explains. Every row is the released checkpoint over all ten episodes, n_independent = 20 400-step trajectories.

what is removed	correlation	95% CI	what survives it
nothing (pooled)	+0.605	[+0.545, +0.695]	—
the step index, linearly	+0.596	—	depth explains 0.010 of it, from +0.605
log(1 + index)	+0.589	[+0.512, +0.688]	a non-linear model of depth
a cubic in the index	+0.582	[+0.495, +0.676]	any polynomial trend in depth
any monotone function of depth (rank partial)	+0.906	[+0.856, +0.921]	depth in any monotone form
depth exactly, within each forecast step	**+0.739**	[+0.711, +0.852]	positive at 368 of 368 steps, median +0.737[1]
forecast-step means only	+0.589	—	the within-step control, pooled
trajectory means only	+0.470	[+0.389, +0.604]	—
everything within a trajectory — the 20 trajectory means alone	+0.878	[+0.817, +0.956]	do harder rollouts disagree more?
both, additively (r_dd)	**+0.419**	**[+0.318, +0.576]**	**at a given depth, in a given rollout, does disagreement know?**

[1] Adjacent forecast steps on the same 20 trajectories are heavily dependent — structurally the same problem that §6.5's permutation null corrects for across the 45 coupled state dimensions (Appendix K). The count is descriptive; the interval [+0.711, +0.852] is the statistic, and no P-value attaches to 368/368.

The weakest figure across the 5 depth controls is +0.582, so disagreement is not re-encoding the clock: at a fixed depth it still knows which rollouts are going wrong. But depth was never the only confound. Per-episode difficulty spans 0.523 to 1.585 and is uncorrelated with commanded speed, so if harder trajectories simply have both larger error and larger disagreement, the pooled correlation would look exactly as it does with disagreement carrying no within-rollout information at all. Two things pointed there: the 20-point h = 1 figure of +0.994, which is not the shape of a genuine per-step signal, and the within-step control coming out *above* the pooled figure, the signature of a between-unit effect.

The verdict. The between-trajectory correlation is +0.878 and the two components contribute 51.7% and 48.3% of the pooled covariance, so a large part of what §6.6 reports is a between-rollout effect. That reinterprets the within-step control: it is a mean of 368 between-trajectory correlations, which is why it reads +0.739 against the pooled +0.605 rather than below it. The decisive statistic is the double-demeaned one, and it survives: +0.419 [+0.318, +0.576] at n_independent = 20, against the rule's pre-committed threshold that the interval exclude zero and a minimum detectable effect of 0.183 from the bootstrap's standard error; a dilution study missed an effect of 0.086 and caught one of 0.172. **The rule returns DISAGREEMENT CARRIES WITHIN-ROLLOUT INFORMATION**: with both the rollout and the depth held constant, disagreement still tracks error rather than merely reporting which episode is hard.

Two qualifications go with that. The within-rollout effect is **materially smaller than the pooled figure**, +0.419 against +0.605, so a practitioner should expect disagreement to separate *rollouts* better than it separates *moments within a rollout*. And it is **not established at short horizon**: r_dd's interval excludes zero at h=100, h=128, h=368 and spans zero at h=8 and h=32, where too few steps exist to demean against. At h = 100 it is established, at +0.385 [+0.283, +0.701].

The h = 1 figure survives the same test, but it is not what it looked like. At h = 1 the panel has one column, so +0.994 is a correlation over 20 *trajectory-level* points and nothing within a rollout is being tested. Disagreement correlates +0.006 with commanded speed and +0.027 with per-episode difficulty, and partialling both out of the disagreement–error correlation leaves +0.995: it does not move, so the figure is real and not a difficulty artifact. It is nevertheless a statement about **ranking whole rollouts at one step ahead**, on 20 points, and §9 states it that way.

Does it hold on a model we trained? Everything else in §6.6 is measured on the released checkpoint, because our main arms run at ensemble size 1 where the epistemic term is identically zero. We therefore trained three Arm A arms at **ensemble size 5**, identical in every other setting, under a rule committed to git before the runs existed (rule M-43, Appendix E). It asked that disagreement lead the index at every horizon, and that the paired difference exclude zero at a majority of them.

It returns DOES NOT GENERALISE. The first condition passes completely: disagreement leads the index in **12 of 12** seed-horizon cells, every paired estimate positive, +0.204 to +0.545. The second fails: the paired difference excludes zero at 1 of 4 horizons, not a majority. We report the verdict the rule returns and do not rewrite the rule, nor its denominator: it was committed over 4 horizons, and adding h = 100 after the fact would change what "a majority" means in a discharged rule. §6.6's released-checkpoint table follows the six-horizon grid, because no pre-registration is stated over it, so the two counts are deliberately different.

What separates the two conditions is sample size, and we measured that rather than asserting it. Our own arms can only be scored out-of-sample on the held-out pair, n_independent = 4, where the released checkpoint's finding used 20. Subsampling four trajectories at a time from a twenty-trajectory pool, the rule's criterion fires on 75% of draws on average and on only 24% at h=8. That estimate is an **upper bound**, because the pool it subsamples is in-sample for these arms, where the effect is +0.425 to +0.790 against +0.204 to +0.545 on the held-out pair. So the rule was under-powered at the sample size it faced, decisively at one horizon. We do not claim it could not have passed, only that it was committed without checking what it could detect: a failure of ours, the same one the ledger records as M-24, a rule anchored without regard to the regime it would be applied in.

Reported as a companion and not as a discharge: on all ten episodes (n_independent = 20 400-step trajectories, **in-sample** for these arms, which trained on eight of them) the same measurement excludes zero at 4

of 4 horizons and would have satisfied both conditions. It cannot discharge the rule, which is stated over the out-of-sample arena; we record it only so the comparison with the released checkpoint's 20 is like for like.

Why the within-step control is undefined for the forecast index. The within-step control holds the forecast step fixed and correlates across trajectories, so it annihilates any quantity that is constant across trajectories at a fixed step — which is the forecast index, whose within-step correlation is therefore undefined rather than zero. It does **not** annihilate a per-trajectory scalar like `entry-res`, which varies across exactly the axis the control varies over. Rule M-51 said the opposite, and `M-54` records the correction.

O Rule M-44 in full: an ensemble that shares nothing

§6.4 establishes the topology as a fact and the mechanism as a hypothesis. Rule M-44 tests the hypothesis, under a rule committed to git before any of the artifacts below existed (rule M-44, Appendix E), with a power check estimating what it could detect at the sample size it would face.

The contrast, and why it is affordable. Training 5 genuinely independent models from scratch costs about 4.8 h of wall clock on two cores at these runs' iteration count, against the 49.8 h Appendix B gives for the 33 runs of §5–§7 it times (§5.2's sweep and §5.3's baselines are counted apart). Arm A at ensemble size 1 already existed at seeds 0, 1 and 2; we added two more at about 0.9 h each, 1.7 h in total, and scored the 5 together as an ensemble **at evaluation time**. There is no new training code and no new architecture, and the disagreement across 5 independently initialised *full models* is exactly the contrast §6.4 asks for. The rollout protocol mirrors the shared-trunk one in every respect except the one under test: each member sees the same input state, each keeps **its own** recurrent hidden state, and the ensemble mean is fed back to all of them.

The rule returns MECHANISM SUPPORTED. All 6 of its 6 conditions hold, against every one of the 3 shared-trunk seeds. At h = 100 the independent ensemble's overconfidence factor is 0.485–0.503× the shared-trunk arms', a **2.03**× improvement against a pre-registered minimum detectable effect of 1.45×, and its $\pm 1\sigma$ coverage is 6.69 to 7.33 points higher, a mean of +7.12 against an MDE of 2.26. Every paired interval excludes zero. **Members that share a feature extractor do produce a smaller spread, and the effect is large enough to matter.**

Where the improvement comes from, which is not all one thing. The overconfidence factor is error over σ, so it improves if σ grows *or* if error shrinks, and only the first is the trunk-sharing mechanism; five independent models also denoise better than five heads on one trunk, an ordinary ensembling effect. Splitting the improvement into its two multiplicative parts:

h	σ larger by	error smaller by	total	share from σ	share from accuracy
1	1.56×	0.97×	1.52×	106%	-6%
8	1.81×	1.02×	1.84×	97%	3%
32	1.68×	1.08×	1.83×	87%	13%
100	**1.65**×	1.23×	2.03×	**71%**	29%
128	1.62×	1.29×	2.09×	65%	35%
368	1.49×	1.70×	2.53×	43%	57%

On the held-out pair's 4 independent trajectories, independent against shared-trunk ensembles, all at 2,500 iterations. Shares are of the log improvement, so they add to 100%. A share above 100% at h=1 means the independent ensemble is very slightly the ***worse*** *predictor there and the σ gain more than covers it.*

The reading, at the horizon the rule is stated over. σ is larger at every horizon, by 1.49× at its weakest (h = 368) and 1.81× at its strongest (h = 8), the direction trunk-sharing predicts, and at h = 100 it is 1.65×, **71%** of the improvement. So the mechanism is supported, and it is the larger part of the effect where the method operates. At the open-loop diagnostic horizon of h = 368 the split reverses: 57% of the improvement there is the ensemble simply predicting better, so a reader who takes the 2.53× figure at that horizon as a measure of the architectural effect would overstate it.

What this does and does not license. It licenses saying that **the released ensemble's disagreement understates epistemic uncertainty partly because its members are not independent models**, with a measured size at the horizon that matters. It does not license attributing the whole gap to trunk-sharing: independently seeded runs differ in *both* initialisation and data ordering (§11), so this comparison **bounds** the architectural effect rather than isolating it, and the bound is generous to the mechanism by construction.

And it does not repair the interval. The independent ensemble is 5.2× overconfident at h = 100 with 15.31% coverage where a calibrated Gaussian gives 68.27%. Better by a factor of 2.03, and still not an interval. Building the ensemble properly is worth doing and it is not sufficient; §6.7's per-horizon multiplier remains the only correction in this paper whose estimates all land within the band, though only on the released checkpoint and no single cell is resolvable at this arena. On Arm A, whose model never saw the test episodes, it manages 17 of 36 epistemic cells (§6.7).

P Rule M-68 in full: both fixes on the same models

§6.8 changes the **topology** and holds the objective at `mse`; §6.3's arms change the **objective** and hold the topology at ensemble size 1, where disagreement is zero by construction. Neither answers the practitioner's question of what the two give together, and adding two effects never measured on the same model is the arithmetic this paper criticises elsewhere. Rule M-68 runs the combination, under a rule committed to git before either new member existed (rule M-68, Appendix E), with a minimum detectable effect estimated in advance as well.

The arm. 5 independently initialised full models, sharing no trunk and no hidden state, trained under `gaussian_nll`, every setting identical to §6.8's arm except the loss type. Three of the five, seeds 0, 1 and 2, already existed under that objective; two more were trained, and the 5 are scored together as an ensemble at evaluation time under §6.8's rollout protocol, on the same held-out arena of non-overlapping 400-step trajectories at n_independent = 4, over the same six horizons.

Which σ the coverage is against. The combined arm is the first arm in this paper carrying **both** an aleatoric head that has not collapsed to zero and an across-member epistemic spread. The figures below are the **epistemic** one, the standard deviation across the five members' mean predictions, taken per state dimension, rather than the aleatoric head's output: the quantity the rule names and §6.8's table reports, so the two tables are comparable line for line. The aleatoric head is reported in §6.3 and does not enter here.

The rule returns THE COMBINATION IMPROVES CALIBRATION, branch 2 of the four it names. All 5 of its 5 conditions hold, against every one of the 3 shared-trunk seeds. At h = 100 the combined arm's overconfidence factor is 0.506–0.525× the shared-trunk arms', a **1.94**× improvement against a minimum detectable effect of 1.218× fixed in advance (results/p3_combined_arm_power.json, binding_mde at h = 100 (largest of the four calibrations)); its $\pm 1\sigma$ coverage is 6.92 to 7.56 points higher, a mean of +7.34 against an MDE of 2.50 points; and every paired interval excludes zero.

Condition (e), and how we read it. The rule's fifth condition asks that both statistics move in the improving direction at at least four of the six horizons, "against every shared-trunk seed", and the rule states globally that a condition holds only if it holds against all three. We applied the strict reading: a horizon counts only when **both** statistics improve there against **all three** seeds. It holds at 6 of 6 horizons, so the looser per-seed reading would not have changed the verdict; we record which we applied because the rule does not spell the difference out.

h	σ larger by	error smaller by	total	share from σ	share from accuracy
1	1.51×	1.00×	1.51×	101%	-1%
8	1.75×	1.05×	1.83×	93%	7%
32	1.63×	1.11×	1.81×	82%	18%
100	**1.55**×	1.25×	1.94×	**66%**	34%
128	1.51×	1.32×	2.00×	60%	40%
368	1.35×	1.77×	2.39×	34%	66%

On the held-out pair, n_independent = 4 400-step trajectories, every model at 2,500 iterations: the same σ-versus-accuracy split Appendix O uses for rule M-44, so the two arms' improvements can be compared part by part. Shares are of the log improvement, so the two columns add to one.

The objective's own contribution is below what this design can resolve. The rule scores the combined arm against the **shared-trunk** arms, so its verdict measures the two fixes together. The comparison that isolates the objective is against §6.8's independent arm, which differs from this one in the loss type and nothing else, from the same script on the same trajectories. At h = 100 switching `mse` for `gaussian_nll` multiplies the overconfidence factor by **1.044**× (95% interval from the same cluster bootstrap over whole trajectories, [1.029, 1.054]), the wrong direction, slightly; moves $\pm 1\sigma$ coverage by +0.22 points ([-0.31, +1.09], which spans zero); and multiplies the epistemic σ by 0.940×. Both effects sit under the minimum detectable effect fixed in advance, 1.218× on the factor and 2.50 points on coverage (results/p3_combined_arm_power.json, binding_mde at h = 100 (largest of the four calibrations)), so although the factor's interval lies wholly on the worse side of no change, the objective's separate effect is **below the size this design was built to resolve at n_independent = 4**, and the point estimate points away from an improvement. It does not establish that the objective contributes nothing: reading a null out of an effect smaller than its own floor is the error M-24 and M-43 record, and the rule's fourth branch exists to say underpowered rather than null. What the arm does settle is the combination, which is better calibrated than the shared-trunk arms by essentially the amount §6.8 measured for independence alone (2.03× there, 1.94× here). **The two fixes do not add. A reader who added them would have been wrong, which is why the arm was run.** That the objective's separate effect is at most small is not a surprise once stated: the objective governs the **aleatoric** head, and the epistemic term is a spread across members that the loss never sees.

And it still does not repair the interval. The combined arm is 5.4× overconfident at h = 100 with 15.53% coverage where a calibrated Gaussian gives 68.27%. Better than the shared-trunk arms, no better than independence alone, and still not an interval. §6.7's per-horizon multiplier remains the only correction in this paper whose estimates all land within the band, though only on the released checkpoint and no single cell is resolvable at this arena. On Arm A, whose model never saw the test episodes, it manages 17 of 36 epistemic cells (§6.7).

What the arm does not separate. It differs from the shared-trunk arms on three axes at once: trunk sharing, the objective, and capacity (a factor of 3.49 in state-pathway parameters, as §6.8's contrast also does; §11), and independently seeded runs also differ in data ordering. The rule said so in advance: it bounds the combination and attributes nothing. The one further comparison here, the `mse`/`gaussian_nll` pair above, which holds every other axis fixed, sits outside the rule's governing statistics and under the design's own minimum detectable effect, so it bounds the objective's separate contribution rather than measuring it.

Q What the spliced windows cost: the contaminated arm and the duplication control (§7.4)

The design. We trained a contaminated arm on 7,882 windows, the clean 7,687 plus 195 splices, and, because that confounds *content* with *count*, a duplication control adding the same 195 windows as exact copies of windows already present.

The arm's contamination rate is 2.47%, against the reference pipeline's 3.53%. It is deliberately lower: we splice only the 5 boundaries whose *both* sides are training episodes, because 4 of the 9 between the ten full episodes put held-out rows into training, a leakage problem rather than a physics one that would have invalidated our own comparison. So this experiment measures the cost of training on physically impossible transitions, not the reference's full exposure.

Training loss over the final 250 iterations: duplication costs 0.90%, splicing costs 21.57%. The bootstrap interval on duplicated − clean is [-0.0061, 0.0215], including zero, so the rise is caused by splice content, not by dataset size.

In rollout, across 32 cells (the held-out pair and the training episodes, 200 and 400-step trajectories, the 500 and 2,500-iteration checkpoints, n_independent 4 to 39, at two horizons on two metrics), contamination hurts in **0** of 32 and helps in 9 (Figure 6a). The control is inert, differing from clean in 2 cells.

So "costs nothing" is too strong. Splicing raises training loss by 21.57% against duplication's 0.90%, and improves rollout in 9 of 32 cells: measured effects in opposite directions, the signature of regularisation. The spliced windows contain transitions the model cannot fit, it fits the rest less tightly as a result, and it rolls out slightly better. The defensible statement is that **at this rate the splices do not harm rollout and appear to help slightly. The unmarked boundaries remain a real defect on leakage grounds; the physically-impossible-transition component costs nothing detectable at this rate.**

R The limitations in full (§11)

Effective sample size bounds every long-horizon claim. The out-of-sample arena has 4 independent 400-step trajectories (the n = 4 caveat of §3). That is the binding constraint on §5, and no amount of trajectory oversampling changes it. Nor can a larger dataset be generated here: the data generator is in neither pinned repository, and the one the lite release points to runs in a simulator this work did not have (Appendix C). The released checkpoint's lack of a held-out arena is therefore a constraint, not a choice.

Ensemble size: supported, not established. Our main arms run at ensemble size 1, where the epistemic term is identically zero, so the epistemic measurements were first made on the released checkpoint alone. The 3 ensemble-5 Arm A arms (§6.6) reproduce the *direction* of §6.6's finding in 12 of 12 seed-horizon cells and the *calibration* failure, 10.5× at h = 100, but the pre-registered rule governing the replication returns **DOES NOT GENERALISE**: its second condition needs the paired difference to exclude zero at a majority of horizons, and it does at 1 of 4. Our arms have a held-out arena of only 4 independent trajectories, and the rule was written without checking what it could detect there. **So §6.6's finding is established on the released checkpoint and supported but not established on a model we trained.**

One dataset, one gait, one terrain. All commands are drawn from one bounded box and the gait is a single trot throughout. "Generalisation" here means across velocity commands, not across gaits or terrain.

The per-horizon recalibration is fitted and tested on two episodes only. §6.7's remedy puts every released-checkpoint estimate within the band across the two held-out episodes in both directions, though at this n no single cell is resolvable, and those cells are unseen by the multiplier only, because the checkpoint trained on both episodes. On Arm A, whose model never saw them, the same recipe manages 17 of 36 epistemic and 10 of 36 aleatoric cells, and two episodes is not a demonstration that the multipliers transfer to a new robot, gait or terrain. Treat the lookup table as a recipe to refit, not as constants to copy.

Two secondary analyses rest on a single training seed, the long-horizon trend fit and the per-dimension matched comparison, both on seed 1 alone, as their artifacts record. The headline A/B verdict rests on one seed too, seed 1, the one its rule ran on; the magnitudes beside it are three-seed means with per-seed values (§5).

The ranking claim is not established as needing an ensemble. Under a rule committed before the comparison (rule M-51, Appendix E), the model's own predicted state change, a subtraction needing no ensemble, ranks error at +0.4697 against disagreement's +0.6053, and the margin is smaller than this sample can resolve (§6.6). Disagreement retains +0.5430 with that baseline partialled out, so what is open is whether its increment is worth five models. **If the observed margin is the true one, settling it needs 25 independent 400-step trajectories, where all ten episodes provide 20**, 1.25× the present sample, by the rule's own construction applied to the step-size margin's standard error (`results/q2_free_baseline_power.json`). That is a required-sample-size estimate under an assumed effect: it takes the true margin to be the observed +0.1357 and new trajectories to vary as these do, and if the true margin is smaller the requirement rises as its inverse square, to 45 at +0.1000. The margin is the only test left to pass, since the partial already clears its threshold, +0.5430 against 0.1131. And it is closer to resolving than §6.6's threshold of 0.2891 suggests: the rule had to fix that threshold before this baseline existed, from the forecast-index margin, whose standard error is 1.94× the step-size margin's. The observed margin is 91.1% of what its own statistic resolves at this sample and 46.9% of what the rule's threshold

demands, and re-run exactly as pre-registered that protocol would need 91 trajectories. §6.6's verdict stands on either reading, because the margin is below both.

We did not measure what the miscalibration costs. The penalty the follow-up applies is miscalibrated as a scale, 33.4× overconfident at h = 100, the horizon its own imagination rollouts run to, but the method's only use of that quantity is to shape policy learning, and we did not train a policy. A miscalibrated scale that enters as a relative penalty across candidate actions may cost little or a great deal; our measurements cannot tell which. **The finding bounds what the quantity reports, not what it costs**, and the ratio is not a measure of harm. Other sections refer back to this as the policy caveat of §11.

A proxy that needs no policy finds no reordering, and is worth only what its bound allows. Within one horizon the per-horizon correction multiplies the penalty by a positive constant, which cannot change any ranking, so a rule committed before the statistic was computed (rule M-70, Appendix E) compares instead the penalty accumulated along each whole rollout, before and after correction. The penalty is the released checkpoint's own ensemble disagreement, on the 4 independent trajectories of the held-out pair, which are unseen by the multipliers applied to them but not by the checkpoint (§3). Of the 6 pairs, 0 change order and the rule returns **DOES NOT REORDER**: the correction leaves every pairwise ordering of accumulated penalty unchanged, so whatever it changes downstream must act through the penalty's magnitude rather than through which rollout is penalised more. The multipliers differ by 9.31× across horizons, so the design could have produced reordering (`results/q3_penalty_reordering.json`). Three things limit what that shows. **It is the ordering of the penalty component alone, not of the penalised return**: this work has no reward function and no tuned penalty weight, so the result bounds what the correction could do downstream and licenses no statement about a policy or about the size of any downstream effect. **The null is partly structural**, by a diagnostic computed after the verdict: the last horizon band, h = 368, holds 240 of the 368 steps and 81.35% of the corrected penalty, and the overall ordering is exactly that band's, so the per-horizon weights had little room to act. **And the rule's bootstrap interval corroborates nothing**: resampling trajectories creates no new pairs, so with none reordering no resample can return anything else, and the interval is degenerate by construction (`M-71`).

The per-dimension ordering tests are underpowered at every sample size we can reach. Once the coupling between state dimensions is respected (§6.5), the out-of-sample arena's 4 independent trajectories cannot reject at any effect size, and the larger arenas can and do not: over all ten episodes the smallest P in the family is 0.0037 against a threshold of 0.001667. Resolving it at h = 368 needs more episodes than the released dataset contains, not a better test. At h = 128 and below that is no longer true: a shorter unit gives 14 independent units at h = 100 where the 400-step unit gives 4 (Appendix K), and we did not rerun this permutation family there. This limits the *per-dimension* evidence only; the aggregate scalar the method applies is one test rather than forty-five coupled ones, and more strongly supported (§6.6).

No family-wide correction is applied across our own pre-registered rules. There are 23 of them with per-rule verdicts (Appendix E), each reported against the thresholds it was committed with. Pre-registration is what licenses that: each rule is a separate question committed before its data, not one search over many outcomes, and a rule that fails is reported as failing. A reader who prefers a corrected family threshold can apply it from that count.

The independent-ensemble comparison bounds the trunk-sharing effect rather than isolating it. §6.8's contrast trains five models at five seeds and scores them together. Independently seeded runs differ in **both** initialisation *and* data ordering, whereas the shared-trunk heads differ only in head initialisation. They also differ in **capacity**: the independent arm carries 3,570,820 state-pathway parameters against the shared-trunk arm's 1,024,132, a factor of 3.49, because each member brings its own trunk, and greater capacity can inflate σ as well as shrink error. **Capacity is controlled separately.** Rule M-49 (Appendix E), committed with its minimum detectable effect before any of its models existed, trains 5 independent members at `rnn_hidden_size` 124 against the released 256, giving 1,023,880 state-pathway parameters against the shared-trunk arm's 1,024,132, a ratio of 0.9998 where §6.8's contrast carried 3.49. **With capacity held fixed the independent ensemble is still better calibrated on every shared-trunk seed, every paired interval still excludes zero, and the coverage gain of +6.42 points still clears its own MDE.** The effect does not vanish when the confound is removed.

It does shrink, and by more than this design can resolve. The overconfidence improvement falls from 2.03× unmatched to **1.79× matched**, against an MDE of 2.00×, so the rule returns **UNDER-POWERED — favours the matched ensemble by less than the MDE**: 5 of its 6 conditions hold, and the ratio threshold is the one that does not. That is the third branch the rule names, and it names it because its MDE was almost exactly the size of the effect it re-tested, as rule M-49's own text said before the runs (ledger `M-49`). So trunk-sharing is **not** explained away by capacity: the effect points the same way on every pair, with every interval excluding zero. Whether capacity accounts for *any* of it this design does not answer: the point estimates fall by about 0.24, a difference no artifact here tests and far below the 2.00× this comparison can resolve, and closing it needs more independent trajectories than the dataset contains. The comparison still conflates trunk-sharing with data-order diversity, which the rule does not address. That asymmetry is generous to the mechanism: had the factor barely moved despite the handicap, architecture would not be the explanation; since it moved, the design flaw is identified but not attributed to trunk-sharing alone. Isolating it would need an ensemble that shares data ordering and not parameters, a different experiment, as rule M-44 states in its own text.

§6.4's mechanism is a structural fact plus a hypothesis. That the five members share a trunk, a hidden state and 89.15% of each member's parameters is measured; that this *causes* the epistemic miscalibration is the hypothesis, and only §6.8 bears on it.

The configuration and architecture verdicts hold at our budget and on our reading. §5.2 varies one factor at a time at 0.133% of the reference's data, so it cannot say what the original's budget would favour. §5.3's baselines are our reconstruction of a table that fixes only their shapes, tested on one robot where the original has several. Both sections state these limits beside their results.

Deliberately out of scope. No policy-learning result of either paper is reproduced or tested: there is no simulator, no RL loop, no ANYmal, and no policy is trained anywhere in this work. The sample-efficiency comparison (roughly 6M against 250M transitions) is not tested for the same reason, and nothing here uses a GPU. And **we did not test whether the $\sigma = 0$ optimum affects other descendants of the PETS parameterisation** (§2): the clamp is inherited line for line, while the objective and the tie between the bounds are this codebase's own, so the hypothesis is well-founded only for a descendant that makes the substitution and leaves nothing pushing its floor back up, and it is untested for any. We counted how often the substitution is made among the 10 repositories we examined (§2), which bears on how far the hypothesis reaches, but we tested the mechanism in none of them.

S The conclusion's discussion of the ranking, the scale and the per-dimension evidence (§12)

The more useful finding is asymmetric, and it cuts both ways. The scale failure is established and large. The scale may be repairable per horizon, but on a model that has not seen the test episodes the evidence is mixed. On the released checkpoint a per-horizon multiplier, fitted on one episode and scored on another, brings every coverage estimate within 10 points of nominal, though no single cell is resolvable at this arena and the cells are not independent trials (§6.7), where a global multiplier manages 2 of them. On Arm A, whose model never saw those episodes, its own multipliers manage 17 of 36 epistemic cells. And the ranking use the follow-up claims does survive a real test: against the forecast step index, a free baseline neither original paper ran, ensemble disagreement wins at every horizon and keeps +0.596 once the index is partialled out. Against a second free baseline it does less well: the model's own predicted step size ranks error at +0.4697 against disagreement's +0.6053, a margin this sample cannot resolve, so the verdict is SURVIVES entry-res ONLY (disagreement beats the one-step error before the window, but not the model's predicted step size). **The control this rests on removes trajectory difficulty rather than forecast depth**: with both the rollout and the depth held constant, disagreement still correlates +0.419 [+0.318, +0.576] with realised error (§6.6). That is smaller than the +0.605 pooled figure, and it is the one that means what a practitioner needs it to mean: not a re-encoding of the clock, and not merely a report of which episode is hard. It is the closest either original work comes to a claim this reproduction strengthens rather than qualifies, and even there the strengthening is of the ordering, not of the ensemble that produces it, since a free subtraction ranks nearly as well.

What does not survive is the per-dimension form of the ordering evidence. Three of the five σ estimates we measured order their own errors better than chance in direction: the epistemic term on every one of the 45 dimensions at h=368, and the faithful and teacher-forced arms. That count is a direction, not a tally of independent trials, since the dimensions are physically coupled (§6.5). The released checkpoint's *aleatoric* head does the opposite, ranking error inversely at h = 368 on every one of 45 dimensions over all ten episodes and at chance on the held-out pair alone, a dependence on arena that §6.5 sets out. The corrected arm sits at chance in both. Once the coupling is respected by permuting whole trajectories, no per-dimension count in this paper reaches significance after multiplicity correction; the independent-trials P-values an earlier draft carried were wrong by up to a factor of about 10^{13} and are withdrawn (`S-15`). Neither quantity yields a usable interval. Per dimension, uncertainty in this family of models should be read as a weak ordering at best, or fixed at the objective. As the scalar the method applies, ensemble disagreement gets the order right (§6.6) and the size wrong: it should not be read as a scale, and a ranking use deserves its own validation on the deployment distribution rather than trust inherited from here.

T Whether the per-horizon multipliers transfer between models (rule M-69, §6.7)

The fourth column: the table is a property of the model, not of the horizon. Everything else in §6.7 is established across *episodes*, on one model. Whether the same lookup table works on a *different* model is a separate claim, and rule M-69 (Appendix E) fixed what an answer would look like — criterion, arena, horizons and minimum detectable effect — before any cross-model multiplier was computed. Multipliers were fitted on Arm A at ensemble size 5 (3 seeds) and scored on the released checkpoint, then the reverse, over the same 4 held-out trajectories and 6 horizons. The statistic the rule governs on is the **paired** change in held-out coverage, coverage under the other model's multiplier minus coverage under the same model's, on the same trajectories, because this arena can resolve that (largest minimum detectable effect 5.73 points against the ±10-point band) and cannot resolve the absolute one. The verdict is **DOES NOT TRANSFER — A PROPERTY OF THE MODEL**: of 72 governing cells, 52 have a 95% interval on the paired change lying entirely outside the band, 3 entirely inside and 17 straddling an edge, and the largest paired change has magnitude 67.5 points. That is branch 1 of the rule's three. Per direction the verdict is the same: fitting on Arm A, **DOES NOT TRANSFER — A PROPERTY OF THE MODEL** (29 of 36 cells outside the band); fitting on the released checkpoint, **DOES NOT TRANSFER — A PROPERTY OF THE MODEL** (23 of 36). The plainest statement of the gap is the multipliers themselves: Arm A's are 0.0069× to 1.36× the released checkpoint's at the same horizon.

The *different model* column is the absolute test, and it is unpowered for the reason §6.7 gives, so a cell inside its band is not evidence the multiplier transferred. It is reported because dropping it would hide that this arena cannot resolve it; it is not the verdict and cannot move it. §6.7's caution applies here unchanged and harder: the 72 governing cells are not 72 independent tests, the 3 Arm A seeds share their training data and differ only in initialisation and ordering, and no P-value attaches to any count here. The two models also differ on several axes at once, so this bounds transfer between these two models rather than attributing it to any one difference.

U The settings sweep at equal training compute

Post hoc (ledger R-78; `results/mn_compute_matched.json`); it re-opens no rule, and rule M-74's verdict, NOT OPTIMAL AT OUR BUDGET, stands as a statement about the 2,500-iteration budget (§5.2). Each row sets one of the four configurations that beat the centre at 2,500 iterations against the centre trained to 2,500, 5,000 or 10,000 iterations: twelve rows of four readings each. A longer forecast costs more per iteration than the centre, a shorter history less, so the centre's longer runs ask whether a configuration's win survives giving the centre at least as much training computation.

configuration (cost per iteration ÷ the centre's)	centre trained to (iterations)	compute ratio (centre's total ÷ configuration's)	h = 100, held-out pair (4)	h = 100, in-sample (16)	h = 368, held-out pair (4)	h = 368, in-sample (16)
(2, 8) (0.26)	2,500	3.78	**-0.1274 [-0.2175, -0.0373]**	**-0.0219 [-0.0341, -0.0107]**	**-0.1139 [-0.2988, -0.0088]**	**-0.0219 [-0.0364, -0.0081]**
(2, 8) (0.26)	5,000	7.55	-0.0742 [-0.1586, +0.0101]	**+0.0619 [+0.0448, +0.0813]**	-0.0301 [-0.1976, +0.0711]	**+0.0927 [+0.0681, +0.1230]**
(2, 8) (0.26)	10,000	15.11	-0.0176 [-0.0921, +0.0475]	**+0.1085 [+0.0846, +0.1367]**	**+0.1135 [+0.0983, +0.1357]**	**+0.1403 [+0.1075, +0.1802]**
(8, 8) (0.40)	2,500	2.50	**-0.0755 [-0.1410, -0.0269]**	**-0.0142 [-0.0239, -0.0053]**	**-0.1004 [-0.2449, -0.0167]**	**-0.0208 [-0.0462, -0.0009]**
(8, 8) (0.40)	5,000	4.99	-0.0223 [-0.0782, +0.0244]	**+0.0696 [+0.0538, +0.0870]**	-0.0166 [-0.1434, +0.0719]	**+0.0937 [+0.0762, +0.1123]**
(8, 8) (0.40)	10,000	9.99	+0.0343 [-0.0198, +0.0719]	**+0.1162 [+0.0924, +0.1436]**	**+0.1270 [+0.0987, +0.1589]**	**+0.1414 [+0.1163, +0.1710]**
(32, 16) (1.35)	2,500	0.74	**-0.1254 [-0.2054, -0.0454]**	**-0.0438 [-0.0675, -0.0258]**	**-0.2509 [-0.5218, -0.0956]**	**-0.1003 [-0.1355, -0.0719]**
(32, 16) (1.35)	5,000	1.48	-0.0723 [-0.1466, +0.0020]	**+0.0400 [+0.0326, +0.0481]**	**-0.1671 [-0.4301, -0.0117]**	**+0.0142 [+0.0068, +0.0205]**
(32, 16) (1.35)	10,000	2.96	-0.0157 [-0.0707, +0.0393]	**+0.0866 [+0.0729, +0.1005]**	-0.0235 [-0.1158, +0.0388]	**+0.0618 [+0.0531, +0.0706]**
(32, 32) (1.81)	2,500	0.55	**-0.1450 [-0.2490, -0.0409]**	**-0.0384 [-0.0637, -0.0186]**	**-0.2869 [-0.5951, -0.1005]**	**-0.1062 [-0.1422, -0.0768]**
(32, 32) (1.81)	5,000	1.11	-0.0918 [-0.1901, +0.0065]	**+0.0454 [+0.0356, +0.0563]**	**-0.2031 [-0.5033, -0.0275]**	+0.0084 [-0.0006, +0.0161]
(32, 32) (1.81)	10,000	2.21	-0.0352 [-0.1142, +0.0438]	**+0.0920 [+0.0770, +0.1068]**	-0.0594 [-0.1903, +0.0340]	**+0.0560 [+0.0483, +0.0639]**

Arenas: the held-out pair (episodes 1 and 8, 4 non-overlapping 400-step trajectories, n_independent = 4) and the in-sample arena (the training episodes' 16 non-overlapping 400-step trajectories, n_independent = 16). Checkpoints: each configuration at 2,500 iterations, the centre at the iterations in the second column; three seeds each. Each cell is D, the configuration's relative-L1 minus the centre's, the mean over trajectories of the three-seed means, with its 95% interval: exact over all 256 ordered resamples on the held-out pair, 20,000 Monte Carlo resamples (seed 0) in-sample. Negative favours the configuration, so every reading here is signed as in §5.2's table; bold where the interval excludes zero. Cost per iteration is steady training time per iteration, timed without contention; the compute ratio is the centre's total training computation over the configuration's.

V Rule X2: whether our models respond to the action they are given

Rule X2 was committed and pushed before any of its readings existed (ledger M-84; Appendix E gives its lead time) and is discharged in M-85. It is exploratory and re-opens no rule. It asks whether our trained models condition their forecasts on the action they are given. Only the actions the forecast steps read are changed,

so the history and the recurrent state that meets the first forecast action are untouched. E is the change in relative-L1 error, cumulative over forecast steps 1..h, against the true actions: the ratio err(I) / err(true), less one, in percent; for our arms, the mean over three seeds of each seed's E. Its 95% interval resamples whole trajectories, one resample for all seeds and both interventions in each draw: exact over all 256 ordered resamples on the held-out pair, 20,000 Monte Carlo resamples (seed 0) otherwise. The interventions are the stale action (the released evaluation's offset), another trajectory's actions (a swap, each trajectory taking the next one's), each action dimension's mean over the model's training rows, and the true actions plus Gaussian noise at k times each dimension's training standard deviation (eight draws). **The readings** are Arm A's, under the swap at h = 8 on its in-sample arena: **RESPONDS TO THE ACTION** at 2,500 iterations (+227.8% [+131.6, +398.2]) and at 10,000 (+676.3% [+454.8, +1060.0]). Arm B and the released checkpoint get the same statistic, alongside and not as readings.

model, checkpoint	actions given	h = 1	h = 8	h = 32	h = 100
Arm A, 2,500	stale action	+27.8 [+15.6, +49.4]	+31.5 [+19.2, +53.0]	+53.8 [+34.6, +83.5]	+48.7 [+34.8, +65.7]
Arm A, 2,500	another trajectory's actions	+73.2 [+33.7, +162.7]	**+227.8 [+131.6, +398.2]**	+376.0 [+248.7, +578.8]	+349.3 [+264.7, +454.0]
Arm A, 2,500	training mean	+49.6 [+25.5, +101.3]	+169.5 [+104.6, +278.0]	+309.6 [+215.5, +450.5]	+300.5 [+234.8, +380.6]
Arm A, 2,500	noise, k = 0.1	+0.5 [+0.2, +1.1]	+0.6 [+0.4, +1.1]	+1.3 [+0.7, +2.2]	+1.3 [+0.8, +1.8]
Arm A, 2,500	noise, k = 0.5	+8.2 [+3.9, +17.9]	+12.3 [+7.1, +21.6]	+24.7 [+14.4, +41.4]	+23.6 [+16.6, +32.3]
Arm A, 10,000	stale action	+124.6 [+75.8, +201.9]	+161.8 [+121.3, +231.1]	+205.9 [+156.7, +275.9]	+209.3 [+167.3, +257.6]
Arm A, 10,000	another trajectory's actions	+278.9 [+145.8, +548.3]	**+676.3 [+454.8, +1060.0]**	+986.6 [+723.5, +1381.1]	+1081.0 [+896.2, +1301.0]
Arm A, 10,000	training mean	+207.1 [+114.5, +385.7]	+531.7 [+380.0, +781.9]	+841.6 [+657.6, +1100.7]	+899.9 [+761.2, +1058.4]
Arm A, 10,000	noise, k = 0.1	+3.9 [+1.7, +8.6]	+6.0 [+3.3, +10.5]	+10.4 [+6.6, +16.1]	+11.4 [+8.6, +14.8]
Arm A, 10,000	noise, k = 0.5	+53.3 [+27.2, +106.9]	+74.8 [+47.5, +122.0]	+126.3 [+87.8, +181.8]	+141.2 [+115.1, +172.0]
Arm B, 2,500 (alongside)	stale action	+18.7 [+8.9, +36.1]	+15.9 [+7.9, +29.5]	+20.3 [+10.3, +36.8]	+9.3 [+5.1, +14.9]
Arm B, 2,500 (alongside)	another trajectory's actions	+63.6 [+31.1, +131.4]	+128.8 [+67.5, +241.8]	+173.4 [+101.6, +289.0]	+111.0 [+78.3, +155.4]
Arm B, 2,500 (alongside)	training mean	+41.4 [+22.7, +77.7]	+94.7 [+54.1, +164.4]	+145.9 [+89.1, +233.8]	+149.3 [+113.0, +195.4]
Arm B, 2,500 (alongside)	noise, k = 0.1	+0.4 [+0.1, +0.8]	+0.3 [+0.1, +0.6]	+0.5 [+0.3, +1.0]	+0.4 [-0.2, +1.2]
Arm B, 2,500 (alongside)	noise, k = 0.5	+7.2 [+3.7, +14.5]	+5.9 [+3.2, +11.4]	+9.8 [+5.2, +17.7]	+7.6 [+4.5, +11.5]
Arm B, 10,000 (alongside)	stale action	+75.1 [+44.2, +118.9]	+68.6 [+45.9, +105.2]	+60.0 [+39.9, +98.7]	+32.5 [+20.1, +50.3]
Arm B, 10,000 (alongside)	another trajectory's actions	+239.5 [+133.3, +439.8]	+358.2 [+220.9, +618.1]	+319.0 [+189.0, +556.8]	+218.4 [+154.0, +311.4]
Arm B, 10,000 (alongside)	training mean	+161.1 [+99.6, +272.2]	+295.7 [+193.3, +477.5]	+290.9 [+178.3, +485.9]	+242.2 [+172.3, +338.6]
Arm B, 10,000 (alongside)	noise, k = 0.1	+2.6 [+1.3, +4.9]	+2.1 [+1.0, +4.3]	+2.5 [+1.1, +5.2]	+2.3 [+0.1, +4.7]
Arm B, 10,000 (alongside)	noise, k = 0.5	+37.3 [+20.3, +69.3]	+30.8 [+18.0, +55.9]	+36.9 [+19.7, +69.5]	+34.7 [+21.4, +54.5]
released checkpoint (alongside)	stale action	+55.0 [+26.3, +92.7]	+54.4 [+37.5, +82.3]	+37.4 [+21.3, +65.4]	+8.2 [-10.4, +24.9]
released checkpoint (alongside)	another trajectory's actions	+515.3 [+295.2, +883.1]	+500.9 [+314.6, +824.2]	+363.5 [+219.0, +607.8]	+111.0 [+59.4, +192.7]
released checkpoint (alongside)	training mean	+296.3 [+176.8, +487.4]	+334.9 [+215.9, +538.9]	+267.3 [+168.0, +436.2]	+73.1 [+29.7, +141.7]
released checkpoint (alongside)	noise, k = 0.1	+15.0 [+7.7, +27.1]	+14.7 [+8.0, +26.5]	+10.2 [+5.5, +18.2]	+2.6 [+0.5, +5.9]
released checkpoint (alongside)	noise, k = 0.5	+143.0 [+80.7, +245.2]	+147.1 [+86.3, +251.7]	+109.9 [+61.2, +193.0]	+22.8 [+3.0, +51.8]

Arena: each model's own training data. For Arm A and Arm B, the in-sample arena: the training episodes' 16 non-overlapping 400-step trajectories, n_independent = 16. For the released checkpoint, all ten episodes: 20 trajectories, n_independent = 20. Checkpoints: Arm A and Arm B at 2,500 and 10,000 iterations, three seeds each; the released checkpoint. Each cell is E in percent with its 95% interval; bold, the two readings. Arm B and the released checkpoint are alongside, not readings.

model, checkpoint	actions given	h = 1	h = 8	h = 32	h = 100
Arm A, 2,500	stale action	+1.8 [-1.6, +11.6]	+3.6 [-0.6, +21.9]	+5.6 [-0.2, +30.8]	+6.9 [+2.6, +20.0]
Arm A, 2,500	another trajectory's actions	+63.3 [+43.4, +125.0]	+114.3 [+54.8, +351.6]	+141.9 [+59.0, +348.3]	+154.5 [+114.0, +265.0]
Arm A, 2,500	training mean	+44.4 [+36.6, +67.3]	+70.3 [+24.4, +208.9]	+88.5 [+31.5, +267.3]	+99.5 [+60.7, +218.1]
Arm A, 2,500	noise, k = 0.1	+0.6 [-0.2, +1.5]	+0.6 [+0.3, +1.5]	+0.7 [+0.2, +1.9]	+0.4 [+0.1, +1.0]
Arm A, 2,500	noise, k = 0.5	+9.3 [+5.3, +27.9]	+8.9 [+4.4, +28.3]	+11.5 [+5.1, +27.7]	+9.8 [+7.2, +17.6]
Arm A, 10,000	stale action	-1.0 [-6.1, +27.7]	+4.7 [-5.0, +73.6]	+13.9 [-2.4, +104.1]	+16.6 [+1.9, +72.6]
Arm A, 10,000	another trajectory's actions	+135.5 [+91.3, +448.8]	+179.4 [+78.7, +729.4]	+227.0 [+104.9, +660.2]	+242.7 [+176.8, +494.7]
Arm A, 10,000	training mean	+82.4 [+57.0, +264.6]	+115.7 [+39.5, +459.8]	+152.0 [+63.9, +537.0]	+151.9 [+83.7, +411.1]
Arm A, 10,000	noise, k = 0.1	+2.0 [+0.7, +10.2]	+1.1 [+0.2, +6.4]	+2.1 [+0.9, +6.1]	+1.9 [+1.2, +4.5]
Arm A, 10,000	noise, k = 0.5	+25.6 [+9.1, +121.4]	+22.6 [+9.5, +93.3]	+35.1 [+15.8, +91.3]	+29.2 [+20.1, +63.9]
Arm B, 2,500 (alongside)	stale action	+1.1 [-0.8, +8.3]	+0.3 [-0.8, +4.5]	+3.4 [+0.4, +14.6]	+0.7 [-1.2, +5.4]
Arm B, 2,500 (alongside)	another trajectory's actions	+67.4 [+30.6, +124.8]	+77.3 [+41.9, +187.0]	+64.6 [+31.4, +140.4]	+43.2 [+20.6, +131.4]
Arm B, 2,500 (alongside)	training mean	+34.8 [+20.8, +57.8]	+39.9 [+15.8, +100.8]	+43.4 [+14.5, +149.5]	+43.3 [+20.5, +127.4]
Arm B, 2,500 (alongside)	noise, k = 0.1	+0.6 [+0.2, +1.6]	+0.2 [-0.1, +0.8]	+0.0 [-0.5, +0.5]	-0.7 [-1.1, -0.2]
Arm B, 2,500 (alongside)	noise, k = 0.5	+8.4 [+4.8, +22.2]	+4.4 [+2.6, +11.9]	+6.2 [+0.3, +11.3]	-1.8 [-3.5, +4.5]
Arm B, 10,000 (alongside)	stale action	-1.0 [-2.7, +9.3]	+1.4 [-0.5, +12.4]	+6.1 [+2.5, +25.5]	+6.2 [+3.8, +17.3]
Arm B, 10,000 (alongside)	another trajectory's actions	+79.7 [+56.8, +218.8]	+113.2 [+36.4, +405.8]	+83.6 [+27.8, +247.9]	+61.4 [+13.1, +215.3]
Arm B, 10,000 (alongside)	training mean	+51.8 [+32.2, +134.0]	+65.1 [+11.7, +215.3]	+64.6 [+17.1, +239.8]	+51.1 [+13.5, +212.4]
Arm B, 10,000 (alongside)	noise, k = 0.1	+0.6 [-0.1, +4.3]	+0.4 [+0.0, +1.9]	-1.0 [-1.8, +0.1]	+2.0 [-2.0, +6.1]
Arm B, 10,000 (alongside)	noise, k = 0.5	+11.3 [+3.6, +56.9]	+9.5 [+4.1, +32.0]	+8.6 [+2.6, +25.6]	+5.3 [-2.8, +31.0]
released checkpoint (alongside)	stale action	+34.2 [+10.6, +75.0]	+37.2 [+7.9, +95.6]	+47.7 [+5.8, +131.8]	+22.5 [+6.3, +41.7]
released checkpoint (alongside)	another trajectory's actions	+940.5 [+707.2, +1183.7]	+1196.9 [+1042.5, +1459.9]	+807.4 [+696.6, +868.5]	+355.7 [+218.5, +487.3]
released checkpoint (alongside)	training mean	+600.7 [+447.0, +710.2]	+784.0 [+704.1, +924.9]	+530.9 [+420.1, +752.8]	+302.6 [+176.3, +452.5]
released checkpoint (alongside)	noise, k = 0.1	+44.0 [+30.6, +56.3]	+56.9 [+45.8, +76.5]	+29.7 [+25.1, +37.4]	+13.9 [+6.7, +23.0]
released checkpoint (alongside)	noise, k = 0.5	+377.7 [+299.5, +445.6]	+483.1 [+428.7, +586.0]	+295.7 [+265.0, +351.2]	+116.1 [+79.6, +182.6]

Arena: the held-out pair (episodes 1 and 8, 4 non-overlapping 400-step trajectories, n_independent = 4), out-of-sample for our arms and in-sample for the released checkpoint. Checkpoints as in the first table. No reading is made on this arena; every row is alongside.

model, checkpoint	arena	stale action	another trajectory's actions	training mean	noise, k = 0.1	noise, k = 0.5
Arm A, 2,500	own training data	0.139	0.525	0.417	0.014	0.071
Arm A, 2,500	held-out pair	0.060	0.623	0.431	0.026	0.128
Arm A, 10,000	own training data	0.200	0.654	0.528	0.023	0.110
Arm A, 10,000	held-out pair	0.084	0.801	0.584	0.044	0.215
Arm B, 2,500 (alongside)	own training data	0.109	0.386	0.303	0.011	0.055
Arm B, 2,500 (alongside)	held-out pair	0.040	0.481	0.336	0.021	0.098
Arm B, 10,000 (alongside)	own training data	0.161	0.545	0.465	0.018	0.088
Arm B, 10,000 (alongside)	held-out pair	0.052	0.629	0.451	0.030	0.140
released checkpoint (alongside)	own training data	0.204	0.930	0.666	0.066	0.327
released checkpoint (alongside)	held-out pair	0.078	1.042	0.712	0.093	0.461

Δ at h = 8, the relative-L1 of the forecast made with the changed actions measured against the forecast made with the true ones (`rollout_eval.relative_error`), which is zero for a model that ignores its actions. Arenas: each model's own training data (16 trajectories for our arms, 20 for the released checkpoint) and the held-out pair (4). Checkpoints: Arm A and Arm B at 2,500 and 10,000 iterations, three seeds each; the released checkpoint. Descriptive, no reading. How far the interventions move the actions: on the forecast rows, the action changes from one step to the next on 100% of steps on the in-sample arena, 100% on the held-out pair and 100% over all ten episodes. A one-step shift moves the action by 0.32, 0.27 and 0.31 of its spread (the mean absolute change from one step to the next over the mean absolute deviation from the action's mean), and the swap by 1.44, 1.43 and 1.42 (`results/action_sensitivity.json`).